TRATADO DE ALGORITMO Y DATOS II

Nahuel Villafañe

Estudiante avanzado de Física

@clasesconnahu

Universidad de Buenos Aires
Facultad de Ciencias Exactas y Naturales
2026

Figura 1: Enter Caption

Capítulo 1

Unidad I: Introducción al Análisis de Algoritmos

1.1. Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Definir formalmente un algoritmo y comprender su rol como tecnología fundamental en la resolución de problemas computacionales.
- Analizar rigurosamente el tiempo de ejecución de un algoritmo en el modelo RAM.
- Utilizar con soltura la notación asintótica (O, Ω, Θ, o, ω) para describir el crecimiento de funciones y comparar eficiencias.
- Diferenciar y aplicar el análisis del mejor caso, peor caso y caso promedio.
- Identificar y aplicar las técnicas básicas de análisis de algoritmos iterativos y recursivos, incluyendo la resolución de recurrencias mediante el Teorema Maestro.

1.2. ¿Qué es un Algoritmo? Fundamentos y Definiciones

Un algoritmo es un concepto central en computación. Informalmente, es un procedimiento computacional bien definido que toma un conjunto de valores como **entrada** y produce un conjunto de valores como **salida**. Es una secuencia de pasos computacionales que transforma la entrada en la salida.

1.2.1. Definición Formal

Podemos definir un algoritmo como una secuencia finita de instrucciones precisas, no ambiguas y ejecutables que, partiendo de un estado inicial (la entrada), culmina en un estado final (la salida). La palabra .ªlgoritmo”deriva del nombre del matemático persa Al-Khwārizmī.

1.2.2. Propiedades de un Algoritmo

Un algoritmo debe cumplir con las siguientes propiedades:

- **Entrada:** Recibe cero o más valores de entrada.
- **Salida:** Produce al menos un valor de salida.
- **Definitud:** Cada paso debe estar definido con precisión y sin ambigüedades.
- **Finitud:** El algoritmo debe terminar después de un número finito de pasos.
- **Efectividad:** Todas las operaciones deben ser lo suficientemente básicas como para poder ser realizadas, en principio, por un ser humano con lápiz y papel o por una computadora.

1.2.3. Ejemplo Clásico: El Problema de la Ordenación

El problema de ordenar una secuencia de números es un ejemplo paradigmático.

- **Entrada**: Una secuencia de n números $\langle a_1, a_2, \ldots, a_n \rangle$.
- **Salida**: Una permutación (reordenamiento) de la secuencia de entrada $\langle a'_1, a'_2, \ldots, a'_n \rangle$ tal que $a'_1 \leq a'_2 \leq \cdots \leq a'_n$.

Un algoritmo para este problema, como el de inserción (Insertion Sort), es un procedimiento que especifica cómo reordenar los números.

1.2.4. Algoritmos como Tecnología

La eficiencia de un algoritmo es una medida crucial. Consideremos dos algoritmos para el mismo problema: Insertion Sort ($\Theta(n^2)$) y Merge Sort ($\Theta(n \log n)$). Para $n = 10^6$:

- Insertion Sort: $\approx 10^{12}$ operaciones.
- Merge Sort: $\approx 20 \times 10^6$ operaciones.

La diferencia es abismal. Incluso si la computadora que ejecuta Insertion Sort es 100 veces más rápida, Merge Sort será más rápido para n grande. El *tiempo de ejecución* es nuestro recurso principal de análisis.

1.3. El Modelo RAM (Random-Access Machine)

Para analizar el tiempo de ejecución de manera rigurosa, necesitamos un modelo computacional. Usaremos el modelo RAM, que es una abstracción simple pero poderosa de una computadora real.

1.3.1. Supuestos del Modelo RAM

- **Instrucciones secuenciales:** Las instrucciones se ejecutan una tras otra, sin concurrencia.
- **Tiempo constante:** Las instrucciones comunes (aritméticas: suma, resta, multiplicación, división; de datos: carga, almacenamiento, copia; de control: saltos condicionales e incondicionales, llamadas a subrutinas) toman un tiempo constante.
- **Acceso aleatorio:** Se accede a la memoria en tiempo constante, independientemente de la dirección.
- **Tamaño de palabra:** Los números se representan con un tamaño de palabra fijo (e.g., $c \log n$ bits para un problema de tamaño n), lo que nos permite indexar arreglos y realizar operaciones aritméticas en tiempo constante.

Este modelo es lo suficientemente simple para el análisis y lo suficientemente realista para predecir el comportamiento en computadoras reales.

1.3.2. Limitaciones del Modelo

El modelo RAM ignora la jerarquía de memoria (cachés, memoria virtual) y la paralelización, aspectos importantes en la práctica pero que complican el análisis. Sin embargo, para la mayoría de los algoritmos, el modelo RAM es un excelente predictor de rendimiento.

1.4. Análisis del Tiempo de Ejecución

El tiempo de ejecución de un algoritmo depende de varios factores.

1.4.1. Tamaño de la Entrada

La medida más natural depende del problema.

- **Ordenación:** n, el número de elementos.
- **Multiplicación de enteros:** El número total de bits de los operandos.
- **Algoritmos de grafos:** $|V|$ (número de vértices) y $|E|$ (número de aristas).

En notación asintótica, usaremos n para denotar la medida del tamaño de la entrada, a menos que se especifique lo contrario.

1.4.2. Tiempo de Ejecución como Función

Definimos el tiempo de ejecución $T(n)$ como el número de **operaciones primitivas** o **pasos** ejecutados por el algoritmo para una entrada de tamaño n. Asumimos que cada línea de pseudocódigo toma un tiempo constante c_i, aunque diferentes líneas pueden tener diferentes constantes.

1.4.3. Mejor Caso, Peor Caso y Caso Promedio

Dado que el tiempo puede variar para diferentes entradas del mismo tamaño, analizamos:

- **Mejor caso:** El tiempo mínimo para cualquier entrada de tamaño n.
- **Peor caso:** El tiempo máximo para cualquier entrada de tamaño n. Es el más utilizado porque proporciona una garantía de que el algoritmo nunca tardará más.
- **Caso promedio:** El tiempo esperado, promediado sobre todas las entradas de tamaño n bajo una distribución de probabilidad dada.

1.4.4. Análisis Detallado del Insertion Sort

Consideremos el pseudocódigo de Insertion Sort. Analizaremos cada línea, asignando un costo c_i y contando su ejecución.

```
INSERTION-SORT(A)
1   for j = 2 to A.length
2       key = A[j]
3       // Inserta A[j] en la secuencia ordenada A[1..j-1]
4       i = j - 1
5       while i > 0 and A[i] > key
6           A[i+1] = A[i]
7           i = i - 1
8       A[i+1] = key
```

Listing 1.1: Insertion Sort (CLRS)

Línea	Costo	Vec. Ejec.
1	c_1	n
2	c_2	$n-1$
3	c_3	$n-1$ (comentario, 0)
4	c_4	$n-1$
5	c_5	$\sum_{j=2}^{n} t_j$
6	c_6	$\sum_{j=2}^{n}(t_j-1)$
7	c_7	$\sum_{j=2}^{n}(t_j-1)$
8	c_8	$n-1$

Donde t_j es el número de veces que se ejecuta la prueba del bucle `while` para un valor de j. El tiempo total es:

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^{n} t_j + c_6 \sum_{j=2}^{n}(t_j-1) + c_7 \sum_{j=2}^{n}(t_j-1) + c_8(n-1)$$

- **Mejor caso:** El arreglo ya está ordenado. Entonces $t_j = 1$ para cada j.

  $$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) = (c_1+c_2+c_4+c_5+c_8)n - (c_2+c_4+c_5+c_8)$$

  Es una función lineal: $T(n) = an + b = \Theta(n)$.

- **Peor caso:** El arreglo está en orden inverso. Entonces $t_j = j$ para cada j.

$$\sum_{j=2}^{n} t_j = \sum_{j=2}^{n} j = \frac{n(n+1)}{2} - 1$$

$$\sum_{j=2}^{n} (t_j - 1) = \sum_{j=2}^{n} (j-1) = \sum_{k=1}^{n-1} k = \frac{n(n-1)}{2}$$

Sustituyendo, obtenemos una función cuadrática: $T(n) = \frac{c_5+c_6+c_7}{2} n^2 + \cdots = \Theta(n^2)$.

1.5. Notación Asintótica: El Lenguaje del Crecimiento

Para simplificar, ignoramos las constantes y los términos de menor orden, enfocándonos en la **tasa de crecimiento** de la función. Esto nos permite comparar algoritmos independientemente de la máquina o de las constantes de implementación.

1.5.1. Notación Θ (Theta) - Cota Ajustada Asintóticamente

$$\Theta(g(n)) = \{f(n) : \exists c_1, c_2, n_0 > 0 \text{ tal que } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ para todo } n \geq n_0\}$$

$f(n) = \Theta(g(n))$ significa que $f(n)$ está .ªtrapada.ªntre dos múltiplos constantes de $g(n)$ para n suficientemente grande.

Ejemplo

Demostrar que $\frac{1}{2}n^2 - 3n = \Theta(n^2)$. Buscamos c_1, c_2, n_0 tales que $c_1 n^2 \leq \frac{1}{2}n^2 - 3n \leq c_2 n^2$ para $n \geq n_0$. Dividiendo por n^2: $c_1 \leq \frac{1}{2} - \frac{3}{n} \leq c_2$. Para $n \geq 1$, $\frac{1}{2} - \frac{3}{n} \leq \frac{1}{2}$, así que podemos elegir $c_2 = 1/2$. Para $n \geq 12$, $\frac{1}{2} - \frac{3}{n} \geq \frac{1}{4}$, así que podemos elegir $c_1 = 1/4$ y $n_0 = 12$.

1.5.2. Notación O (Big-O) - Cota Superior Asintótica

$$O(g(n)) = \{f(n) : \exists c, n_0 > 0 \text{ tal que } 0 \leq f(n) \leq cg(n) \text{ para todo } n \geq n_0\}$$

Proporciona un límite superior. Si $f(n) = O(g(n))$, entonces $g(n)$ crece al menos tan rápido como $f(n)$.

Ejemplo

$2n = O(n^2)$ porque para $c = 2, n_0 = 1$, $2n \leq 2n^2$. $2n = O(n)$ también es cierto.

1.5.3. Notación Ω (Omega) - Cota Inferior Asintótica

$$\Omega(g(n)) = \{f(n) : \exists c, n_0 > 0 \text{ tal que } 0 \leq cg(n) \leq f(n) \text{ para todo } n \geq n_0\}$$

Proporciona un límite inferior. Si $f(n) = \Omega(g(n))$, entonces $f(n)$ crece al menos tan rápido como $g(n)$.

Ejemplo

$\sqrt{n} = \Omega(\lg n)$ porque para $c = 1, n_0 = 16$, $\sqrt{16} = 4 \geq \lg 16 = 4$, y para $n > 16$ la desigualdad se mantiene.

1.5.4. Notación o (small-o) - Cota Superior no Ajustada

$$o(g(n)) = \{f(n) : \forall c > 0, \exists n_0 > 0 \text{ tal que } 0 \leq f(n) < cg(n) \text{ para todo } n \geq n_0\}$$

Indica que $f(n)$ es asintóticamente despreciable frente a $g(n)$. Equivalente a $\lim_{n\to\infty} f(n)/g(n) = 0$.

Ejemplo

$n^{1,9999} = o(n^2)$.

1.5.5. Notación ω (small-omega) - Cota Inferior no Ajustada

$$\omega(g(n)) = \{f(n) : \forall c > 0, \exists n_0 > 0 \text{ tal que } 0 \leq cg(n) < f(n) \text{ para todo } n \geq n_0\}$$

Indica que $f(n)$ es asintóticamente dominante frente a $g(n)$. Equivalente a $\lim_{n\to\infty} f(n)/g(n) = \infty$.

Ejemplo

$n^{2,0001} = \omega(n^2)$.

1.5.6. Teorema Fundamental

$$f(n) = \Theta(g(n)) \iff f(n) = O(g(n)) \text{ y } f(n) = \Omega(g(n))$$

Este teorema nos permite obtener cotas ajustadas combinando cotas superiores e inferiores.

1.5.7. Relaciones y Analogías

Podemos pensar en las notaciones asintóticas como análogas a las comparaciones entre números reales:

- $f(n) = O(g(n))$ como $a \leq b$
- $f(n) = \Omega(g(n))$ como $a \geq b$
- $f(n) = \Theta(g(n))$ como $a = b$
- $f(n) = o(g(n))$ como $a < b$
- $f(n) = \omega(g(n))$ como $a > b$

Advertencia: No todas las funciones son comparables. Por ejemplo, $n^{1+\sin n}$ oscila y no es comparable con n.

1.6. Propiedades y Manipulación de la Notación Asintótica

1.6.1. Transitividad

- $f(n) = \Theta(g(n))$ y $g(n) = \Theta(h(n)) \implies f(n) = \Theta(h(n))$
- Lo mismo aplica para O, Ω, o, ω.

1.6.2. Reflexividad

- $f(n) = \Theta(f(n))$, $f(n) = O(f(n))$, $f(n) = \Omega(f(n))$.

1.6.3. Simetría

- $f(n) = \Theta(g(n)) \iff g(n) = \Theta(f(n))$.

1.6.4. Simetría Transpuesta

- $f(n) = O(g(n)) \iff g(n) = \Omega(f(n))$.
- $f(n) = o(g(n)) \iff g(n) = \omega(f(n))$.

1.6.5. Operaciones con Notación Asintótica

Podemos usar la notación asintótica en expresiones algebraicas:

- $2n^2 + 3n + 1 = 2n^2 + \Theta(n)$ significa que existe una función $f(n) \in \Theta(n)$ tal que la expresión es igual a $2n^2 + f(n)$.
- $2n^2 + \Theta(n) = \Theta(n^2)$ significa que para cualquier $f(n) \in \Theta(n)$, existe $g(n) \in \Theta(n^2)$ tal que $2n^2 + f(n) = g(n)$.

1.7. Análisis de Algoritmos Recursivos y Recurrencias

Los algoritmos recursivos se analizan mediante **recurrencias**. Una recurrencia es una ecuación que define una función en términos de sus valores para entradas más pequeñas.

1.7.1. Ejemplo: Merge Sort

Para Merge Sort, el algoritmo divide el problema en dos mitades, las ordena recursivamente y luego las combina en tiempo lineal.

$$T(n) = \begin{cases} \Theta(1) & \text{si } n = 1 \\ 2T(n/2) + \Theta(n) & \text{si } n > 1 \end{cases}$$

1.7.2. Teorema Maestro

El Teorema Maestro proporciona una solución directa para recurrencias de la forma $T(n) = aT(n/b) + f(n)$, donde $a \geq 1$, $b > 1$ y $f(n)$ es una función asintóticamente positiva.

Teorema 1.1 (Teorema Maestro)**.** *Sean $a \geq 1$, $b > 1$ constantes, $f(n)$ una función y $T(n)$ definida sobre los enteros no negativos por:*

$$T(n) = aT(n/b) + f(n)$$

Entonces:

1. *Si $f(n) = O(n^{\log_b a - \epsilon})$ para alguna constante $\epsilon > 0$, entonces $T(n) = \Theta(n^{\log_b a})$.*
2. *Si $f(n) = \Theta(n^{\log_b a} \log^k n)$ para alguna constante $k \geq 0$, entonces $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$.*
3. *Si $f(n) = \Omega(n^{\log_b a + \epsilon})$ para alguna constante $\epsilon > 0$, y si $af(n/b) \leq cf(n)$ para alguna constante $c < 1$ y todo n suficientemente grande, entonces $T(n) = \Theta(f(n))$.*

Ejemplo

[Caso 1] $T(n) = 9T(n/3) + n$. Aquí $a = 9$, $b = 3$, $f(n) = n$, $\log_b a = \log_3 9 = 2$. $f(n) = n = O(n^{2-1})$ con $\epsilon = 1$. Por lo tanto, $T(n) = \Theta(n^2)$.

Ejemplo

[Caso 2] $T(n) = T(2n/3) + 1$. Aquí $a = 1$, $b = 3/2$, $f(n) = 1$, $\log_b a = \log_{3/2} 1 = 0$. $f(n) = 1 = \Theta(n^0 \log^0 n)$. Por lo tanto, $T(n) = \Theta(\log n)$.

Ejemplo

[Caso 3] $T(n) = 3T(n/4) + n \log n$. Aquí $a = 3$, $b = 4$, $\log_b a = \log_4 3 \approx 0{,}792$. $f(n) = n \log n = \Omega(n^{0{,}792+0{,}2})$. Verificamos la condición de regularidad: $af(n/b) = 3(n/4)\log(n/4) = \frac{3}{4}n(\log n - 2) \leq \frac{3}{4}n \log n$. Con $c = 3/4$, se cumple. Por lo tanto, $T(n) = \Theta(n \log n)$.

1.8. Ejercicios Resueltos (Ampliados)

Ejercicio

[Ordenamiento por Inserción con Inversiones] Demuestre que el número de comparaciones realizadas por Insertion Sort en el peor caso es $\frac{n(n-1)}{2}$ y que este es el número de inversiones en una permutación inversa.

Solución

En el peor caso, el arreglo está en orden inverso. Para cada j (de 2 a n), la clave se compara con todos los $j-1$ elementos anteriores. El número total de comparaciones es:

$$\sum_{j=2}^{n}(j-1) = \sum_{k=1}^{n-1} k = \frac{n(n-1)}{2}$$

Una **inversión** es un par (i, j) con $i < j$ y $A[i] > A[j]$. En una permutación inversa, cada par (i, j) es una inversión. Hay $\binom{n}{2} = \frac{n(n-1)}{2}$ inversiones. Por lo tanto, el número de comparaciones en el peor caso es exactamente el número de inversiones.

Ejercicio

[Análisis de un Bucle Triple] Analice el tiempo de ejecución del siguiente fragmento de código:

```
for i = 1 to n
    for j = 1 to i
        for k = 1 to j
            x = x + 1

```

Solución

El número de veces que se ejecuta la operación $x = x + 1$ es:

$$\sum_{i=1}^{n}\sum_{j=1}^{i}\sum_{k=1}^{j} 1 = \sum_{i=1}^{n}\sum_{j=1}^{i} j = \sum_{i=1}^{n}\frac{i(i+1)}{2} = \frac{1}{2}\left(\sum_{i=1}^{n} i^2 + \sum_{i=1}^{n} i\right)$$

Usando $\sum i = n(n+1)/2$ y $\sum i^2 = n(n+1)(2n+1)/6$:

$$= \frac{1}{2}\left(\frac{n(n+1)(2n+1)}{6} + \frac{n(n+1)}{2}\right) = \frac{n(n+1)}{4}\left(\frac{2n+1}{3} + 1\right)$$

$$= \frac{n(n+1)}{4} \cdot \frac{2n+4}{3} = \frac{n(n+1)(n+2)}{6} = \Theta(n^3)$$

Ejercicio

[Demostración con Definición de O] Demuestre formalmente que $3n^2 + 2n + 1 = O(n^2)$ usando la definición.

Solución

Debemos encontrar $c > 0$ y $n_0 > 0$ tales que $3n^2 + 2n + 1 \leq cn^2$ para todo $n \geq n_0$. Para $n \geq 1$, tenemos $2n \leq 2n^2$ y $1 \leq n^2$. Entonces:

$$3n^2 + 2n + 1 \leq 3n^2 + 2n^2 + n^2 = 6n^2$$

Por lo tanto, podemos elegir $c = 6$ y $n_0 = 1$.

Ejercicio

[Demostración con Definición de Ω] Demuestre formalmente que $3n^2 + 2n + 1 = \Omega(n^2)$ usando la definición.

Solución

Debemos encontrar $c > 0$ y $n_0 > 0$ tales que $3n^2 + 2n + 1 \geq cn^2$ para todo $n \geq n_0$. Para $n \geq 1$, tenemos $3n^2 + 2n + 1 \geq 3n^2$. Por lo tanto, podemos elegir $c = 3$ y $n_0 = 1$.

Ejercicio

[Demostración de Θ combinando O y Ω] Demuestre que $3n^2 + 2n + 1 = \Theta(n^2)$.

Solución

Por los dos ejercicios anteriores, tenemos $3n^2+2n+1 = O(n^2)$ y $3n^2+2n+1 = \Omega(n^2)$. Por el teorema fundamental, $3n^2 + 2n + 1 = \Theta(n^2)$.

Ejercicio

[Crecimiento de Logaritmos] Demuestre que $\log(n!) = \Theta(n \log n)$.

Solución

Cota inferior: $n! \geq (n/2)^{n/2}$. Tomando logaritmo:

$$\log(n!) \geq \frac{n}{2} \log\left(\frac{n}{2}\right) = \frac{n}{2}(\log n - 1) = \Omega(n \log n)$$

Cota superior: $n! \leq n^n$. Tomando logaritmo:

$$\log(n!) \leq n \log n = O(n \log n)$$

Por lo tanto, $\log(n!) = \Theta(n \log n)$.

Ejercicio

[Comparación de Funciones] Ordene las siguientes funciones por orden de crecimiento asintótico: $n^{3/2}$, $n \log n$, 2^n, $n!$, $\log n$, n, $n^{1/2}$.

Solución

Orden de menor a mayor crecimiento:

$$\log n, \quad n^{1/2}, \quad n, \quad n \log n, \quad n^{3/2}, \quad 2^n, \quad n!$$

- $\log n$ crece más lentamente que cualquier potencia positiva de n.
- $n^{1/2} = \sqrt{n}$ es menor que n.
- n es lineal.
- $n \log n$ es mayor que lineal pero menor que cualquier potencia n^c con $c > 1$.
- $n^{3/2}$ es una potencia > 1.
- 2^n es exponencial.
- $n!$ es mayor que 2^n.

Ejercicio

[Resolución de Recurrencia con Teorema Maestro] Resuelva $T(n) = 16T(n/4) + n^2$.

Solución

$a = 16$, $b = 4$, $\log_b a = \log_4 16 = 2$. $f(n) = n^2 = \Theta(n^2)$. Es el Caso 2 con $k = 0$. Por lo tanto:

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n) = \Theta(n^2 \log n)$$

Ejercicio

[Resolución de Recurrencia con Teorema Maestro] Resuelva $T(n) = 7T(n/2) + n^3$.

Solución

$a = 7$, $b = 2$, $\log_b a = \log_2 7 \approx 2{,}807$. $f(n) = n^3 = \Omega(n^{2{,}807+0{,}1})$ con $\epsilon = 0{,}1$. Verificamos la condición de regularidad:

$$af(n/b) = 7(n/2)^3 = 7n^3/8 = \frac{7}{8}n^3$$

Elegimos $c = 7/8 < 1$. Se cumple. Por lo tanto, $T(n) = \Theta(n^3)$.

Ejercicio

[Recurrencia que no aplica al Teorema Maestro] Resuelva $T(n) = 2T(n/2) + n \log n$.

Solución

Aquí $a = 2$, $b = 2$, $\log_b a = 1$. $f(n) = n \log n = \Theta(n \log^1 n)$. Esto parece el Caso 2 con $k = 1$, pero el Teorema Maestro requiere $f(n) = \Theta(n^{\log_b a} \log^k n)$. Aquí $n^{\log_b a} = n$. $f(n) = n \log n$ es exactamente de esa forma con $k = 1$. Por lo tanto:

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n) = \Theta(n \log^2 n)$$

Nota: Algunas formulaciones del Teorema Maestro incluyen este caso explícitamente.

Ejercicio

[Análisis de Búsqueda Binaria] Escriba la recurrencia para la búsqueda binaria y resuélvala.

Solución

La búsqueda binaria divide el problema en una mitad (o $n/2$) y realiza una comparación constante.

$$T(n) = T(n/2) + O(1)$$

Aplicando el Teorema Maestro: $a = 1$, $b = 2$, $\log_b a = 0$, $f(n) = O(1) = O(n^0)$. Es el Caso 2 con $k = 0$. Por lo tanto:

$$T(n) = \Theta(\log n)$$

Ejercicio

[Análisis de un Algoritmo de División Desigual] Resuelva $T(n) = T(n/3) + T(2n/3) + n$.

Solución

Este es un caso clásico de divide y vencerás con subproblemas de tamaño desigual. Podemos usar un árbol de recursión. La altura máxima es $h = \log_{3/2} n$ (cuando siempre tomamos el subproblema más grande). En cada nivel, la suma de los tamaños de los subproblemas es n, y el trabajo combinado es n. Por lo tanto, el costo total es $O(n \log n)$. Un análisis más cuidadoso muestra que $T(n) = \Theta(n \log n)$. El Teorema Maestro no aplica directamente porque los subproblemas no son del mismo tamaño.

Ejercicio

[Algoritmo de Strassen] El algoritmo de Strassen para multiplicación de matrices tiene recurrencia $T(n) = 7T(n/2) + O(n^2)$. Resuélvala.

Solución

$a = 7$, $b = 2$, $\log_b a = \log_2 7 \approx 2{,}807$. $f(n) = O(n^2) = O(n^{2{,}807-\epsilon})$ con $\epsilon = 0{,}807$. Es el Caso 1. Por lo tanto:

$$T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2{,}807})$$

Esto es mejor que el algoritmo clásico $O(n^3)$.

1.9. Ejercicios Propuestos (Ampliados)

1. Analice el tiempo de ejecución de un algoritmo que cuenta el número de pares (i, j) en un arreglo de tamaño n tales que $i < j$ y $A[i] > A[j]$ (inversiones).

2. Determine si las siguientes afirmaciones son verdaderas o falsas:

   a) $n^2 = O(n^3)$
   b) $n^2 = \Omega(n^3)$
   c) $n^2 = o(n^3)$
   d) $n^2 = \omega(n^3)$
   e) $n^2 = \Theta(n^3)$
   f) $2^{n+1} = \Theta(2^n)$
   g) $2^{2n} = \Theta(2^n)$
   h) $\log(n^2) = \Theta(\log n)$

3. Demuestre formalmente que $5n^3 + 10n^2 + 3n = \Theta(n^3)$.

4. Demuestre que $n^{1,5} = \omega(n \log n)$.

5. Resuelva las siguientes recurrencias usando el Teorema Maestro:

   a) $T(n) = 4T(n/2) + n$
   b) $T(n) = 4T(n/2) + n^2$
   c) $T(n) = 4T(n/2) + n^3$
   d) $T(n) = 2T(n/2) + n \log n$
   e) $T(n) = 3T(n/3) + n \log n$
   f) $T(n) = 2T(n/2) + \sqrt{n}$
   g) $T(n) = T(n/2) + n$

6. Implemente una versión iterativa de la búsqueda binaria y analice su tiempo de ejecución.

7. Demuestre que si $f(n) = O(g(n))$ entonces $f(n) + g(n) = \Theta(g(n))$.

8. Encuentre el número de operaciones (en términos de n) que realiza el siguiente fragmento de código:

```
for i = 1 to n
    for j = i to n
        for k = i to j
            print(i*j*k)

```

9. Ordene las siguientes funciones por orden de crecimiento asintótico: n^3, 2^n, $n \log^2 n$, $n^{0,5}$, $n^{2,5}$, $\log n$, $n!$, $n \log n$, n.

10. Resuelva $T(n) = T(n-1) + 1/n$. (Pista: ¿Qué serie conocida aparece?)

11. Un algoritmo tiene un tiempo de ejecución $T(n) = 3T(n/2) + n \log n$. ¿Qué nos dice el Teorema Maestro sobre su complejidad?

12. Explique por qué el análisis del peor caso es más utilizado que el análisis del mejor caso. Dé un ejemplo donde el mejor caso no sea representativo.

13. ¿Es correcto afirmar que $f(n) = O(g(n))$ implica $2^{f(n)} = O(2^{g(n)})$? Justifique.

14. Diseñe un algoritmo recursivo para encontrar el máximo de un arreglo. Escriba su recurrencia y resuélvala.

15. El algoritmo de multiplicación de Karatsuba tiene recurrencia $T(n) = 3T(n/2) + O(n)$. ¿Cuál es su complejidad asintótica?

Capítulo 2

Unidad II: Notación Asintótica: O, Ω, Θ, o y ω

2.1. Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender la necesidad de la notación asintótica para abstraer detalles de implementación y comparar algoritmos.
- Definir formalmente las notaciones O, Ω, Θ, o y ω.
- Demostrar relaciones asintóticas entre funciones utilizando las definiciones formales.
- Aplicar las propiedades de transitividad, reflexividad, simetría y simetría transpuesta.
- Manipular expresiones que contienen notación asintótica en ecuaciones.
- Clasificar funciones por su orden de crecimiento y comparar eficiencias de algoritmos.

2.2. Introducción: ¿Por qué necesitamos la notación asintótica?

Cuando analizamos un algoritmo, a menudo obtenemos expresiones exactas para el tiempo de ejecución, como:

$$T(n) = 3n^2 + 5n + 2$$

Sin embargo, esta expresión depende de constantes que varían según la máquina, el lenguaje de programación y la implementación. Lo que realmente nos interesa es cómo crece $T(n)$ a medida que n se hace grande. La **notación asintótica** nos permite:

- Ignorar constantes multiplicativas y términos de menor orden.
- Centrarnos en la **tasa de crecimiento** de la función.
- Comparar algoritmos independientemente de la plataforma.

Decimos que $3n^2 + 5n + 2$ çrece como"n^2, y lo escribimos como $3n^2 + 5n + 2 = \Theta(n^2)$.

2.3. Notación Θ (Theta): Cota Ajustada Asintóticamente

La notación Θ se utiliza para acotar una función tanto superior como inferiormente, dentro de un factor constante. Es la más fuerte de las notaciones porque establece que dos funciones tienen el mismo orden de crecimiento.

2.3.1. Definición Formal

$$\Theta(g(n)) = \{f(n) : \exists c_1, c_2, n_0 > 0 \text{ tal que } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ para todo } n \geq n_0\}$$

Se lee: "$f(n)$ es theta de $g(n)$". Intuitivamente, $f(n)$ está .ªtrapada.ªntre dos múltiplos constantes de $g(n)$ para n suficientemente grande.

Ejemplo

Demostrar que $\frac{1}{2}n^2 - 3n = \Theta(n^2)$.

- Buscamos c_1, c_2, n_0 tales que $c_1 n^2 \leq \frac{1}{2}n^2 - 3n \leq c_2 n^2$ para $n \geq n_0$.
- Dividiendo por n^2: $c_1 \leq \frac{1}{2} - \frac{3}{n} \leq c_2$.
- Para $n \geq 1$, $\frac{1}{2} - \frac{3}{n} \leq \frac{1}{2}$, así que elegimos $c_2 = \frac{1}{2}$.
- Para $n \geq 12$, $\frac{1}{2} - \frac{3}{n} \geq \frac{1}{4}$, así que elegimos $c_1 = \frac{1}{4}$ y $n_0 = 12$.

Por lo tanto, $\frac{1}{2}n^2 - 3n = \Theta(n^2)$.

2.3.2. Teorema Fundamental de la Notación Asintótica

Teorema 2.1. *$f(n) = \Theta(g(n))$ si y solo si $f(n) = O(g(n))$ y $f(n) = \Omega(g(n))$.*

Este teorema nos permite obtener cotas ajustadas combinando cotas superiores e inferiores.

2.4. Notación O (Big-O): Cota Superior Asintótica

La notación O proporciona un límite superior. Si $f(n) = O(g(n))$, entonces $g(n)$ crece al menos tan rápido como $f(n)$.

2.4.1. Definición Formal

$$O(g(n)) = \{f(n) : \exists c, n_0 > 0 \text{ tal que } 0 \leq f(n) \leq cg(n) \text{ para todo } n \geq n_0\}$$

Se lee: "$f(n)$ es O de $g(n)$". Indica que $f(n)$ no crece más rápido que $g(n)$.

Ejemplo

Demostrar que $3n^2 + 2n + 1 = O(n^2)$. Para $n \geq 1$, tenemos $2n \leq 2n^2$ y $1 \leq n^2$. Entonces:

$$3n^2 + 2n + 1 \leq 3n^2 + 2n^2 + n^2 = 6n^2$$

Con $c = 6$ y $n_0 = 1$, se cumple la definición.

Ejemplo

Demostrar que $2^n = O(3^n)$. Para $n \geq 0$, $2^n \leq 1 \cdot 3^n$, con $c = 1$, $n_0 = 0$. Por lo tanto, $2^n = O(3^n)$.

2.4.2. Observaciones sobre O

- O proporciona un límite superior, no necesariamente ajustado. Por ejemplo, $n = O(n^2)$ es cierto, aunque n crece más lentamente.
- Si $f(n) = \Theta(g(n))$, entonces $f(n) = O(g(n))$ y $f(n) = \Omega(g(n))$.

2.5. Notación Ω (Omega): Cota Inferior Asintótica

La notación Ω proporciona un límite inferior. Si $f(n) = \Omega(g(n))$, entonces $f(n)$ crece al menos tan rápido como $g(n)$.

2.5.1. Definición Formal

$$\Omega(g(n)) = \{f(n) : \exists c, n_0 > 0 \text{ tal que } 0 \leq cg(n) \leq f(n) \text{ para todo } n \geq n_0\}$$

Se lee: "$f(n)$ es omega de $g(n)$". Indica que $f(n)$ no crece más lentamente que $g(n)$.

Ejemplo

Demostrar que $3n^2 + 2n + 1 = \Omega(n^2)$. Para $n \geq 1$, $3n^2 + 2n + 1 \geq 3n^2$. Con $c = 3$ y $n_0 = 1$, se cumple.

Ejemplo

Demostrar que $\sqrt{n} = \Omega(\log n)$. Para $n \geq 16$, $\sqrt{n} \geq \log n$. Con $c = 1$ y $n_0 = 16$, se cumple.

2.6. Notación o (Small-o): Cota Superior no Ajustada

La notación o indica que $f(n)$ es asintóticamente despreciable frente a $g(n)$. Es una cota superior estricta.

2.6.1. Definición Formal

$$o(g(n)) = \{f(n) : \forall c > 0, \exists n_0 > 0 \text{ tal que } 0 \leq f(n) < cg(n) \text{ para todo } n \geq n_0\}$$

Se lee: "$f(n)$ es pequeña-o de $g(n)$". Equivalentemente, $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$.

Ejemplo

Demostrar que $n^{1,9999} = o(n^2)$.

$$\lim_{n\to\infty} \frac{n^{1,9999}}{n^2} = \lim_{n\to\infty} \frac{1}{n^{0,0001}} = 0$$

Por lo tanto, $n^{1,9999} = o(n^2)$.

Ejemplo

$n^2 \neq o(n^2)$ porque el límite es 1, no 0.

2.7. Notación ω (Small-omega): Cota Inferior no Ajustada

La notación ω indica que $f(n)$ es asintóticamente dominante frente a $g(n)$. Es una cota inferior estricta.

2.7.1. Definición Formal

$$\omega(g(n)) = \{f(n) : \forall c > 0, \exists n_0 > 0 \text{ tal que } 0 \leq cg(n) < f(n) \text{ para todo } n \geq n_0\}$$

Se lee: "$f(n)$ es pequeña-omega de $g(n)$". Equivalentemente, $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$.

Ejemplo

Demostrar que $n^{2,0001} = \omega(n^2)$.

$$\lim_{n\to\infty} \frac{n^{2,0001}}{n^2} = \lim_{n\to\infty} n^{0,0001} = \infty$$

Por lo tanto, $n^{2,0001} = \omega(n^2)$.

2.8. Propiedades de las Notaciones Asintóticas

2.8.1. Transitividad

- $f(n) = \Theta(g(n))$ y $g(n) = \Theta(h(n)) \implies f(n) = \Theta(h(n))$

- $f(n) = O(g(n))$ y $g(n) = O(h(n)) \implies f(n) = O(h(n))$
- $f(n) = \Omega(g(n))$ y $g(n) = \Omega(h(n)) \implies f(n) = \Omega(h(n))$
- $f(n) = o(g(n))$ y $g(n) = o(h(n)) \implies f(n) = o(h(n))$
- $f(n) = \omega(g(n))$ y $g(n) = \omega(h(n)) \implies f(n) = \omega(h(n))$

2.8.2. Reflexividad

- $f(n) = \Theta(f(n))$
- $f(n) = O(f(n))$
- $f(n) = \Omega(f(n))$

Nota: o y ω no son reflexivas porque $f(n) \neq o(f(n))$ (el límite sería 1, no 0).

2.8.3. Simetría

- $f(n) = \Theta(g(n)) \iff g(n) = \Theta(f(n))$

2.8.4. Simetría Transpuesta

- $f(n) = O(g(n)) \iff g(n) = \Omega(f(n))$
- $f(n) = o(g(n)) \iff g(n) = \omega(f(n))$

2.8.5. Relaciones de Comparación

Podemos establecer una analogía con la comparación de números reales:

Notación Asintótica	Comparación Numérica
$f(n) = O(g(n))$	$a \leq b$
$f(n) = \Omega(g(n))$	$a \geq b$
$f(n) = \Theta(g(n))$	$a = b$
$f(n) = o(g(n))$	$a < b$
$f(n) = \omega(g(n))$	$a > b$

Advertencia: No todas las funciones son comparables. Por ejemplo, $n^{1+\sin n}$ y n no se pueden comparar asintóticamente porque la razón oscila entre $n^0 = 1$ y n^2.

2.9. Manipulación de Expresiones con Notación Asintótica

2.9.1. Notación en Ecuaciones

Cuando la notación asintótica aparece en una expresión, representa una función anónima de esa clase.

- $2n^2 + 3n + 1 = 2n^2 + \Theta(n)$ significa que existe $f(n) \in \Theta(n)$ tal que $2n^2 + 3n + 1 = 2n^2 + f(n)$.
- $2n^2 + \Theta(n) = \Theta(n^2)$ significa que para cualquier $f(n) \in \Theta(n)$, existe $g(n) \in \Theta(n^2)$ tal que $2n^2 + f(n) = g(n)$.

2.9.2. Sumas y Productos

- $\sum_{i=1}^{n} O(i) = O(n^2)$ porque cada término es $O(i)$ y la suma es $O(\sum i) = O(n^2)$.
- $O(f(n)) \cdot O(g(n)) = O(f(n)g(n))$.

2.10. Clasificación de Funciones por Orden de Crecimiento

2.10.1. Funciones Polinomiales

Para $p(n) = a_d n^d + a_{d-1} n^{d-1} + \cdots + a_0$ con $a_d > 0$:

$$p(n) = \Theta(n^d)$$

2.10.2. Funciones Logarítmicas

Para cualquier $a > 0$, $\log^a n = o(n^c)$ para cualquier $c > 0$. El logaritmo crece más lentamente que cualquier potencia positiva de n.

2.10.3. Funciones Exponenciales

Para $a > 1$, $n^c = o(a^n)$ para cualquier c. La exponencial crece más rápido que cualquier polinomio.

2.10.4. Funciones Factoriales

$n! = \omega(2^n)$ y $\log(n!) = \Theta(n \log n)$.

2.10.5. Jerarquía Común de Funciones

Orden de menor a mayor crecimiento:

$$1, \log\log n, \log n, \sqrt{n}, n, n\log n, n^2, n^3, \ldots, 2^n, n!, 2^{n^2}$$

2.11. Ejercicios Resueltos (Ampliados)

Ejercicio

Demuestre que $n^2 + 100n = \Theta(n^2)$.

Solución

Buscamos c_1, c_2, n_0 tales que $c_1 n^2 \leq n^2 + 100n \leq c_2 n^2$.

- **Cota superior:** $n^2 + 100n \leq n^2 + n^2 = 2n^2$ para $n \geq 100$. Con $c_2 = 2$, $n_0 = 100$.
- **Cota inferior:** $n^2 + 100n \geq n^2$ para $n \geq 0$. Con $c_1 = 1$, $n_0 = 0$.

Por lo tanto, $n^2 + 100n = \Theta(n^2)$.

Ejercicio

Demuestre que $n^2 \neq O(n)$.

Solución

Supongamos que existe c, n_0 tal que $n^2 \leq cn$ para todo $n \geq n_0$. Esto implica $n \leq c$ para todo $n \geq n_0$, lo cual es falso porque n crece sin cota. Por lo tanto, $n^2 \neq O(n)$.

Ejercicio

Demuestre que $2^{n+1} = O(2^n)$.

Solución

$2^{n+1} = 2 \cdot 2^n$. Con $c = 2$ y $n_0 = 0$, se cumple $2^{n+1} \leq 2 \cdot 2^n$.

Ejercicio

Demuestre que $2^{2n} \neq O(2^n)$.

Solución

$2^{2n} = (2^2)^n = 4^n$. Supongamos que existe c, n_0 tal que $4^n \leq c2^n$ para $n \geq n_0$. Esto es $2^n \leq c$, lo cual es falso para n grande.

Ejercicio

Demuestre que $\log(n!) = \Theta(n \log n)$.

Solución

Cota inferior: $n! \geq (n/2)^{n/2}$. Tomando logaritmo:

$$\log(n!) \geq \frac{n}{2}\log\left(\frac{n}{2}\right) = \frac{n}{2}(\log n - 1) = \Theta(n\log n)$$

Cota superior: $n! \leq n^n$. Tomando logaritmo:

$$\log(n!) \leq n\log n = O(n\log n)$$

Por lo tanto, $\log(n!) = \Theta(n\log n)$.

Ejercicio

Demuestre que $\sqrt{n} = \omega(\log n)$.

Solución

Calculamos el límite:

$$\lim_{n\to\infty}\frac{\sqrt{n}}{\log n} = \infty$$

Por lo tanto, $\sqrt{n} = \omega(\log n)$.

Ejercicio

Demuestre que $n\log n = o(n^2)$.

Solución

$$\lim_{n\to\infty}\frac{n\log n}{n^2} = \lim_{n\to\infty}\frac{\log n}{n} = 0$$

Por lo tanto, $n\log n = o(n^2)$.

Ejercicio

Demuestre que $f(n) = \Theta(g(n))$ si y solo si $\lim_{n\to\infty}\frac{f(n)}{g(n)} = c$, con $0 < c < \infty$.

Solución

(Si): Si $\lim_{n\to\infty}\frac{f(n)}{g(n)} = c$, entonces para $\epsilon = c/2$, existe n_0 tal que para $n \geq n_0$:

$$c - \epsilon < \frac{f(n)}{g(n)} < c + \epsilon \implies \frac{c}{2}g(n) < f(n) < \frac{3c}{2}g(n)$$

Por lo tanto, $f(n) = \Theta(g(n))$. **(Solo si)**: Si $f(n) = \Theta(g(n))$, entonces existen c_1, c_2, n_0 tales que $c_1 g(n) \leq f(n) \leq c_2 g(n)$. Esto implica $\frac{f(n)}{g(n)}$ está acotada entre c_1 y c_2. Sin embargo, el límite puede no existir si la función oscila. Por ejemplo, $f(n) = (2 + \sin n)n$, $g(n) = n$, $f(n) = \Theta(g(n))$ pero el límite no existe.

Ejercicio

Ordene las siguientes funciones por orden de crecimiento asintótico: n^3, 2^n, $n \log^2 n$, $n^{0,5}$, $n^{2,5}$, $\log n$, $n!$, $n \log n$, n, $n^2 \log n$.

Solución

Orden de menor a mayor:

$$\log n, \quad n^{0,5}, \quad n, \quad n \log n, \quad n \log^2 n, \quad n^2 \log n, \quad n^{2,5}, \quad n^3, \quad 2^n, \quad n!$$

Justificación:

- $\log n$ crece más lentamente que cualquier potencia positiva.
- $n^{0,5} = \sqrt{n}$ es menor que n.
- n es lineal.
- $n \log n$ es mayor que lineal pero menor que $n^{1+\epsilon}$.
- $n \log^2 n$ es mayor que $n \log n$.
- $n^2 \log n$ es mayor que n^2.
- $n^{2,5}$ es mayor que $n^2 \log n$ porque $n^{0,5}$ crece más rápido que $\log n$.
- n^3 es una potencia mayor.
- 2^n es exponencial.
- $n!$ es mayor que 2^n.

Ejercicio

Determine si las siguientes afirmaciones son verdaderas o falsas:

1. $n^2 = O(n^3)$
2. $n^2 = \Omega(n^3)$
3. $n^2 = o(n^3)$
4. $n^2 = \omega(n^3)$
5. $n^2 = \Theta(n^3)$
6. $2^{n+1} = \Theta(2^n)$
7. $2^{2n} = \Theta(2^n)$
8. $\log(n^2) = \Theta(\log n)$
9. $n \log n = \Theta(n)$

Solución

1. **Verdadero:** $n^2 \leq 1 \cdot n^3$ para $n \geq 1$.
2. **Falso:** $n^2/n^3 = 1/n \to 0$, no hay constante c tal que $cn^3 \leq n^2$ para n grande.
3. **Verdadero:** $\lim_{n\to\infty} n^2/n^3 = 0$.
4. **Falso:** El límite es 0, no ∞.
5. **Falso:** No es Θ porque n^2 crece más lentamente que n^3.
6. **Verdadero:** $2^{n+1} = 2 \cdot 2^n$, por lo que es $\Theta(2^n)$.
7. **Falso:** $2^{2n} = 4^n$, que crece más rápido que 2^n.
8. **Verdadero:** $\log(n^2) = 2\log n = \Theta(\log n)$.
9. **Falso:** $n \log n$ crece más rápido que n por un factor $\log n$.

Ejercicio

Demuestre que $f(n) = O(g(n))$ y $g(n) = O(h(n))$ implica $f(n) = O(h(n))$.

Solución

Por definición, existen c_1, n_1 tales que $f(n) \leq c_1 g(n)$ para $n \geq n_1$, y existen c_2, n_2 tales que $g(n) \leq c_2 h(n)$ para $n \geq n_2$. Tomando $n_0 = \text{máx}(n_1, n_2)$ y $c = c_1 c_2$, tenemos:

$$f(n) \leq c_1 g(n) \leq c_1 c_2 h(n) = ch(n)$$

Por lo tanto, $f(n) = O(h(n))$.

Ejercicio

Demuestre que $f(n) = O(g(n))$ implica $g(n) = \Omega(f(n))$.

Solución

Si $f(n) = O(g(n))$, existen c, n_0 tales que $f(n) \leq cg(n)$ para $n \geq n_0$. Esto equivale a $g(n) \geq \frac{1}{c} f(n)$, que es la definición de $g(n) = \Omega(f(n))$.

Ejercicio

Demuestre que $f(n) = o(g(n))$ implica $f(n) = O(g(n))$, pero el recíproco no es cierto.

Solución

Si $f(n) = o(g(n))$, entonces $\lim f(n)/g(n) = 0$, lo que implica que existe n_0 tal que para $n \geq n_0$, $f(n) \leq 1 \cdot g(n)$. Por lo tanto, $f(n) = O(g(n))$. El recíproco no es cierto porque $n = O(n)$ pero $n \neq o(n)$ (el límite es 1, no 0).

Ejercicio

Demuestre que si $f(n) = \Theta(g(n))$, entonces $\log f(n) = \Theta(\log g(n))$, asumiendo que $f(n), g(n) \geq 2$ para n grande.

Solución

Si $f(n) = \Theta(g(n))$, existen c_1, c_2, n_0 tales que $c_1 g(n) \leq f(n) \leq c_2 g(n)$. Tomando logaritmo:

$$\log(c_1) + \log g(n) \leq \log f(n) \leq \log(c_2) + \log g(n)$$

Para n grande, $\log(c_1)$ y $\log(c_2)$ son constantes despreciables frente a $\log g(n)$. Por lo tanto, $\log f(n) = \Theta(\log g(n))$.

Ejercicio

Demuestre que $n^c = \omega(\log^d n)$ para cualquier $c > 0$, $d > 0$.

Solución

$$\lim_{n\to\infty} \frac{n^c}{\log^d n} = \infty$$

Esto se debe a que cualquier potencia positiva de n crece más rápido que cualquier potencia del logaritmo. Por lo tanto, $n^c = \omega(\log^d n)$.

Ejercicio

Demuestre que $2^n = \omega(n^k)$ para cualquier k constante.

Solución

$$\lim_{n\to\infty} \frac{2^n}{n^k} = \infty$$

Las exponenciales crecen más rápido que cualquier polinomio. Por lo tanto, $2^n = \omega(n^k)$.

Ejercicio

Determine la relación asintótica entre $f(n) = n^{1+\sin n}$ y $g(n) = n$.

Solución

La función $\sin n$ oscila entre -1 y 1. Por lo tanto, $f(n) = n^{1+\sin n}$ oscila entre $n^0 = 1$ y n^2. No existe una relación asintótica fija: a veces $f(n)$ es mucho menor que $g(n)$ (cuando $\sin n = -1$), a veces mucho mayor (cuando $\sin n = 1$). Por lo tanto, ni $f(n) = O(g(n))$ ni $f(n) = \Omega(g(n))$ se cumplen.

Ejercicio

Demuestre que $n^{\log n} = \omega(n^k)$ para cualquier k constante.

Solución

Tomando logaritmos:

$$\log(n^{\log n}) = \log n \cdot \log n = \log^2 n$$

$$\log(n^k) = k \log n$$

Dado que $\log^2 n = \omega(\log n)$, tenemos $n^{\log n} = \omega(n^k)$.

Ejercicio

Demuestre que $\log(n!) = \Theta(n \log n)$.

Solución

Cota inferior: $n! \geq (n/2)^{n/2}$. Tomando logaritmo:

$$\log(n!) \geq \frac{n}{2} \log\left(\frac{n}{2}\right) = \frac{n}{2}(\log n - 1) = \Omega(n \log n)$$

Cota superior: $n! \leq n^n$. Tomando logaritmo:

$$\log(n!) \leq n \log n = O(n \log n)$$

Por lo tanto, $\log(n!) = \Theta(n \log n)$.

2.12. Ejercicios Propuestos (Ampliados)

1. Demuestre que $5n^3 + 10n^2 + 3n = \Theta(n^3)$ usando la definición.
2. Demuestre que $n \log n = O(n^2)$ pero $n \log n \neq \Omega(n^2)$.
3. Demuestre que $n^{1,5} = \omega(n \log n)$.
4. Determine si $2^n = \Theta(3^n)$.
5. Demuestre que $\log n = o(n^\epsilon)$ para cualquier $\epsilon > 0$.
6. Demuestre que $n^2 = \omega(n \log n)$.
7. Ordene las siguientes funciones por orden de crecimiento asintótico: $n^{2,5}$, $n \log n$, 2^n, $n!$, $\log n$, n, $n^{1/2}$, $n^{3/2}$, $n^2 \log n$, n^3.
8. Demuestre que $f(n) = O(g(n))$ implica $f(n) + g(n) = \Theta(g(n))$.
9. Demuestre que $f(n) = o(g(n))$ implica $f(n) + g(n) = \Theta(g(n))$.
10. Demuestre que $f(n) = \Theta(g(n))$ si y solo si $f(n) = O(g(n))$ y $f(n) = \Omega(g(n))$.

11. Demuestre que $f(n) = o(g(n))$ si y solo si $g(n) = \omega(f(n))$.

12. Demuestre que la relación $f(n) = O(g(n))$ es reflexiva y transitiva, pero no simétrica.

13. Determine si las siguientes afirmaciones son verdaderas o falsas:

    a) $n^{0,5} = O(n^{0,4})$
    b) $n^{0,5} = \Omega(n^{0,4})$
    c) $n^{0,5} = \Theta(n^{0,4})$
    d) $\log n = O(1)$
    e) $\log n = \Omega(1)$
    f) $2^{\log n} = \Theta(n)$
    g) $2^{\log^2 n} = O(2^{\log n})$

14. Demuestre que $n^{\log n} = o(2^n)$.

15. Demuestre que $\log \log n = o(\log n)$.

16. Demuestre que $n^{1/2} = o(n^{2/3})$.

17. Demuestre que $n^{1/2} = \omega(\log n)$.

18. Demuestre que si $f(n) = \Theta(g(n))$, entonces $f(n)^k = \Theta(g(n)^k)$ para cualquier $k > 0$.

19. Demuestre que $\sum_{i=1}^{n} i = \Theta(n^2)$.

20. Demuestre que $\sum_{i=1}^{n} i^2 = \Theta(n^3)$.

21. Demuestre que $\sum_{i=1}^{n} \log i = \Theta(n \log n)$.

22. Demuestre que $\sum_{i=1}^{n} 2^i = \Theta(2^n)$.

Capítulo 3

Unidad III: Análisis de Casos: Mejor, Peor y Promedio

3.1. Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender la diferencia fundamental entre el análisis del mejor caso, peor caso y caso promedio.
- Identificar en qué situaciones cada tipo de análisis es relevante.
- Calcular el tiempo de ejecución esperado de algoritmos utilizando técnicas de probabilidad básica.
- Utilizar variables aleatorias indicadoras para simplificar el análisis de casos promedio.
- Aplicar el análisis probabilístico a algoritmos como la búsqueda lineal, la contratación y otros.
- Distinguir entre análisis probabilístico (sobre la entrada) y algoritmos aleatorizados (sobre elecciones internas).

3.2. Introducción: ¿Por qué analizar diferentes casos?

Cuando analizamos un algoritmo, el tiempo de ejecución no es el mismo para todas las entradas de un tamaño dado. Por ejemplo, Insertion Sort puede ser mucho más rápido si el arreglo ya está ordenado que si está en orden inverso. Por lo tanto, necesitamos diferentes tipos de análisis:

- **Mejor caso:** El tiempo mínimo para cualquier entrada de tamaño n. Es útil para entender el límite inferior del algoritmo, pero rara vez es representativo.
- **Peor caso:** El tiempo máximo para cualquier entrada de tamaño n. Proporciona una garantía de que el algoritmo nunca tardará más. Es el más utilizado en la práctica.

- **Caso promedio:** El tiempo esperado, promediado sobre todas las entradas de tamaño n bajo una distribución de probabilidad. Es útil para entender el comportamiento típico.

3.3. El Problema de la Contratación (Hiring Problem)

Para introducir el análisis de casos, estudiaremos un problema clásico que modela la búsqueda de un máximo en una secuencia.

3.3.1. Descripción del Problema

Una agencia de empleo envía un candidato por día. Después de entrevistar a cada candidato, debemos decidir si contratarlo inmediatamente. Solo contratamos si es mejor que todos los anteriores. El costo es c_i por entrevista y c_h por contratación ($c_h > c_i$). Queremos analizar el costo esperado.

3.3.2. Pseudocódigo

```
HIRE-ASSISTANT(n)
1   best = 0   // candidato ficticio peor que todos
2   for i = 1 to n
3       entrevistar al candidato i
4       if candidato i es mejor que best
5           best = i
6           contratar candidato i
```

Listing 3.1: HIRE-ASSISTANT

3.3.3. Análisis del Peor Caso

En el peor caso, los candidatos vienen en orden de calidad creciente. Entonces contratamos a todos los n candidatos. El costo total es:

$$\text{Costo} = n \cdot c_i + n \cdot c_h = \Theta(nc_h)$$

Este es un costo alto, pero ¿es realista esperar que siempre ocurra?

3.3.4. Análisis del Caso Promedio

Supongamos que los candidatos llegan en orden aleatorio. ¿Cuál es el número esperado de contrataciones?

Variables Aleatorias Indicadoras

Una técnica poderosa para simplificar el análisis probabilístico es usar **variables aleatorias indicadoras**. Para un evento A, definimos:

$$I\{A\} = \begin{cases} 1 & \text{si } A \text{ ocurre} \\ 0 & \text{en caso contrario} \end{cases}$$

Lema 3.1 (Lema 5.1 de CLRS)**.** *Para un evento A, $E[I\{A\}] = \Pr\{A\}$.*

Aplicación al Problema de Contratación

Definimos $X_i = I\{\text{el candidato } i \text{ es contratado}\}$. Entonces el número total de contrataciones es $X = \sum_{i=1}^{n} X_i$.

¿Cuál es la probabilidad de que el candidato i sea contratado? Esto ocurre si es mejor que todos los candidatos $1, 2, \ldots, i-1$. Como todos los órdenes son igualmente probables, la probabilidad de que el mejor de los primeros i esté en la posición i es $1/i$.

Por lo tanto:

$$E[X_i] = \Pr\{\text{candidato } i \text{ contratado}\} = \frac{1}{i}$$

Ahora, por linealidad de la expectativa:

$$E[X] = \sum_{i=1}^{n} E[X_i] = \sum_{i=1}^{n} \frac{1}{i} = H_n$$

donde H_n es el n-ésimo número armónico.

Sabemos que $H_n = \ln n + \gamma + O(1/n)$, con $\gamma \approx 0{,}577$. Por lo tanto:

$$E[X] = \Theta(\log n)$$

El costo esperado es:

$$\text{Costo esperado} = n \cdot c_i + c_h \cdot H_n = \Theta(n + c_h \log n)$$

¡Mucho mejor que el peor caso!

3.4. Análisis de Búsqueda Lineal

3.4.1. Descripción

Dado un arreglo $A[1..n]$ y un valor v, queremos encontrar un índice i tal que $A[i] = v$, o devolver NIL si no existe.

3.4.2. Pseudocódigo

```
LINEAR-SEARCH(A, v)
1   for i = 1 to n
2       if A[i] == v
3           return i
4   return NIL
```

Listing 3.2: LINEAR-SEARCH

3.4.3. Análisis del Peor Caso

El peor caso ocurre cuando v no está en el arreglo, o está en la última posición. Se realizan n comparaciones:

$$T_{\text{peor}}(n) = \Theta(n)$$

3.4.4. Análisis del Mejor Caso

El mejor caso ocurre cuando v está en la primera posición. Se realiza 1 comparación:

$$T_{\text{mejor}}(n) = \Theta(1)$$

3.4.5. Análisis del Caso Promedio

Supongamos que v está en el arreglo con probabilidad p, y que si está, es igualmente probable que esté en cualquier posición. Además, supongamos que las búsquedas exitosas e infructuosas tienen diferentes distribuciones.

Búsqueda Exitosa

Si v está en el arreglo, la posición es uniforme entre 1 y n. El número de comparaciones es igual a la posición. El valor esperado es:

$$E[\text{comparaciones} \mid \text{exitosa}] = \frac{1}{n}\sum_{i=1}^{n} i = \frac{n+1}{2} = \Theta(n)$$

Búsqueda Infructuosa

Si v no está en el arreglo, se realizan n comparaciones. El valor esperado es n.

Caso Promedio General

Si la probabilidad de que v esté en el arreglo es p, entonces:

$$E[\text{comparaciones}] = p \cdot \frac{n+1}{2} + (1-p) \cdot n$$

Si $p = 1/2$, por ejemplo:

$$E = \frac{1}{2} \cdot \frac{n+1}{2} + \frac{1}{2} \cdot n = \frac{n+1}{4} + \frac{n}{2} = \frac{3n+1}{4} = \Theta(n)$$

3.5. Análisis de Búsqueda Binaria

3.5.1. Descripción

La búsqueda binaria trabaja sobre un arreglo ordenado, comparando con el elemento medio y reduciendo el espacio de búsqueda a la mitad.

3.5.2. Pseudocódigo

```
BINARY-SEARCH(A, low, high, v)
1   if low > high
2       return NIL
3   mid = floor((low+high)/2)
4   if A[mid] == v
5       return mid
6   else if A[mid] > v
```

```
7        return BINARY-SEARCH(A, low, mid-1, v)
8    else
9        return BINARY-SEARCH(A, mid+1, high, v)
```

Listing 3.3: BINARY-SEARCH

3.5.3. Análisis del Peor Caso

En el peor caso, v no está en el arreglo o está en la última comparación. Cada comparación reduce el tamaño del problema a la mitad. La recurrencia es $T(n) = T(n/2) + \Theta(1)$, cuya solución es $T(n) = \Theta(\log n)$.

3.5.4. Análisis del Mejor Caso

El mejor caso ocurre cuando v es el elemento medio. Se realiza 1 comparación: $T_{\text{mejor}}(n) = \Theta(1)$.

3.5.5. Análisis del Caso Promedio

El análisis del caso promedio es más complejo. Para búsqueda binaria, el número esperado de comparaciones es aproximadamente $\log n$, pero la constante es ligeramente menor. Se puede demostrar que:

$$E[\text{comparaciones}] \approx \log_2 n - 1$$

para búsqueda exitosa, y aproximadamente $\log_2 n$ para búsqueda infructuosa.

3.6. El Problema de la Contratación en Línea (Online Hiring)

3.6.1. Descripción

Ahora supongamos que queremos contratar al mejor candidato, pero solo podemos contratar una vez. Decidimos entrevistar a los primeros k candidatos (para tener una referencia) y luego contratar al primer candidato que sea mejor que todos los vistos hasta ese momento.

3.6.2. Pseudocódigo

```
ON-LINE-MAXIMUM(k, n)
1    best = -infty
2    for i = 1 to k
3        if score(i) > best
4            best = score(i)
5    for i = k+1 to n
6        if score(i) > best
7            return i
8    return n
```

Listing 3.4: ON-LINE-MAXIMUM

3.6.3. Análisis de Probabilidad de Éxito

Queremos maximizar la probabilidad de contratar al mejor candidato.

Teorema 3.1. *La probabilidad de éxito con la estrategia de rechazar los primeros k candidatos es aproximadamente $\frac{k}{n} \ln\left(\frac{n}{k}\right)$.*

3.6.4. Demostración

Sea S_i el evento de que el mejor candidato está en la posición i y es contratado. Para que esto ocurra:

1. El mejor candidato debe estar en la posición i (probabilidad $1/n$).
2. Ningún candidato entre $k+1$ y $i-1$ debe ser mejor que el máximo de los primeros k. Esto ocurre si el máximo de los primeros $i-1$ está entre los primeros k (probabilidad $k/(i-1)$).

Entonces:

$$\Pr\{S_i\} = \frac{1}{n} \cdot \frac{k}{i-1}$$

La probabilidad total de éxito es:

$$\Pr\{\text{éxito}\} = \sum_{i=k+1}^{n} \frac{k}{n(i-1)} = \frac{k}{n} \sum_{j=k}^{n-1} \frac{1}{j} \approx \frac{k}{n} \ln\left(\frac{n}{k}\right)$$

3.6.5. Optimización

Para maximizar esta probabilidad, derivamos respecto a k:

$$\frac{d}{dk}\left(\frac{k}{n} \ln\left(\frac{n}{k}\right)\right) = \frac{1}{n} \ln\left(\frac{n}{k}\right) - \frac{1}{n} = 0 \implies \ln\left(\frac{n}{k}\right) = 1 \implies k = \frac{n}{e}$$

La probabilidad máxima es aproximadamente $1/e \approx 0{,}368$.

3.7. Algoritmos Aleatorizados vs. Análisis Probabilístico

3.7.1. Diferencia Fundamental

- **Análisis Probabilístico:** Asumimos una distribución sobre las entradas (ej. todas las permutaciones igualmente probables). El algoritmo es determinista.
- **Algoritmos Aleatorizados:** El algoritmo toma decisiones aleatorias (usa un generador de números aleatorios). No se asume nada sobre la entrada.

3.7.2. Ejemplo: Quicksort Aleatorizado

El Quicksort determinista tiene peor caso $O(n^2)$ cuando la entrada está ordenada. El Quicksort aleatorizado elige el pivote al azar, evitando que una entrada particular cause el peor caso. El tiempo esperado es $O(n \log n)$ para cualquier entrada.

3.7.3. Pseudocódigo de Quicksort Aleatorizado

```
RANDOMIZED-QUICKSORT(A, p, r)
1   if p < r
2       q = RANDOMIZED-PARTITION(A, p, r)
3       RANDOMIZED-QUICKSORT(A, p, q-1)
4       RANDOMIZED-QUICKSORT(A, q+1, r)

RANDOMIZED-PARTITION(A, p, r)
1   i = RANDOM(p, r)
2   swap(A[r], A[i])
3   return PARTITION(A, p, r)
```

Listing 3.5: RANDOMIZED-QUICKSORT

3.8. Variables Aleatorias Indicadoras: Un Caso de Estudio

3.8.1. Definición Formal

Una variable aleatoria indicadora $I\{A\}$ es 1 si el evento A ocurre, 0 en caso contrario. Su valor esperado es $\Pr\{A\}$.

3.8.2. Ejemplo: Número Esperado de Caras en n Lanzamientos

Sean $X_i = I\{\text{lanzamiento } i \text{ es cara}\}$. Entonces $X = \sum_{i=1}^{n} X_i$ es el número total de caras.

$$E[X] = \sum_{i=1}^{n} E[X_i] = \sum_{i=1}^{n} \frac{1}{2} = \frac{n}{2}$$

3.8.3. Ejemplo: Número Esperado de Inversiones en una Permutación Aleatoria

Una inversión es un par (i, j) con $i < j$ y $A[i] > A[j]$. Para cada par, definimos $X_{ij} = I\{A[i] > A[j]\}$. En una permutación aleatoria, $\Pr\{A[i] > A[j]\} = 1/2$.

$$E[X] = \sum_{1 \le i < j \le n} E[X_{ij}] = \binom{n}{2} \cdot \frac{1}{2} = \frac{n(n-1)}{4}$$

3.9. Ejercicios Resueltos (Ampliados)

Ejercicio

En el problema de contratación, ¿cuál es la probabilidad de contratar exactamente una vez?

Solución

Contratar exactamente una vez significa que solo se contrata al primer candidato. Esto ocurre si el primer candidato es el mejor de todos. La probabilidad de que el mejor de n candidatos esté en la primera posición es $1/n$.

$$\Pr\{\text{una contratación}\} = \frac{1}{n}$$

Ejercicio

En el problema de contratación, ¿cuál es la probabilidad de contratar exactamente dos veces?

Solución

Para contratar exactamente dos veces, el candidato 1 debe tener algún rango r, y el mejor candidato debe estar después de él, y todos los candidatos entre el 1 y el mejor deben tener rango menor que r.

$$\Pr\{\text{dos contrataciones}\} = \frac{1}{n}\sum_{r=1}^{n-1}\frac{1}{n-r} = \frac{H_{n-1}}{n}$$

donde H_{n-1} es el $(n-1)$-ésimo número armónico.

Ejercicio

En búsqueda lineal, si v está en el arreglo con probabilidad p y es uniforme, ¿cuál es el número esperado de comparaciones?

Solución

Sea X el número de comparaciones. Condicionamos:

$$E[X] = \Pr\{\text{exitosa}\} \cdot E[X \mid \text{exitosa}] + \Pr\{\text{infructuosa}\} \cdot E[X \mid \text{infructuosa}]$$

$$E[X] = p \cdot \frac{n+1}{2} + (1-p) \cdot n$$

Si $p = 1$, $E[X] = (n+1)/2 = \Theta(n)$. Si $p = 0$, $E[X] = n = \Theta(n)$. En cualquier caso, es $\Theta(n)$.

Ejercicio

En el problema de contratación en línea, ¿cuál es la probabilidad de éxito si $k = n/2$?

Solución

$$\Pr\{\text{éxito}\} \approx \frac{k}{n}\ln\left(\frac{n}{k}\right) = \frac{1}{2}\ln(2) \approx 0{,}3466$$

Ejercicio

Demuestre que el número esperado de caras en n lanzamientos de una moneda justa es $n/2$ usando variables indicadoras.

Solución

Sea $X_i = I\{\text{lanzamiento } i \text{ es cara}\}$. $E[X_i] = 1/2$. Entonces $X = \sum X_i$ y $E[X] = \sum E[X_i] = n \cdot 1/2 = n/2$.

Ejercicio

En una permutación aleatoria de n elementos, ¿cuál es el número esperado de puntos fijos (elementos que están en su posición original)?

Solución

Sea $X_i = I\{A[i] = i\}$. $\Pr\{A[i] = i\} = 1/n$. Entonces $E[X] = \sum_{i=1}^{n} 1/n = 1$. ¡En promedio, hay exactamente un punto fijo!

Ejercicio

En una permutación aleatoria de n elementos, ¿cuál es la probabilidad de que un elemento dado esté en su posición original? ¿Y de que dos elementos específicos estén en sus posiciones?

Solución

Para un elemento dado, $\Pr\{A[i] = i\} = 1/n$. Para dos elementos i y j, $\Pr\{A[i] = i \text{ y } A[j] = j\} = \frac{1}{n} \cdot \frac{1}{n-1}$, porque después de fijar i, quedan $n-1$ posiciones para j.

Ejercicio

En el problema de contratación, ¿cuál es el número esperado de contrataciones si los candidatos llegan en orden aleatorio?

Solución

Ya calculamos: $E[X] = H_n = \ln n + \gamma + O(1/n) = \Theta(\log n)$.

Ejercicio

En el problema de contratación, ¿cuál es la probabilidad de que el mejor candidato sea contratado?

Solución

El mejor candidato siempre es contratado porque es mejor que todos los anteriores. ¡La probabilidad es 1!

Ejercicio

En búsqueda binaria, ¿cuál es el número de comparaciones en el peor caso para $n = 1000$?

Solución

El número de comparaciones es $\lceil \log_2 n \rceil = \lceil \log_2 1000 \rceil = \lceil 9{,}97 \rceil = 10$.

Ejercicio

Demuestre que en una permutación aleatoria, el número esperado de inversiones es $n(n-1)/4$.

Solución

Usando variables indicadoras $X_{ij} = I\{A[i] > A[j]\}$ para $i < j$. $\Pr\{A[i] > A[j]\} = 1/2$. Hay $\binom{n}{2}$ pares. Entonces:

$$E[X] = \sum_{i<j} \frac{1}{2} = \binom{n}{2} \cdot \frac{1}{2} = \frac{n(n-1)}{4}$$

Ejercicio

En el problema de la contratación en línea, ¿cuál es la probabilidad de éxito si $k = n/3$?

Solución

$$\Pr\{\text{éxito}\} \approx \frac{k}{n} \ln\left(\frac{n}{k}\right) = \frac{1}{3}\ln(3) \approx 0{,}3662$$

Ejercicio

Demuestre que la probabilidad de éxito en el problema de contratación en línea es máxima cuando $k \approx n/e$.

Solución

Derivamos $f(k) = \frac{k}{n}\ln(n/k) = \frac{k}{n}(\ln n - \ln k)$. Derivando respecto a k:

$$f'(k) = \frac{1}{n}(\ln n - \ln k) - \frac{k}{n} \cdot \frac{1}{k} = \frac{1}{n}(\ln n - \ln k - 1)$$

Igualando a 0: $\ln n - \ln k = 1 \implies \ln(n/k) = 1 \implies n/k = e \implies k = n/e$.

Ejercicio

En búsqueda lineal, si $p = 0{,}5$, ¿cuál es el número esperado de comparaciones?

Solución

$$E[X] = 0{,}5 \cdot \frac{n+1}{2} + 0{,}5 \cdot n = \frac{n+1}{4} + \frac{n}{2} = \frac{3n+1}{4}$$

Ejercicio

En búsqueda lineal, si $p = 0$, ¿cuál es el número esperado de comparaciones?

Solución

$$E[X] = 0 \cdot \frac{n+1}{2} + 1 \cdot n = n$$

Ejercicio

En búsqueda lineal, si $p = 1$, ¿cuál es el número esperado de comparaciones?

Solución

$$E[X] = 1 \cdot \frac{n+1}{2} + 0 \cdot n = \frac{n+1}{2}$$

Ejercicio

Demuestre que en una permutación aleatoria, la probabilidad de que el primer elemento sea el máximo es $1/n$.

Solución

El máximo es igualmente probable que esté en cualquier posición. La probabilidad de que esté en la primera es $1/n$.

Ejercicio

En el problema de contratación, ¿cuál es la probabilidad de contratar n veces?

Solución

Contratar n veces significa que cada candidato es mejor que todos los anteriores. Esto ocurre solo si los candidatos vienen en orden estrictamente creciente. La probabilidad es $1/n!$.

Ejercicio

En el problema de contratación, ¿cuál es la probabilidad de contratar al candidato i?

Solución

$\Pr\{\text{contratar } i\} = 1/i$.

Ejercicio

En el problema de contratación en línea, ¿cuál es la probabilidad de contratar al mejor candidato si $k = 0$?

Solución

Si $k = 0$, contratamos al primer candidato. La probabilidad de que sea el mejor es $1/n$.

Ejercicio

En el problema de contratación en línea, ¿cuál es la probabilidad de contratar al mejor candidato si $k = n - 1$?

Solución

Si $k = n - 1$, entrevistamos a todos menos al último, y contratamos al último. La probabilidad de que el último sea el mejor es $1/n$.

3.10. Ejercicios Propuestos (Ampliados)

1. En el problema de contratación, calcule el número esperado de contrataciones para $n = 10$ usando la fórmula H_n.
2. En una permutación aleatoria de $n = 5$ elementos, calcule el número esperado de puntos fijos.
3. Demuestre que $\Pr\{\text{una inversión específica ocurra}\} = 1/2$ en una permutación aleatoria.
4. En búsqueda lineal, si $p = 0{,}2$, ¿cuál es el número esperado de comparaciones para $n = 100$?
5. En búsqueda binaria, ¿cuántas comparaciones se hacen en el peor caso para $n = 1,000,000$?
6. En el problema de contratación en línea, calcule la probabilidad de éxito para $n = 100$ y $k = 37$.
7. Demuestre que $\sum_{i=1}^{n} 1/i = \Theta(\log n)$.
8. En una permutación aleatoria, ¿cuál es el número esperado de elementos que son mayores que todos los anteriores?
9. En el problema de contratación, ¿cuál es la varianza del número de contrataciones? (Pista: use que $\mathrm{Var}(X) = E[X^2] - E[X]^2$ y que $E[X_i X_j] = E[X_i]E[X_j]$ para $i \neq j$ porque son independientes.)

10. Diseñe un algoritmo aleatorizado que decida si un número está en un arreglo, y analice su tiempo esperado.

11. En el problema de la búsqueda de un máximo, ¿cuál es la probabilidad de que el máximo esté en la primera mitad?

12. En el problema de la contratación en línea, ¿cuál es la probabilidad de éxito si $k = n/4$?

13. Demuestre que $E[X^2] \geq E[X]^2$ para cualquier variable aleatoria X (desigualdad de Jensen).

14. En una permutación aleatoria, ¿cuál es la probabilidad de que el elemento i sea un punto fijo y el elemento j también?

15. En el problema de contratación, ¿cuál es la probabilidad de contratar exactamente dos veces para $n = 3$?

16. En búsqueda lineal, si $p = 0{,}8$, ¿cuál es el número esperado de comparaciones para $n = 50$?

17. Demuestre que la probabilidad de que el máximo esté en la última posición en una permutación aleatoria es $1/n$.

18. En el problema de contratación en línea, demuestre que la probabilidad de éxito es exactamente $\frac{k}{n} \sum_{j=k}^{n-1} \frac{1}{j}$.

19. En una permutación aleatoria, ¿cuál es el número esperado de elementos que son menores que todos los siguientes?

20. En el problema de contratación, ¿cuál es la probabilidad de que el candidato 1 sea contratado?

Capítulo 4

Unidad IV: Estructuras de Datos Avanzadas - Árboles Balanceados

4.1. Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender la necesidad de árboles balanceados para garantizar operaciones $O(\log n)$ en el peor caso.
- Definir y caracterizar árboles AVL: factor de equilibrio, altura, rotaciones.
- Implementar las operaciones de inserción y eliminación en árboles AVL.
- Comprender los conceptos básicos de árboles rojo-negro: propiedades, coloración, invariantes.
- Analizar la complejidad de las operaciones en árboles AVL y rojo-negro.
- Comparar ambas estructuras y entender sus aplicaciones.

4.2. Introducción: La Necesidad de Árboles Balanceados

Los árboles de búsqueda binaria (BST) son estructuras eficientes en promedio, pero pueden degenerar en una lista enlazada si los elementos se insertan en orden. Esto produce operaciones $O(n)$ en el peor caso.

Ejemplo

[Degeneración de un BST] Insertar las claves 1, 2, 3, 4, 5 en un BST vacío produce:

```
1
 \
  2
   \
    3
     \
      4
       \
        5
```

La búsqueda de 5 requiere 5 comparaciones, $O(n)$ en lugar de $O(\log n)$.

Para garantizar un rendimiento $O(\log n)$ en el peor caso, necesitamos **árboles balanceados**. Los más conocidos son:

- **Árboles AVL** (Adelson-Velsky y Landis, 1962)
- **Árboles Rojo-Negro** (Bayer, 1972)

4.3. Árboles AVL: Definición y Propiedades

4.3.1. Definición

Un árbol AVL es un árbol de búsqueda binaria que satisface la **propiedad de equilibrio AVL**: para cada nodo x, las alturas de sus subárboles izquierdo y derecho difieren en a lo sumo 1.

El **factor de equilibrio** de un nodo x se define como:

$$\text{FE}(x) = \text{altura}(x.\text{izquierdo}) - \text{altura}(x.\text{derecho})$$

En un árbol AVL, $\text{FE}(x) \in \{-1, 0, 1\}$ para todo nodo x.

4.3.2. Altura de un Árbol AVL

Teorema 4.1. *Un árbol AVL de n nodos tiene altura $h \leq 1{,}44 \log_2(n+1)$.*

Demostración. Sea N_h el número mínimo de nodos en un árbol AVL de altura h.

- $N_0 = 1$ (un nodo)
- $N_1 = 2$ (raíz con un hijo)
- Para $h \geq 2$, el árbol tiene raíz con subárbol izquierdo de altura $h-1$ y subárbol derecho de altura $h-2$ (el mínimo ocurre cuando la diferencia es máxima).

Por lo tanto:

$$N_h = N_{h-1} + N_{h-2} + 1$$

Esta recurrencia es similar a los números de Fibonacci. Se puede demostrar que $N_h \geq F_{h+2} - 1$, donde F_k es el k-ésimo número de Fibonacci. Como $F_k \approx \phi^k/\sqrt{5}$ con $\phi = (1+\sqrt{5})/2 \approx 1{,}618$, tenemos:

$$N_h \geq \phi^{h+2}/\sqrt{5} - 1 \implies h \leq \log_\phi(N_h + 1) \approx 1{,}44 \log_2(n+1)$$

□

4.3.3. Estructura de un Nodo AVL

Cada nodo almacena:

- `key`: valor de la clave
- `left`: puntero al hijo izquierdo
- `right`: puntero al hijo derecho
- `parent`: puntero al padre (opcional)
- `height`: altura del subárbol enraizado en el nodo

4.4. Rotaciones en Árboles AVL

Las rotaciones son operaciones locales que reestructuran el árbol manteniendo la propiedad de BST. Son la herramienta fundamental para restaurar el equilibrio después de inserciones o eliminaciones.

4.4.1. Rotación a la Derecha (Right Rotate)

La rotación a la derecha se aplica cuando un nodo tiene un hijo izquierdo desbalanceado.

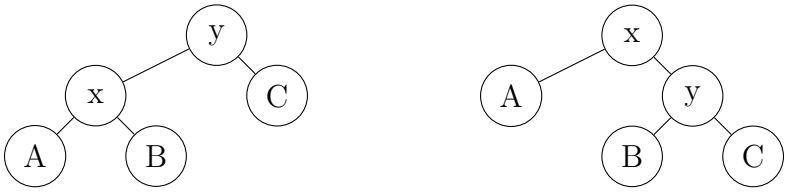


Figura 4.1: Rotación a la derecha: antes y después

```
RIGHT-ROTATE(y)
    x = y.left
    y.left = x.right
    if x.right != NIL
        x.right.parent = y
    x.right = y
    x.parent = y.parent
    if y.parent == NIL
        root = x
    else if y == y.parent.left
        y.parent.left = x
```

```
    else
        y.parent.right = x
    y.parent = x
    // Actualizar alturas
    update-height(y)
    update-height(x)
```

Listing 4.1: Rotación a la derecha

4.4.2. Rotación a la Izquierda (Left Rotate)

La rotación a la izquierda es simétrica.

```
LEFT-ROTATE(x)
    y = x.right
    x.right = y.left
    if y.left != NIL
        y.left.parent = x
    y.left = x
    y.parent = x.parent
    if x.parent == NIL
        root = y
    else if x == x.parent.left
        x.parent.left = y
    else
        x.parent.right = y
    x.parent = y
    // Actualizar alturas
    update-height(x)
    update-height(y)
```

Listing 4.2: Rotación a la izquierda

4.4.3. Actualización de Alturas

La altura de un nodo se calcula como:

$$\text{height}(x) = 1 + \text{máx}(\text{height}(x.\text{left}), \text{height}(x.\text{right}))$$

con $\text{height}(\text{NIL}) = -1$ (o 0, según convención).

```
UPDATE-HEIGHT(x)
    x.height = 1 + max(height(x.left), height(x.right))
```

Listing 4.3: Actualizar altura

4.5. Inserción en Árboles AVL

4.5.1. Pasos de la Inserción

1. Insertar el nuevo nodo como en un BST normal.
2. Actualizar las alturas de los ancestros.
3. Verificar el factor de equilibrio en cada ancestro.
4. Si algún nodo está desbalanceado ($|\text{FE}| > 1$), aplicar la rotación correspondiente.

4.5.2. Casos de Desbalance

Hay cuatro casos de desbalance después de una inserción:

1. **Caso 1: Izquierda-Izquierda (LL)**: El desbalance ocurre en el hijo izquierdo del hijo izquierdo.
2. **Caso 2: Izquierda-Derecha (LR)**: El desbalance ocurre en el hijo derecho del hijo izquierdo.
3. **Caso 3: Derecha-Derecha (RR)**: El desbalance ocurre en el hijo derecho del hijo derecho.
4. **Caso 4: Derecha-Izquierda (RL)**: El desbalance ocurre en el hijo izquierdo del hijo derecho.

4.5.3. Rotaciones para los Casos

- **Caso LL**: Rotación a la derecha sobre el nodo desbalanceado.
- **Caso RR**: Rotación a la izquierda sobre el nodo desbalanceado.
- **Caso LR**: Rotación a la izquierda sobre el hijo izquierdo, luego rotación a la derecha sobre el nodo desbalanceado.
- **Caso RL**: Rotación a la derecha sobre el hijo derecho, luego rotación a la izquierda sobre el nodo desbalanceado.

4.5.4. Pseudocódigo de Inserción

```
AVL-INSERT(root, key)
    root = AVL-INSERT-REC(root, key)
    return root

AVL-INSERT-REC(node, key)
    if node == NIL
        return new Node(key)
    if key < node.key
        node.left = AVL-INSERT-REC(node.left, key)
        node.left.parent = node
    else if key > node.key
        node.right = AVL-INSERT-REC(node.right, key)
        node.right.parent = node
    else
        return node  // clave duplicada, no insertar

    // Actualizar altura
    UPDATE-HEIGHT(node)

    // Verificar balance
    balance = height(node.left) - height(node.right)

    // Caso LL
    if balance > 1 and key < node.left.key
        return RIGHT-ROTATE(node)
```

```
    // Caso RR
    if balance < -1 and key > node.right.key
        return LEFT-ROTATE(node)

    // Caso LR
    if balance > 1 and key > node.left.key
        node.left = LEFT-ROTATE(node.left)
        return RIGHT-ROTATE(node)

    // Caso RL
    if balance < -1 and key < node.right.key
        node.right = RIGHT-ROTATE(node.right)
        return LEFT-ROTATE(node)

    return node
```

Listing 4.4: Inserción en AVL

4.5.5. Ejemplo de Inserción

Insertar las claves: 10, 20, 30, 40, 50, 25.

1. Insertar 10: árbol balanceado.
2. Insertar 20: balanceado.
3. Insertar 30: desbalance en 10 (factor -2, caso RR). Rotación izquierda sobre 10.
4. Insertar 40: balanceado.
5. Insertar 50: desbalance en 20 (factor -2, caso RR). Rotación izquierda sobre 20.
6. Insertar 25: desbalance en 30 (factor 2, caso LR). Rotación izquierda sobre 25, luego derecha sobre 30.

4.6. Eliminación en Árboles AVL

4.6.1. Pasos de la Eliminación

1. Eliminar el nodo como en un BST normal.
2. Actualizar las alturas de los ancestros.
3. Verificar el factor de equilibrio en cada ancestro.
4. Si algún nodo está desbalanceado, aplicar la rotación correspondiente.

4.6.2. Casos de Desbalance en Eliminación

Los casos son similares a la inserción, pero con una sutileza: después de una rotación, puede ser necesario continuar verificando hacia arriba.

```
AVL-DELETE(root, key)
    root = AVL-DELETE-REC(root, key)
    return root

AVL-DELETE-REC(node, key)
    if node == NIL
        return NIL

    if key < node.key
        node.left = AVL-DELETE-REC(node.left, key)
        if node.left != NIL
            node.left.parent = node
    else if key > node.key
        node.right = AVL-DELETE-REC(node.right, key)
        if node.right != NIL
            node.right.parent = node
    else
        // Nodo encontrado
        if node.left == NIL or node.right == NIL
            // Caso 0 o 1 hijo
            child = node.left if node.left != NIL else node.right
            if child != NIL
                child.parent = node.parent
            return child
        else
            // Dos hijos: encontrar sucesor
            successor = MINIMUM(node.right)
            node.key = successor.key
            node.right = AVL-DELETE-REC(node.right, successor.key)
            if node.right != NIL
                node.right.parent = node

    // Actualizar altura
    UPDATE-HEIGHT(node)

    // Verificar balance
    balance = height(node.left) - height(node.right)

    // Caso LL
    if balance > 1 and height(node.left.left) >= height(node.left.right)
        return RIGHT-ROTATE(node)

    // Caso LR
    if balance > 1 and height(node.left.left) < height(node.left.right)
        node.left = LEFT-ROTATE(node.left)
        return RIGHT-ROTATE(node)

    // Caso RR
    if balance < -1 and height(node.right.right) >= height(node.right.left)
        return LEFT-ROTATE(node)

    // Caso RL
    if balance < -1 and height(node.right.right) < height(node.right.left)
        node.right = RIGHT-ROTATE(node.right)
        return LEFT-ROTATE(node)
```

```
    return node
```

Listing 4.5: Eliminación en AVL

4.7. Árboles Rojo-Negro: Conceptos Básicos

4.7.1. Definición

Un árbol rojo-negro es un árbol de búsqueda binaria con un bit extra de color (rojo o negro) por nodo que satisface las siguientes propiedades:

1. Cada nodo es rojo o negro.
2. La raíz es negra.
3. Cada hoja (NIL) es negra.
4. Si un nodo es rojo, ambos hijos son negros.
5. Para cada nodo, todos los caminos desde el nodo a las hojas descendientes contienen el mismo número de nodos negros (altura negra).

4.7.2. Estructura de un Nodo Rojo-Negro

```
struct Node {
    int key;
    Node *left, *right, *parent;
    bool color;  // true = rojo, false = negro
};
```

4.7.3. Propiedad Fundamental

Teorema 4.2. *Un árbol rojo-negro con n nodos internos tiene altura $h \leq 2\log_2(n+1)$.*

Demostración. Sea $bh(x)$ la altura negra de un nodo x. Por propiedad 4, al menos la mitad de los nodos en cualquier camino desde la raíz hasta una hoja son negros, por lo que $bh(raíz) \geq h/2$. Además, el número mínimo de nodos internos en un árbol con altura negra b es al menos $2^b - 1$. Por lo tanto:

$$n \geq 2^{bh(raíz)-1 \geq 2^{h/2}-1 \Longrightarrow h \leq 2\log_2(n+1)}$$

□

4.7.4. Rotaciones en Árboles Rojo-Negro

Las rotaciones son idénticas a las de AVL, pero se usan junto con recoloraciones para mantener las propiedades.

4.7.5. Inserción en Árboles Rojo-Negro

La inserción tiene dos fases:

1. Insertar el nuevo nodo como rojo (similar a BST).
2. Corregir posibles violaciones de propiedades mediante recoloraciones y rotaciones.

Casos de Corrección

Hay tres casos principales cuando el padre del nuevo nodo es rojo:

1. **Caso 1: El tío es rojo**. Se recoloran padre, tío y abuelo. El abuelo se convierte en el nuevo nodo a verificar.
2. **Caso 2: El tío es negro y el nodo es hijo derecho**. Se rota a la izquierda sobre el padre, transformando al caso 3.
3. **Caso 3: El tío es negro y el nodo es hijo izquierdo**. Se rota a la derecha sobre el abuelo y se recoloran.

```
RB-INSERT(root, key)
    z = new Node(key)
    z.color = RED
    // Insertar como en BST
    BST-INSERT(root, z)
    // Corregir violaciones
    while z != root and z.parent.color == RED
        if z.parent == z.parent.parent.left
            uncle = z.parent.parent.right
            if uncle.color == RED  // Caso 1
                z.parent.color = BLACK
                uncle.color = BLACK
                z.parent.parent.color = RED
                z = z.parent.parent
            else
                if z == z.parent.right  // Caso 2
                    z = z.parent
                    LEFT-ROTATE(z)
                // Caso 3
                z.parent.color = BLACK
                z.parent.parent.color = RED
                RIGHT-ROTATE(z.parent.parent)
        else  // Sim trico
            // ... c digo an logo con izquierda/derecha
    intercambiadas
    root.color = BLACK
```

Listing 4.6: Inserción en árbol rojo-negro (simplificada)

4.7.6. Eliminación en Árboles Rojo-Negro

La eliminación es más compleja. Cuando se elimina un nodo negro, se crea un "negro extra"que debe ser manejado. Hay varios casos para restaurar las propiedades.

4.8. Comparación: AVL vs. Rojo-Negro

Característica	AVL	Rojo-Negro
Altura máxima	$\approx 1{,}44 \log n$	$\approx 2 \log n$
Búsqueda	Más rápida (más balanceado)	Ligeramente más lenta
Inserción	Más rotaciones	Menos rotaciones
Eliminación	Más rotaciones	Menos rotaciones
Aplicaciones	Búsquedas frecuentes	Inserciones/eliminaciones frecuentes

4.9. Ejercicios Resueltos (Ampliados)

Ejercicio

Construya un árbol AVL insertando las claves 10, 20, 30, 40, 50, 25 en ese orden.

Solución

1. Insertar 10: árbol con raíz 10.
2. Insertar 20: 10 (der) = 20. Alturas: 10(1), 20(0). Balanceados.
3. Insertar 30: 10 (der) = 20 (der) = 30. Factor en 10 = 0-2 = -2 (desbalance). Caso RR: rotación izquierda sobre 10. Resultado: raíz 20, izquierdo 10, derecho 30.
4. Insertar 40: 20 (der) = 30 (der) = 40. Factor en 30 = 0-1 = -1, en 20 = 0-2 = -2. Caso RR sobre 20: rotación izquierda sobre 20. Resultado: raíz 30, izquierdo 20, derecho 40; 20 tiene izquierdo 10.
5. Insertar 50: 30 (der) = 40 (der) = 50. Factor en 40 = -1, en 30 = -2. Caso RR sobre 30: rotación izquierda sobre 30. Resultado: raíz 40, izquierdo 30, derecho 50; 30 tiene izquierdo 20, derecho?; 20 tiene izquierdo 10.
6. Insertar 25: 40 (izq) = 30 (izq) = 20 (der) = 25. Verificar balances: 20 tiene factor 1 (izq 10, der 25), 30 tiene factor 1 (izq 20, der ?), 40 tiene factor -1. En 30, balance ¿1 y 25 ¿20, caso LR: rotación izquierda sobre 20 (20-25), luego derecha sobre 30.

Árbol final: raíz 30, izquierdo 20, derecho 40; 20 tiene izquierdo 10, derecho 25; 40 tiene derecho 50.

Ejercicio

Demuestre que un árbol AVL de altura h tiene al menos F_{h+2} nodos, donde F es la sucesión de Fibonacci.

Solución

Sea N_h el mínimo número de nodos en un AVL de altura h.

- $N_0 = 1 = F_2$
- $N_1 = 2 = F_3$
- Para $h \geq 2$, $N_h = N_{h-1} + N_{h-2} + 1 \geq F_{h+1} + F_h + 1 = F_{h+2} + 1 - 1$? En realidad $F_{h+2} = F_{h+1} + F_h$, entonces $N_h \geq F_{h+2} + 1$? Verificar: $N_2 = 3$, $F_4 = 3$, entonces $N_2 \geq F_4$.

Por inducción, $N_h \geq F_{h+2}$.

Ejercicio

¿Cuál es la altura máxima de un árbol AVL con 100 nodos?

Solución

$h \leq 1{,}44 \log_2(101) \approx 1{,}44 \times 6{,}66 \approx 9{,}6$. Por lo tanto, $h \leq 9$.

Ejercicio

Explique por qué en un árbol rojo-negro, la altura máxima es $2 \log_2(n+1)$.

Solución

Por propiedad 4, al menos la mitad de los nodos en cualquier camino son negros. Si la altura negra es b, entonces $b \geq h/2$. Por propiedad 5, el número de nodos $n \geq 2^b - 1 \geq 2^{h/2} - 1$, por lo que $h \leq 2 \log_2(n+1)$.

Ejercicio

Realice una rotación a la derecha sobre el nodo 30 en el árbol: 50(izq=30, der=70); 30(izq=20, der=40).

Solución

Después de rotación derecha sobre 30:

- 20 se convierte en hijo izquierdo de 50.
- 30 se convierte en hijo derecho de 20.
- El subárbol derecho de 20 (que estaba vacío) se convierte en hijo izquierdo de 30.

Resultado: raíz 50, izquierdo 20; 20 tiene derecho 30; 30 tiene izquierdo ?, derecho 40.

Ejercicio

Inserte la clave 15 en un árbol AVL que tiene raíz 10 con derecho 20, y luego verifique el balance.

Solución

Insertar 15: 10 (der) = 20 (izq) = 15. Ahora 20 tiene factor -1 (izq 15, der ?), 10 tiene factor -2 (izq ?, der 20). Es caso LR: rotación izquierda sobre 15, luego derecha sobre 10.

Ejercicio

¿Cuál es la altura negra de un árbol rojo-negro con raíz negra y todos los nodos negros?

Solución

Si todos los nodos son negros, la altura negra es igual a la altura del árbol. Para n nodos, $n \geq 2^h - 1$, por lo que $h \leq \log_2(n+1)$.

Ejercicio

Demuestre que en un árbol rojo-negro, el camino más largo es a lo sumo el doble del camino más corto.

Solución

El camino más corto tiene solo nodos negros. El camino más largo alterna rojo-negro, por lo que tiene a lo sumo el doble de nodos.

Ejercicio

¿Qué sucede si insertamos una clave en un árbol AVL y después de la inserción el factor de equilibrio de la raíz es 2?

Solución

Indica que el subárbol izquierdo es más alto. Depende de dónde se insertó:

- Si la inserción fue en el subárbol izquierdo del hijo izquierdo (LL), rotación derecha.
- Si fue en el subárbol derecho del hijo izquierdo (LR), rotación izquierda-derecha.

Ejercicio

En un árbol rojo-negro, ¿puede un nodo rojo tener un hijo rojo?

Solución

No, por propiedad 4, si un nodo es rojo, ambos hijos deben ser negros.

Ejercicio

Construya un árbol rojo-negro insertando las claves 1, 2, 3, 4, 5 en ese orden.

Solución

1. Insertar 1 (raíz negra).
2. Insertar 2 (rojo, hijo derecho de 1). Árbol balanceado.
3. Insertar 3: 1(der=2(der=3)). Violación: tío de 3 es NIL (negro), caso 3. Rotación izquierda sobre 1, recolorar. Resultado: raíz 2 (negro), izquierdo 1 (rojo), derecho 3 (rojo).
4. Insertar 4: 2(der=3(der=4)). Tío de 4 es 1 (rojo), caso 1. Recolorar: 3 a negro, 1 a negro, 2 a rojo. Como 2 es raíz, se recolora a negro.
5. Insertar 5: 2(der=3(der=4(der=5))). Tío de 5 es NIL (negro), caso 3. Rotación izquierda sobre 4, luego izquierda sobre 3, recolorar.

Ejercicio

¿Cuál es la diferencia principal entre la inserción en AVL y en rojo-negro?

Solución

AVL usa rotaciones inmediatas para mantener el factor de equilibrio estricto, mientras que rojo-negro usa recoloraciones primero y rotaciones solo cuando es necesario. AVL es más estricto (más balanceado), rojo-negro tiene menos rotaciones.

Ejercicio

Demuestre que un árbol AVL con $n = 7$ puede tener altura 3.

Solución

El árbol AVL mínimo con altura 3 tiene $N_3 = N_2 + N_1 + 1 = 4 + 2 + 1 = 7$ nodos. Por lo tanto, es posible.

Ejercicio

¿Qué es una rotación doble y cuándo se usa?

Solución

Una rotación doble consiste en dos rotaciones simples. Se usa en los casos LR y RL, cuando el desbalance ocurre en el "nieto"del nodo desbalanceado.

Ejercicio

En un árbol rojo-negro, si la raíz es roja, ¿se viola alguna propiedad?

Solución

Sí, propiedad 2: la raíz debe ser negra.

Ejercicio

¿Cuántas rotaciones puede requerir una inserción en un árbol AVL en el peor caso?

Solución

En el peor caso, puede requerir $O(\log n)$ rotaciones, pero en la práctica solo una o dos rotaciones son comunes.

Ejercicio

¿Cuántas rotaciones puede requerir una inserción en un árbol rojo-negro?

Solución

Como máximo dos rotaciones, porque los casos 2 y 3 implican una rotación cada uno, y el caso 1 solo recoloraciones.

4.10. Ejercicios Propuestos (Ampliados)

1. Construya un árbol AVL insertando las claves: 50, 40, 30, 20, 10, 25, 35.
2. Construya un árbol AVL insertando las claves: 5, 3, 7, 2, 4, 6, 8, 1.
3. Elimine la clave 30 del árbol AVL construido en el ejercicio 1.
4. Demuestre que $N_h \geq F_{h+2}$ por inducción.
5. ¿Cuál es la altura máxima de un árbol AVL con 1000 nodos?
6. Construya un árbol rojo-negro insertando las claves: 10, 20, 30, 40, 50, 60.
7. Construya un árbol rojo-negro insertando las claves: 1, 2, 3, 4, 5, 6, 7.
8. Explique por qué en un árbol rojo-negro, un nodo rojo no puede tener un hijo rojo.
9. ¿Qué sucede si eliminamos un nodo rojo de un árbol rojo-negro? ¿Y si es negro?
10. Implemente una función que verifique si un árbol es AVL.
11. Implemente una función que verifique si un árbol es rojo-negro.
12. Compare el número de rotaciones necesarias para insertar 1000 elementos aleatorios en un AVL vs. rojo-negro.

13. ¿Cuál es la complejidad en el peor caso de búsqueda en un AVL? ¿Y en un rojo-negro?

14. Dibuje un árbol AVL de altura 3 con el mínimo número de nodos.

15. Dibuje un árbol rojo-negro de altura 4 con el máximo número de nodos.

16. Demuestre que la altura negra de un árbol rojo-negro es al menos $h/2$.

17. ¿Puede un árbol rojo-negro tener todos los nodos rojos? ¿Por qué?

18. ¿Qué ventaja tiene el rojo-negro sobre el AVL en aplicaciones con muchas inserciones y eliminaciones?

19. Diseñe un algoritmo para encontrar el k-ésimo elemento más pequeño en un árbol AVL en $O(\log n)$.

20. Demuestre que la altura de un árbol AVL es $\Theta(\log n)$.

Capítulo 5

Unidad V: Árboles B y B+ - Estructuras para Almacenamiento Secundario

5.1. Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender la problemática del almacenamiento secundario (discos) y la necesidad de estructuras optimizadas para E/S.
- Definir la estructura y propiedades de los árboles B.
- Implementar las operaciones de búsqueda, inserción y eliminación en árboles B.
- Comprender la variante B+ y sus ventajas en sistemas de bases de datos.
- Analizar la complejidad de las operaciones en términos de accesos a disco.
- Aplicar árboles B en contextos reales como índices de bases de datos y sistemas de archivos.

5.2. Introducción: El Problema del Almacenamiento Secundario

5.2.1. Jerarquía de Memoria

Los sistemas computacionales tienen una jerarquía de memoria:

- **Registros:** nanosegundos, capacidad muy limitada.
- **Caché:** nanosegundos, capacidad limitada.
- **Memoria RAM:** microsegundos, capacidad moderada.
- **Disco magnético:** milisegundos, capacidad enorme.

5.2.2. Tiempos de Acceso

Nivel	Tiempo de acceso típico
Registros	¡1 ns
Caché L1	1-2 ns
Caché L2	5-10 ns
Memoria RAM	50-100 ns
Disco (búsqueda)	5-10 ms

La diferencia es de 5-6 órdenes de magnitud: ¡un acceso a disco equivale a millones de accesos a RAM!

5.2.3. Principio de Localidad

Para mitigar la lentitud del disco, se utilizan **páginas**: bloques contiguos de datos (típicamente 4-16 KB). Leer una página cuesta lo mismo que leer un byte, por lo que conviene traer más datos de los necesarios.

5.2.4. Objetivo de los Árboles B

Minimizar el número de accesos a disco. Para ello:

- Cada nodo del árbol ocupa una página de disco.
- El factor de ramificación (número de hijos por nodo) es grande (cientos o miles).
- La altura del árbol es muy pequeña ($\log_{1000} n$).

5.3. Definición de Árbol B

5.3.1. Propiedades Fundamentales

Un árbol B de grado mínimo $t \geq 2$ (también llamado orden $m = 2t$) satisface:

1. Cada nodo x tiene los siguientes campos:
   - $n[x]$: número de claves almacenadas en x.
   - Las $n[x]$ claves almacenadas en orden no decreciente: $k_1[x] \leq k_2[x] \leq \cdots \leq k_{n[x]}[x]$.
   - $hoja[x]$: booleano que indica si x es hoja.
2. Cada nodo interno x tiene $n[x] + 1$ punteros $c_1[x], c_2[x], \ldots, c_{n[x]+1}[x]$ a sus hijos.
3. Las claves separan los rangos de los subárboles: para cualquier clave k en el subárbol $c_i[x]$, se cumple:
   $$k_{i-1}[x] \leq k \leq k_i[x]$$
   (con $k_0 = -\infty$ y $k_{n[x]+1} = \infty$).
4. Todas las hojas tienen la misma profundidad (el árbol es perfectamente balanceado).

5. **Límites en el número de claves:**
   - Cada nodo (excepto la raíz) tiene al menos $t-1$ claves.
   - Cada nodo tiene a lo sumo $2t-1$ claves (está **lleno** si tiene $2t-1$ claves).

5.3.2. Altura de un Árbol B

Teorema 5.1. *Un árbol B de altura h con n claves satisface:*

$$h \leq \log_t \left(\frac{n+1}{2} \right)$$

Demostración. El número mínimo de claves en un árbol B de altura h:

- Raíz: 1 clave
- Nivel 1: al menos 2 nodos (raíz con t hijos)
- Nivel 2: al menos $2t$ nodos
- Nivel i: al menos $2t^{i-1}$ nodos

Cada nodo tiene al menos $t-1$ claves, excepto la raíz. Por lo tanto:

$$n \geq 1 + (t-1)(2 + 2t + 2t^2 + \cdots + 2t^{h-1})$$

$$n \geq 1 + 2(t-1)(t^{h-1} - 1)/(t-1) = 2t^{h-1} - 1$$

$$t^{h-1} \leq (n+1)/2 \implies h \leq \log_t((n+1)/2) + 1$$

□

5.3.3. Ejemplo Numérico

Con $t = 200$ (factor de ramificación 200-400) y $n = 10^9$:

$$h \leq \log_{200}(5 \times 10^8) \approx \frac{\log_{10}(5 \times 10^8)}{\log_{10}(200)} \approx \frac{8{,}7}{2{,}3} \approx 3{,}8$$

¡Solo 4 niveles! Es decir, 4 accesos a disco para encontrar cualquier clave.

5.4. Representación de un Árbol B

5.4.1. Estructura de Nodo

```
struct BNode {
    int n;                      // n mero de claves
    int keys[2*t-1];            // arreglo de claves
    BNode* children[2*t];       // arreglo de punteros a hijos
    bool leaf;                  // es hoja?
};
```

Listing 5.1: Estructura de un nodo de árbol B

5.4.2. Operaciones de Disco

En pseudocódigo, usamos:

- `DISK-READ(x)`: lee la página del nodo x del disco a memoria.
- `DISK-WRITE(x)`: escribe la página del nodo x de vuelta al disco.

Asumimos que un nodo cabe en una página de disco.

5.5. Búsqueda en Árbol B

5.5.1. Pseudocódigo

```
B-TREE-SEARCH(x, k)
    i = 1
    while i <= x.n and k > x.keys[i]
        i = i + 1
    if i <= x.n and k == x.keys[i]
        return (x, i)            // encontrado
    if x.leaf
        return NIL                // no encontrado
    else
        DISK-READ(x.children[i])
        return B-TREE-SEARCH(x.children[i], k)
```

Listing 5.2: Búsqueda en árbol B

5.5.2. Análisis de Complejidad

- **Accesos a disco:** $O(h) = O(\log_t n)$.
- **Comparaciones:** $O(t \cdot h) = O(t \log_t n)$.

Para t grande, las comparaciones dominan, pero son en memoria, por lo que el cuello de botella son los accesos a disco.

5.6. Inserción en Árbol B

5.6.1. Estrategia General

1. Buscar la hoja donde debe insertarse la clave.
2. Si la hoja tiene espacio (menos de $2t - 1$ claves), insertar y listo.
3. Si la hoja está llena, **dividirla** en dos nodos con $t - 1$ claves cada uno, y subir la clave mediana al padre.
4. Si el padre está lleno, repetir la división hacia arriba.
5. Si la raíz se divide, se crea una nueva raíz (el árbol crece en altura).

5.6.2. Operación de División

```
B-TREE-SPLIT-CHILD(x, i, y)
    // y = x.children[i] est  lleno (2t-1 claves)
    z = new BNode()
    z.leaf = y.leaf
    z.n = t - 1

    // Copiar las t-1 claves m s grandes de y a z
    for j = 1 to t-1
        z.keys[j] = y.keys[j + t]

    if not y.leaf
        // Copiar los t hijos m s grandes de y a z
        for j = 1 to t
            z.children[j] = y.children[j + t]

    y.n = t - 1

    // Insertar z como hijo de x
    for j = x.n+1 downto i+1
        x.children[j+1] = x.children[j]
    x.children[i+1] = z

    // Subir la clave mediana a x
    for j = x.n downto i
        x.keys[j+1] = x.keys[j]
    x.keys[i] = y.keys[t]
    x.n = x.n + 1

    DISK-WRITE(y)
    DISK-WRITE(z)
    DISK-WRITE(x)
```

Listing 5.3: Dividir un nodo

5.6.3. Inserción Principal

```
B-TREE-INSERT(T, k)
    r = T.root
    if r.n == 2*t - 1   // ra z llena
        s = new BNode()
        s.leaf = false
        s.n = 0
        s.children[1] = r
        T.root = s
        B-TREE-SPLIT-CHILD(s, 1, r)
        B-TREE-INSERT-NONFULL(s, k)
    else
        B-TREE-INSERT-NONFULL(r, k)

B-TREE-INSERT-NONFULL(x, k)
    i = x.n
    if x.leaf
        // Insertar en la hoja
        while i >= 1 and k < x.keys[i]
            x.keys[i+1] = x.keys[i]
```

```
            i = i - 1
        x.keys[i+1] = k
        x.n = x.n + 1
        DISK-WRITE(x)
    else
        // Buscar el hijo adecuado
        while i >= 1 and k < x.keys[i]
            i = i - 1
        i = i + 1
        DISK-READ(x.children[i])
        if x.children[i].n == 2*t - 1
            B-TREE-SPLIT-CHILD(x, i, x.children[i])
            if k > x.keys[i]
                i = i + 1
        B-TREE-INSERT-NONFULL(x.children[i], k)
```

Listing 5.4: Inserción en árbol B

5.7. Eliminación en Árbol B

5.7.1. Estrategia General

La eliminación es más compleja. Hay tres casos principales:

1. **Caso 1:** La clave está en una hoja. Se elimina directamente.
2. **Caso 2:** La clave está en un nodo interno. Se reemplaza por su predecesor o sucesor (que están en hojas), y se elimina recursivamente.
3. **Caso 3:** La clave no está en el nodo actual. Se desciende al hijo correspondiente, pero si ese hijo tiene $t - 1$ claves (mínimo), se **presta** una clave de un hermano o se **fusiona** con un hermano.

5.7.2. Prevención de Subdesbordamiento

Antes de descender a un hijo, si ese hijo tiene el mínimo de claves:

- Si un hermano tiene más de $t - 1$ claves, se **presta** una clave: se baja una clave del padre al hijo y se sube una clave del hermano al padre.
- Si ambos hermanos tienen $t - 1$ claves, se **fusiona** el hijo con un hermano, y se baja una clave del padre.

5.7.3. Pseudocódigo Simplificado

```
B-TREE-DELETE(x, k)
    i = 1
    while i <= x.n and k > x.keys[i]
        i = i + 1

    if i <= x.n and k == x.keys[i]
        // Caso 1: clave en nodo actual
        if x.leaf
```

```
            // Eliminar directamente
            for j = i to x.n-1
                x.keys[j] = x.keys[j+1]
            x.n = x.n - 1
            DISK-WRITE(x)
        else
            // Caso 2: clave en nodo interno
            y = x.children[i]   // hijo izquierdo
            z = x.children[i+1] // hijo derecho
            if y.n >= t
                // Usar predecesor
                pred = B-TREE-FIND-PREDECESSOR(y)
                x.keys[i] = pred.key
                B-TREE-DELETE(y, pred.key)
            else if z.n >= t
                // Usar sucesor
                succ = B-TREE-FIND-SUCCESSOR(z)
                x.keys[i] = succ.key
                B-TREE-DELETE(z, succ.key)
            else
                // Fusionar y, clave i, y z
                B-TREE-MERGE(x, i, y, z)
                B-TREE-DELETE(y, k)
    else
        // Caso 3: clave en sub rbol hijo
        if x.leaf
            return NIL   // no encontrada
        child = x.children[i]
        DISK-READ(child)
        if child.n == t-1
            // Necesita rebalance
            B-TREE-REBALANCE(x, i, child)
        B-TREE-DELETE(child, k)
```

Listing 5.5: Eliminación en árbol B (esquema)

5.8. Árboles B+

5.8.1. Diferencias con Árbol B

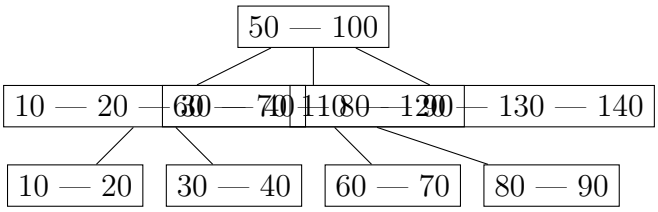
El árbol B+ es una variante optimizada para bases de datos:

1. **Solo las hojas almacenan datos.** Los nodos internos solo almacenan claves y punteros.

2. **Las hojas están enlazadas.** Forman una lista enlazada para recorridos secuenciales eficientes.

3. **Todas las claves aparecen en las hojas.** Las claves en los nodos internos son copias que guían la búsqueda.

5.8.2. Estructura de un Árbol B+



5.8.3. Ventajas del Árbol B+

- **Mayor factor de ramificación:** Los nodos internos solo almacenan claves, no datos, por lo que caben más claves por página.
- **Recorridos secuenciales eficientes:** Las hojas enlazadas permiten escanear rangos sin retroceder en el árbol.
- **Uniformidad:** Todas las búsquedas terminan en hojas, por lo que el tiempo es constante para todas las claves.

5.8.4. Búsqueda en Árbol B+

```
BPLUS-TREE-SEARCH(x, k)
    // Bajar hasta encontrar la hoja
    while not x.leaf
        i = 1
        while i <= x.n and k > x.keys[i]
            i = i + 1
        DISK-READ(x.children[i])
        x = x.children[i]

    // Buscar en la hoja
    i = 1
    while i <= x.n and k > x.keys[i]
        i = i + 1
    if i <= x.n and k == x.keys[i]
        return (x, i)   // encontrado, con datos en hoja
    else
        return NIL
```

Listing 5.6: Búsqueda en árbol B+

5.8.5. Inserción en Árbol B+

La inserción es similar al árbol B, pero:

- Siempre se inserta en la hoja.
- Cuando se divide una hoja, la clave mediana sube al padre, pero **permanece también en la hoja derecha** (para mantener todas las claves en hojas).
- Las hojas se enlazan entre sí.

5.9. Aplicaciones en Bases de Datos y Sistemas de Archivos

5.9.1. Índices en Bases de Datos

- **Índice primario:** Ordenado por la clave primaria. Las hojas contienen los registros completos o punteros a ellos.
- **Índice secundario:** Ordenado por otra clave. Las hojas contienen punteros a los registros.
- **Índice agrupado (clustered):** El orden físico de los datos sigue el orden del índice (generalmente B+).

5.9.2. Sistemas de Archivos

- **Ext4:** Usa árboles Htree (variante de B-tree) para directorios grandes.
- **NTFS:** Usa árboles B+ para el MFT (Master File Table).
- **ZFS:** Usa árboles B+ para la estructura de datos principal.

5.9.3. Optimizaciones Comunes

- **Pre-fetching:** Leer páginas anticipadamente basado en patrones de acceso.
- **Caché de nodos:** Mantener nodos frecuentemente accedidos en memoria.
- **Compresión de claves:** Reducir el espacio ocupado para aumentar el factor de ramificación.

5.10. Ejercicios Resueltos (Ampliados)

Ejercicio

Un árbol B tiene grado mínimo $t = 100$. ¿Cuántos accesos a disco se necesitan en el peor caso para buscar una clave entre 10^9 claves?

Solución

Altura máxima: $h \leq \log_t((n+1)/2) + 1 = \log_{100}(5 \times 10^8) + 1$.

$$\log_{100}(5 \times 10^8) = \frac{\ln(5 \times 10^8)}{\ln 100} \approx \frac{20{,}03}{4{,}605} \approx 4{,}35$$

Por lo tanto, $h \leq 5{,}35$, es decir, como máximo 5-6 accesos a disco.

Ejercicio

¿Cuál es el número máximo de claves en un árbol B de altura 2 con grado mínimo $t = 10$?

Solución

Máximo: raíz con $2t-1 = 19$ claves, cada uno de los 20 hijos con $2t-1 = 19$ claves. Total: $19 + 20 \times 19 = 19 + 380 = 399$ claves.

Ejercicio

Inserte las claves 10, 20, 30, 40, 50, 60, 70 en un árbol B con $t = 2$ (árbol 2-3-4).

Solución

1. Insertar 10,20,30: raíz [10,20,30] (llena).
2. Insertar 40: dividir raíz. Nueva raíz [20], hijos [10] y [30,40].
3. Insertar 50: hoja [30,40,50] llena, dividir. Subir 40. Raíz [20,40], hijos [10], [30], [50].
4. Insertar 60: hoja [50,60].
5. Insertar 70: hoja [50,60,70] llena, dividir. Subir 60. Raíz [20,40,60], hijos [10], [30], [50], [70].

Ejercicio

En un árbol B+ con $t = 100$, ¿cuántas claves caben en un nodo hoja si cada clave ocupa 8 bytes y cada puntero 8 bytes, y la página es de 4 KB?

Solución

Tamaño de página: 4096 bytes. En hoja: n claves + n punteros a datos + puntero a siguiente hoja.

$$8n + 8n + 8 = 16n + 8 \leq 4096 \implies 16n \leq 4088 \implies n \leq 255{,}5$$

Por lo tanto, $n \leq 255$ claves por hoja.

Ejercicio

¿Cuál es la altura máxima de un árbol B con $n = 10^6$ claves y $t = 50$?

Solución

$h \leq \log_{50}((10^6 + 1)/2) + 1 \approx \log_{50}(5 \times 10^5) + 1$. $\log_{50}(5 \times 10^5) = \frac{\ln(5\times10^5)}{\ln 50} \approx \frac{13{,}12}{3{,}91} \approx 3{,}35$. Por lo tanto, $h \leq 4{,}35$, es decir, 4-5 niveles.

Ejercicio

Explique por qué los árboles B+ son preferidos sobre los árboles B en bases de datos.

Solución

1. Mayor factor de ramificación en nodos internos (sin datos).
2. Recorridos secuenciales eficientes (hojas enlazadas).
3. Uniformidad en tiempo de búsqueda (siempre hasta hoja).
4. Mejor localidad de referencia para datos.

Ejercicio

En un árbol B con $t = 3$, ¿cuál es el número mínimo de claves en un nodo que no es raíz?

Solución

Mínimo: $t - 1 = 2$ claves.

Ejercicio

¿Qué sucede cuando se inserta una clave en un árbol B y la raíz está llena?

Solución

Se crea una nueva raíz vacía, se divide la raíz vieja en dos nodos, y la clave mediana sube a la nueva raíz. El árbol aumenta en altura.

Ejercicio

En un árbol B+, ¿dónde se almacenan realmente los datos?

Solución

Los datos se almacenan solo en las hojas. Los nodos internos contienen solo claves y punteros.

Ejercicio

¿Cuántos accesos a disco se necesitan para buscar una clave en un árbol B+ de altura 3?

Solución

Se necesita acceder a cada nivel: raíz (nivel 0), nodo interno (nivel 1), nodo interno (nivel 2), hoja (nivel 3). Total: 4 accesos a disco.

Ejercicio

Un árbol B tiene $t = 100$ y altura 3. ¿Cuál es el número máximo de claves que puede almacenar?

Solución

Nivel 0: $2t - 1 = 199$ claves, $2t = 200$ hijos. Nivel 1: $200 \times 199 = 39,800$ claves, $200 \times 200 = 40,000$ hijos. Nivel 2 (hojas): $40,000 \times 199 = 7,960,000$ claves. Total: $199 + 39,800 + 7,960,000 = 8,000,000$ aproximadamente.

Ejercicio

¿Qué ventaja tiene un árbol B sobre un árbol AVL en sistemas con discos?

Solución

El árbol B tiene altura mucho menor (logaritmo en base t vs. logaritmo en base 2), por lo que requiere menos accesos a disco. Además, utiliza mejor el principio de localidad.

Ejercicio

En un árbol B, ¿qué significa que un nodo está "lleno¿

Solución

Un nodo está lleno cuando tiene $2t - 1$ claves, el máximo permitido.

Ejercicio

¿Por qué los árboles B tienen un factor de ramificación grande?

Solución

Para reducir la altura del árbol y, por lo tanto, el número de accesos a disco. Cada acceso a disco es muy costoso en tiempo.

Ejercicio

En un árbol B+, ¿cómo se realiza una consulta de rango (ej. todas las claves entre 50 y 100)?

Solución

Se busca la clave 50 en el árbol, se encuentra la hoja, y se recorre la lista enlazada de hojas hasta llegar a la clave 100.

Ejercicio

¿Cuál es la diferencia entre un índice primario y uno secundario en bases de datos?

Solución

Índice primario: Ordenado por la clave primaria, las hojas contienen los registros completos. **Índice secundario:** Ordenado por otra clave, las hojas contienen punteros a los registros.

Ejercicio

En un árbol B con $t = 5$, ¿cuántas claves puede tener un nodo como máximo?

Solución

Máximo: $2t - 1 = 9$ claves.

Ejercicio

¿Qué es la .ªltura negra.ᵉⁿ árboles rojo-negro y cómo se relaciona con árboles B?

Solución

La altura negra es el número de nodos negros en un camino. Un árbol rojo-negro de altura negra b es equivalente a un árbol B de orden b si se fusionan nodos rojos con sus padres negros.

Ejercicio

En un árbol B+, ¿qué contiene la lista enlazada de hojas?

Solución

Contiene todas las claves en orden ascendente, con punteros a los datos asociados.

Ejercicio

¿Cuántas operaciones de escritura a disco se requieren para insertar una clave en un árbol B de altura 3 si no se produce ninguna división?

Solución

Se leen los 3 niveles (raíz, interno, hoja) y se escribe la hoja modificada. Total: 1 escritura.

5.11. Ejercicios Propuestos (Ampliados)

1. Construya un árbol B con $t = 2$ insertando las claves: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10.

2. Construya un árbol B con $t = 3$ insertando las claves: 10, 20, 30, 40, 50, 60, 70, 80, 90.

3. Elimine la clave 30 del árbol construido en el ejercicio 2.

4. Calcule la altura máxima de un árbol B con $n = 10^8$ y $t = 200$.
5. Diseñe un árbol B+ para almacenar 1 millón de registros con claves de 8 bytes y punteros de 8 bytes, usando páginas de 4 KB. ¿Cuántas claves por hoja? ¿Cuál es la altura?
6. Explique el algoritmo de división de un nodo en árbol B.
7. ¿Qué sucede si intentamos eliminar una clave de un árbol B y el nodo queda con menos de $t - 1$ claves?
8. Compare árboles B y B+ en términos de eficiencia de búsqueda y recorrido secuencial.
9. ¿Por qué los sistemas de archivos modernos usan árboles B+?
10. Implemente en pseudocódigo la función de búsqueda en árbol B+.
11. ¿Qué es un "índice agrupadoz cómo se implementa con árboles B+?
12. ¿Cuántos accesos a disco se necesitan para insertar 1000 claves en un árbol B de altura 3?
13. Explique la diferencia entre el grado mínimo t y el orden m en un árbol B.
14. ¿Qué es un "árbol B*z en qué se diferencia del árbol B estándar?
15. Diseñe un algoritmo para encontrar el k-ésimo elemento más pequeño en un árbol B.
16. ¿Cómo afecta el tamaño de página al rendimiento de un árbol B?
17. ¿Qué es la çompresión de clavesz cómo ayuda en árboles B?
18. Explique cómo se manejan las claves duplicadas en un árbol B+.
19. ¿Qué ventaja tiene un árbol B+ sobre un árbol B para consultas de rango?
20. Investigue: ¿qué estructura de índice usa PostgreSQL por defecto?

Unidad VI: Heaps (Montículos) - Implementación con Arreglos

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender la estructura de datos heap y su representación eficiente mediante arreglos.
- Implementar las operaciones fundamentales: MAX-HEAPIFY, BUILD-MAX-HEAP, HEAPSORT.
- Implementar operaciones de cola de prioridad: INSERT, EXTRACT-MAX, INCREASE-KEY.
- Analizar la complejidad temporal de cada operación.
- Aplicar heaps en algoritmos como Heapsort y en la implementación de colas de prioridad.

Introducción: ¿Qué es un Heap?

Definición

Un **heap** (montículo) es un árbol binario casi completo que satisface la **propiedad de heap**:

- **Max-heap:** Para todo nodo i (excepto la raíz), $A[Padre(i)] \geq A[i]$. El máximo está en la raíz.
- **Min-heap:** Para todo nodo i (excepto la raíz), $A[Padre(i)] \leq A[i]$. El mínimo está en la raíz.

Propiedades Clave

- Es un árbol binario **casi completo**: todas las filas están completas excepto posiblemente la última, que se llena de izquierda a derecha.
- Se puede representar eficientemente con un **arreglo** sin necesidad de punteros.
- Altura: $\lfloor \log_2 n \rfloor$, donde n es el número de nodos.

Aplicaciones

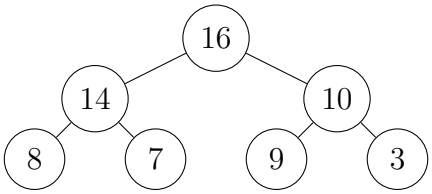
- **Heapsort:** Algoritmo de ordenación $O(n \log n)$ in-place.
- **Colas de prioridad:** Operaciones de inserción y extracción de máximo/mínimo en $O(\log n)$.
- **Algoritmos de grafos:** Dijkstra (caminos mínimos) y Prim (árbol de expansión mínimo).

Representación con Arreglos

Indexación

Dado un arreglo A de tamaño n (1-indexado), las relaciones son:

- **Raíz:** $A[1]$
- **Padre de i:** $\lfloor i/2 \rfloor$
- **Hijo izquierdo de i:** $2i$
- **Hijo derecho de i:** $2i + 1$



Índice:	1	2	3	4	5	6	7
Valor:	16	14	10	8	7	9	3

Figura 2: Un max-heap y su representación como arreglo

Ventajas de la Representación con Arreglos

- **Eficiencia espacial:** Sin punteros, solo el arreglo.
- **Localidad de referencia:** Acceso secuencial en memoria.
- **Cálculo rápido:** Relaciones padre-hijo con operaciones aritméticas simples.

Atributos

```
struct Heap {
    int* array;       // arreglo de elementos
    int size;         // n mero actual de elementos
    int capacity;     // capacidad m xima
};
```

Listing 1: Estructura de un heap

Operación Fundamental: MAX-HEAPIFY

Descripción

MAX-HEAPIFY mantiene la propiedad de max-heap. Asume que los subárboles izquierdo y derecho del nodo i son max-heaps, pero $A[i]$ puede ser menor que sus hijos.

Pseudocódigo

```
MAX-HEAPIFY(A, i, n)
    l = 2*i                 // hijo izquierdo
    r = 2*i + 1             // hijo derecho
    largest = i

    if l <= n and A[l] > A[largest]
        largest = l
    if r <= n and A[r] > A[largest]
        largest = r

    if largest != i
        swap(A[i], A[largest])
        MAX-HEAPIFY(A, largest, n)
```

Listing 2: MAX-HEAPIFY

Ejemplo Visual

Consideremos el arreglo $[16, 4, 10, 14, 7, 9, 3]$ en el nodo $i = 2$ (valor 4):

```
Antes:        16
            /    \
          4       10
        /  \    /  \
      14   7   9    3

Después:       16
            /    \
          14      10
        /  \    /  \
      4    7   9    3
```

Análisis de Complejidad

- Cada llamada recursiva desciende un nivel.
- Altura del heap: $\lfloor \log_2 n \rfloor$.
- Tiempo: $O(\log n)$.

Construcción de un Heap: BUILD-MAX-HEAP

Descripción

Convierte un arreglo desordenado en un max-heap. Aprovecha que las hojas (índices $\lfloor n/2 \rfloor + 1$ hasta n) ya son max-heaps triviales.

Pseudocódigo

```
BUILD-MAX-HEAP(A, n)
    for i = floor(n/2) downto 1
        MAX-HEAPIFY(A, i, n)
```

Listing 3: BUILD-MAX-HEAP

Ejemplo

Construir max-heap desde $[4, 1, 3, 2, 16, 9, 10, 14, 8, 7]$:

1. $n = 10$, $\lfloor 10/2 \rfloor = 5$. Procesar $i = 5$ (valor 16): ya es heap.
2. $i = 4$ (valor 2): MAX-HEAPIFY intercambia con 14. Resultado: $[4, 1, 3, 14, 16, 9, 10, 2, 8, 7]$.
3. $i = 3$ (valor 3): intercambia con 10. $[4, 1, 10, 14, 16, 9, 3, 2, 8, 7]$.
4. $i = 2$ (valor 1): intercambia con 16. $[4, 16, 10, 14, 1, 9, 3, 2, 8, 7]$, luego con 14. $[4, 16, 10, 14, 1, 9, 3, 2, 8$ Revisar.
5. $i = 1$ (valor 4): intercambia con 16. $[16, 4, 10, 14, 1, 9, 3, 2, 8, 7]$, luego con 14. $[16, 14, 10, 4, 1, 9, 3, 2, 8$

Análisis de Complejidad

- Análisis simple: $O(n \log n)$ (n llamadas a MAX-HEAPIFY).
- Análisis ajustado: $O(n)$ porque la mayoría de los nodos están en la parte baja del árbol.

Teorema 0.1. *BUILD-MAX-HEAP tiene complejidad $O(n)$.*

Demostración. La altura de un nodo i es h_i. MAX-HEAPIFY en un nodo de altura h toma $O(h)$. El número de nodos de altura h es a lo sumo $\lceil n/2^{h+1} \rceil$.

$$\sum_{h=0}^{\lfloor \log n \rfloor} \lceil n/2^{h+1} \rceil \cdot O(h) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^{h+1}} \right) = O(n)$$

La suma converge a 2. $\square$

Algoritmo HEAPSORT

Descripción

Heapsort ordena un arreglo en tres pasos:

1. Construir un max-heap con BUILD-MAX-HEAP.
2. Intercambiar la raíz (máximo) con el último elemento.
3. Reducir el tamaño del heap y llamar MAX-HEAPIFY en la nueva raíz.
4. Repetir hasta que el heap tenga un solo elemento.

Pseudocódigo

```
HEAPSORT(A, n)
    BUILD-MAX-HEAP(A, n)
    for i = n downto 2
        swap(A[1], A[i])
        MAX-HEAPIFY(A, 1, i-1)
```

Listing 4: HEAPSORT

Ejemplo

Ordenar $[4, 1, 3, 2, 16, 9, 10, 14, 8, 7]$:

1. Construir max-heap: $[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$
2. Intercambiar 16 con 1: $[1, 14, 10, 8, 7, 9, 3, 2, 4, 16]$
3. MAX-HEAPIFY en raíz: $[14, 8, 10, 4, 7, 9, 3, 2, 1, 16]$
4. Intercambiar 14 con 1: $[1, 8, 10, 4, 7, 9, 3, 2, 14, 16]$
5. ... continuar hasta obtener $[1, 2, 3, 4, 7, 8, 9, 10, 14, 16]$

Análisis de Complejidad

- BUILD-MAX-HEAP: $O(n)$.
- Bucle: $n - 1$ iteraciones.
- Cada iteración: $O(\log n)$ por MAX-HEAPIFY.
- Total: $O(n \log n)$.

Colas de Prioridad con Heaps

Operaciones Básicas

Un max-heap puede implementar una cola de prioridad máxima con:

- **MAXIMUM:** Devuelve la raíz. $O(1)$.
- **EXTRACT-MAX:** Elimina y devuelve el máximo. $O(\log n)$.
- **INCREASE-KEY:** Aumenta la clave de un elemento. $O(\log n)$.
- **INSERT:** Inserta un nuevo elemento. $O(\log n)$.

HEAP-MAXIMUM

```
HEAP-MAXIMUM(A)
    return A[1]
```

Listing 5: HEAP-MAXIMUM

HEAP-EXTRACT-MAX

```
HEAP-EXTRACT-MAX(A, n)
    if n < 1
        error "heap underflow"
    max = A[1]
    A[1] = A[n]
    n = n - 1
    MAX-HEAPIFY(A, 1, n)
    return max
```

Listing 6: HEAP-EXTRACT-MAX

HEAP-INCREASE-KEY

```
HEAP-INCREASE-KEY(A, i, key)
    if key < A[i]
        error "new key is smaller"
    A[i] = key
    while i > 1 and A[parent(i)] < A[i]
        swap(A[i], A[parent(i)])
        i = parent(i)
```

Listing 7: HEAP-INCREASE-KEY

MAX-HEAP-INSERT

```
MAX-HEAP-INSERT(A, n, key)
    n = n + 1
    A[n] = -infinity
    HEAP-INCREASE-KEY(A, n, key)
```

Listing 8: MAX-HEAP-INSERT

Min-Heaps

Definición

Un min-heap es análogo al max-heap pero con la propiedad invertida: $A[Padre(i)] \leq A[i]$. El mínimo está en la raíz.

Operaciones Simétricas

- **MIN-HEAPIFY:** Similar a MAX-HEAPIFY pero con comparación $\leq$.
- **BUILD-MIN-HEAP:** Idéntico a BUILD-MAX-HEAP pero usando MIN-HEAPIFY.
- **HEAP-MINIMUM:** Devuelve $A[1]$.
- **HEAP-EXTRACT-MIN:** Extrae la raíz.
- **HEAP-DECREASE-KEY:** Disminuye una clave.

Pseudocódigo de MIN-HEAPIFY

```
MIN-HEAPIFY(A, i, n)
    l = 2*i
    r = 2*i + 1
    smallest = i

    if l <= n and A[l] < A[smallest]
        smallest = l
    if r <= n and A[r] < A[smallest]
        smallest = r

    if smallest != i
        swap(A[i], A[smallest])
        MIN-HEAPIFY(A, smallest, n)
```

Listing 9: MIN-HEAPIFY

Aplicaciones Prácticas

Heapsort en la Práctica

- Es un algoritmo de ordenación $O(n \log n)$ **in-place**.
- No es estable (no preserva el orden relativo de elementos iguales).
- En la práctica, Quicksort suele ser más rápido, pero Heapsort tiene mejor comportamiento en el peor caso.

Colas de Prioridad en Sistemas Operativos

- **Planificación de procesos:** Los procesos con mayor prioridad se ejecutan primero.
- **Manejo de interrupciones:** Las interrupciones con mayor prioridad se atienden primero.

Algoritmos de Grafos

- **Dijkstra:** Encuentra caminos más cortos usando una cola de prioridad.
- **Prim:** Encuentra el árbol de expansión mínimo.

Ejercicios Resueltos (Ampliados)

Ejercicio

Dado el arreglo $[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$, verifique si es un max-heap.

Solución

Verificamos cada nodo (excepto hojas):

- $i = 1$: $16 \geq 14$ y $16 \geq 10$
- $i = 2$: $14 \geq 8$ y $14 \geq 7$
- $i = 3$: $10 \geq 9$ y $10 \geq 3$
- $i = 4$: $8 \geq 2$ y $8 \geq 4$
- $i = 5$: $7 \geq 1$

Es un max-heap válido.

Ejercicio

Aplique MAX-HEAPIFY al arreglo $[3, 10, 5, 8, 7, 6, 9]$ en $i = 1$.

Solución

1. $i = 1$, $l = 2$ (10), $r = 3$ (5). Mayor es 10 (índice 2).
2. Intercambiar $A[1]$ y $A[2]$: $[10, 3, 5, 8, 7, 6, 9]$.
3. Llamar MAX-HEAPIFY en $i = 2$ (valor 3).
4. $i = 2$, $l = 4$ (8), $r = 5$ (7). Mayor es 8 (índice 4).
5. Intercambiar $A[2]$ y $A[4]$: $[10, 8, 5, 3, 7, 6, 9]$.
6. Llamar MAX-HEAPIFY en $i = 4$ (valor 3). Sus hijos son 8 y 9 (índices 8 y 9, fuera de rango). Fin.

Resultado: $[10, 8, 5, 3, 7, 6, 9]$.

Ejercicio

Construya un max-heap desde $[5, 3, 17, 10, 84, 19, 6, 22, 9]$.

Solución

$n = 9$, $\lfloor 9/2 \rfloor = 4$.

1. $i = 4$ (10): hijos 8 (22) y 9 (9). Mayor es 22, intercambiar: $[5, 3, 17, 22, 84, 19, 6, 10, 9]$.
2. $i = 3$ (17): hijos 6 (19) y 7 (6). Mayor es 19, intercambiar: $[5, 3, 19, 22, 84, 17, 6, 10, 9]$.
3. $i = 2$ (3): hijos 4 (22) y 5 (84). Mayor es 84, intercambiar con 3: $[5, 84, 19, 22, 3, 17, 6, 10, 9]$. Luego verificar $i = 5$ (3): hijos 10 y 11 fuera.
4. $i = 1$ (5): hijos 2 (84) y 3 (19). Mayor es 84, intercambiar: $[84, 5, 19, 22, 3, 17, 6, 10, 9]$. Luego MAX-HEAPIFY en $i = 2$ (5): hijos 4 (22) y 5 (3). Mayor 22, intercambiar: $[84, 22, 19, 5, 3, 17, 6, 10, 9]$. Luego MAX-HEAPIFY en $i = 4$ (5): hijos 8 (10) y 9 (9). Mayor 10, intercambiar: $[84, 22, 19, 10, 3, 17, 6, 5, 9]$.

Resultado: $[84, 22, 19, 10, 3, 17, 6, 5, 9]$.

Ejercicio

Ordene $[5, 13, 2, 25, 7, 17, 20, 8, 4]$ usando Heapsort.

Solución

1. Construir max-heap: proceso similar al ejercicio anterior.
2. Intercambiar raíz con último, reducir tamaño, MAX-HEAPIFY.
3. Repetir hasta ordenar.

Resultado final: $[2, 4, 5, 7, 8, 13, 17, 20, 25]$.

Ejercicio

Demuestre que en un max-heap, la raíz es el elemento máximo.

Solución

Por la propiedad de max-heap, $A[Padre(i)] \geq A[i]$ para todo i. Por transitividad, la raíz $A[1]$ es mayor o igual que cualquier otro nodo.

Ejercicio

¿Cuál es la altura de un heap con 100 elementos?

Solución

Altura $= \lfloor \log_2 100 \rfloor = \lfloor 6{,}64 \rfloor = 6$.

Ejercicio

Demuestre que BUILD-MAX-HEAP tiene complejidad $O(n)$.

Solución

La suma $\sum_{h=0}^{\lfloor \log n \rfloor} \lceil n/2^{h+1} \rceil \cdot O(h) \leq O(n \sum_{h=0}^{\infty} h/2^h) = O(n)$ porque $\sum_{h=0}^{\infty} h/2^h = 2$.

Ejercicio

Implemente HEAP-DELETE para eliminar un elemento en posición i.

Solución

```
HEAP-DELETE(A, n, i)
    if i == n
        n = n - 1
        return
    A[i] = A[n]
    n = n - 1
    if A[i] > A[parent(i)]
        HEAP-INCREASE-KEY(A, i, A[i])
    else
        MAX-HEAPIFY(A, i, n)
```

Ejercicio

¿Cuál es la diferencia entre un max-heap y un min-heap?

Solución

En max-heap, $A[Padre] \geq A[Hijo]$; en min-heap, $A[Padre] \leq A[Hijo]$. El máximo está en la raíz del max-heap; el mínimo en la raíz del min-heap.

Ejercicio

¿Cuántos nodos tiene un heap completo de altura h?

Solución

Un heap completo de altura h tiene $2^{h+1} - 1$ nodos.

Ejercicio

En un max-heap, ¿dónde puede estar el elemento mínimo?

Solución

En las hojas (último nivel). Puede estar en cualquier hoja.

Ejercicio

¿Qué operaciones de heap se usan en Heapsort?

Solución

BUILD-MAX-HEAP y MAX-HEAPIFY.

Ejercicio

¿Cuál es la complejidad de HEAP-EXTRACT-MAX?

Solución

$O(\log n)$ por la llamada a MAX-HEAPIFY.

Ejercicio

¿Qué sucede si llamamos MAX-HEAPIFY en un nodo hoja?

Solución

No hace nada porque los hijos no existen, y termina inmediatamente.

Ejercicio

¿Cuántas comparaciones se hacen en el peor caso en MAX-HEAPIFY?

Solución

En cada nivel, se comparan dos hijos (2 comparaciones). La profundidad es $\log n$, total $O(\log n)$ comparaciones.

Ejercicio

¿Puede un heap estar representado como un árbol binario no completo?

Solución

No, por definición es un árbol binario casi completo. La representación con arreglos requiere que los nodos estén en posiciones contiguas.

Ejercicio

Demuestre que en un heap de n nodos, hay $\lceil n/2 \rceil$ hojas.

Solución

Las hojas son los nodos desde $\lfloor n/2 \rfloor + 1$ hasta n. El número es $n - \lfloor n/2 \rfloor = \lceil n/2 \rceil$.

Ejercicio

¿Cuál es la complejidad de HEAP-INCREASE-KEY en el mejor caso?

Solución

$O(1)$, si la clave ya es mayor que su padre.

Ejercicio

Implemente HEAP-MERGE para fusionar dos heaps.

Solución

```
HEAP-MERGE(A, n1, B, n2)
    C = array of size n1 + n2
    copy A and B into C
    BUILD-MAX-HEAP(C, n1 + n2)
    return C
```

Complejidad: $O(n_1 + n_2)$.

Ejercicio

¿Por qué Heapsort no es un algoritmo estable?

Solución

Porque el intercambio de elementos puede alterar el orden relativo de elementos iguales.

Ejercicio

¿Cuál es la ventaja de Heapsort sobre Merge Sort?

Solución

Heapsort ordena in-place (no requiere memoria adicional), mientras que Merge Sort requiere $O(n)$ espacio auxiliar.

Ejercicios Propuestos (Ampliados)

1. Construya un max-heap desde $[10, 20, 15, 30, 40, 50, 25, 35, 45, 5]$.
2. Construya un min-heap desde $[8, 3, 5, 1, 4, 9, 2, 7, 6]$.
3. Ordene $[3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]$ usando Heapsort.
4. Demuestre que la altura de un heap de n nodos es $\lfloor \log_2 n \rfloor$.
5. Implemente HEAP-DELETE para eliminar el elemento en posición i.

6. ¿Cuál es la complejidad de construir un heap insertando elementos uno a uno (con INSERT)?
7. Explique cómo usar un min-heap para implementar una cola de prioridad mínima.
8. ¿Qué es un "heap binarioz cómo se diferencia de un "heap binomial¿
9. Demuestre que el número de hojas en un heap de n nodos es $\lceil n/2 \rceil$.
10. Implemente HEAP-EXTRACT-MAX iterativamente (sin recursión).
11. ¿Cuántos nodos tiene un heap completo de altura h? ¿Y uno casi completo?
12. ¿Puede un heap tener nodos con el mismo valor? ¿Cómo afecta esto a la propiedad?
13. Implemente HEAP-SORT para ordenar en orden descendente usando un min-heap.
14. ¿Cuál es el peor caso de Heapsort? ¿Y el mejor caso?
15. Demuestre que BUILD-MAX-HEAP es $O(n)$ usando el método de agregación.
16. ¿Qué es la "propiedad de heapz por qué es importante?
17. Implemente una cola de prioridad usando heaps en C o Python.
18. ¿Cómo se puede encontrar el k-ésimo elemento más pequeño usando un heap?
19. Explique la relación entre heaps y árboles de torneo.
20. ¿Qué es un "d-ary heapz cuándo es útil?

Unidad VII: Hashing (Tablas Hash)

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender el concepto de tablas hash y su aplicación en diccionarios.
- Diseñar y analizar funciones hash efectivas.
- Implementar resolución de colisiones mediante encadenamiento y direccionamiento abierto.
- Analizar el rendimiento esperado de las operaciones de hash.
- Aplicar hashing en problemas reales como búsqueda de elementos y conteo de frecuencias.

Introducción: El Problema del Acceso Directo

El Ideal: Direccionamiento Directo

Si tenemos un universo U de claves pequeño (ej. $\{0, 1, \ldots, m-1\}$), podemos usar una **tabla de direccionamiento directo**:

- Arreglo $T[0..m-1]$.
- Insertar clave k en $T[k]$.
- Buscar clave k en $T[k]$.
- Eliminar clave k de $T[k]$.

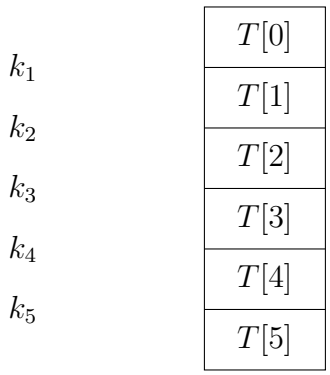
Todas las operaciones: $O(1)$ en el peor caso.

El Problema

- ¿Qué pasa si U es enorme (ej. $U = \{0, 1, \ldots, 2^{64} - 1\}$)?
- No podemos asignar un arreglo de tamaño $|U|$.
- Necesitamos una función que **comprima** el universo a un rango manejable.

Solución: Tablas Hash

Una **tabla hash** es un arreglo $T[0..m-1]$ donde m es mucho menor que $|U|$. Una **función hash** $h : U \to \{0, 1, \ldots, m-1\}$ asigna cada clave a una posición.



El Problema de las Colisiones

Como $|U| > m$, por el principio del palomar, existen al menos dos claves $k_1 \neq k_2$ con $h(k_1) = h(k_2)$. Esto se llama **colisión**. Necesitamos métodos para resolverlas.

Funciones Hash

¿Qué hace una buena función hash?

- **Hash uniforme simple:** Cada clave tiene la misma probabilidad de hashear a cualquier slot, independientemente de otras claves.
- **Determinista:** $h(k)$ debe ser el mismo cada vez.
- **Rápida de calcular:** Idealmente $O(1)$.
- **Dispersión:** Claves similares deben hashear a slots diferentes.

Interpretar Claves como Números Naturales

La mayoría de las funciones hash asumen que las claves son números naturales. Si no lo son, debemos convertirlas:

- **Cadenas:** Interpretar como número en base 128 (ASCII) o 256 (bytes).
- **Floats:** Interpretar su representación binaria como entero.
- **Objetos:** Combinar hashes de sus componentes.

Método de la División

$$h(k) = k \text{ mód } m$$

- **Ventaja:** Muy rápida (una operación de módulo).

- **Desventaja:** Requiere cuidado al elegir m.
- **Elección de m:**
  - Evitar potencias de 2 ($m = 2^p$): $h(k)$ son los p bits menos significativos.
  - Evitar $m = 2^p - 1$ si k es cadena en base 2^p: permutaciones de caracteres tienen el mismo hash.
  - **Bueno:** m primo no cercano a potencia de 2.

Ejemplo

Para $m = 701$ (primo), $h(100) = 100 \text{ mód } 701 = 100$, $h(702) = 702 \text{ mód } 701 = 1$.

Método de la Multiplicación

$$h(k) = \lfloor m(kA \text{ mód } 1) \rfloor$$

donde $0 < A < 1$ es una constante.

- **Ventaja:** El valor de m no es crítico.
- **Elección de A:** Knuth sugiere $A \approx (\sqrt{5} - 1)/2 \approx 0{,}6180339887$.
- **Implementación eficiente:** Si $m = 2^p$, usar aritmética de enteros.

Ejemplo

Para $k = 123456$, $p = 14$, $m = 2^{14} = 16384$, $A = 2654435769/2^{32}$:

- $k \cdot A \approx 0{,}17612864$
- $m \cdot (kA \text{ mód } 1) = 16384 \times 0{,}17612864 \approx 2886{,}6$
- $h(k) = 2886$

Hash Universal

Definición 0.1. *Una colección $\mathcal{H}$ de funciones hash es **universal** si para cualquier par de claves distintas k, l:*

$$\Pr_{h \in \mathcal{H}}[h(k) = h(l)] \leq \frac{1}{m}$$

Construcción Universal

Sea p un primo mayor que cualquier clave. Definimos:

$$h_{a,b}(k) = ((ak + b) \text{ mód } p) \text{ mód } m$$

donde $a \in \{1, 2, \ldots, p-1\}$, $b \in \{0, 1, \ldots, p-1\}$.

Teorema 0.2. *La familia $\mathcal{H} = \{h_{a,b} : 1 \leq a \leq p-1, 0 \leq b \leq p-1\}$ es universal.*

Esbozo. Para $k \neq l$, $(ak + b) \text{ mód } p$ y $(al + b) \text{ mód } p$ son independientes y uniformes. La probabilidad de que difieran en un múltiplo de m es a lo sumo $1/m$. □

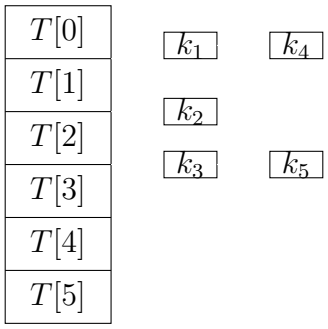
Otras Funciones Hash

- **Hash de cadenas (Polinomial):** $h(s) = \sum_{i=0}^{n-1} s[i] \cdot B^{n-1-i} \text{ mód } m$, con B primo.
- **Hash criptográfico:** MD5, SHA-1, SHA-256 (más lentos, pero resistentes a colisiones maliciosas).
- **Hash con desplazamiento:** $h(k) = (k \oplus constante) \text{ mód } m$ (rápido, pero menos disperso).

Resolución de Colisiones por Encadenamiento

Descripción

Cada slot $T[j]$ contiene una lista enlazada de todos los elementos que hashean a j.



Operaciones

```
CHAINED-HASH-INSERT(T, x)
    // insertar al inicio de la lista
    x.next = T[h(x.key)]
    T[h(x.key)] = x

CHAINED-HASH-SEARCH(T, k)
    x = T[h(k)]
    while x != NIL and x.key != k
        x = x.next
    return x

CHAINED-HASH-DELETE(T, x)
    // asumiendo lista doblemente enlazada
    if x.prev != NIL
        x.prev.next = x.next
    else
        T[h(x.key)] = x.next
    if x.next != NIL
        x.next.prev = x.prev
```

Listing 10: Encadenamiento

Análisis de Rendimiento

Sea $\alpha = n/m$ el **factor de carga** (número promedio de elementos por lista).

Teorema 0.3. *En una tabla hash con encadenamiento y hash uniforme simple:*

- *Una búsqueda fallida toma tiempo esperado* $\Theta(1 + \alpha)$.
- *Una búsqueda exitosa toma tiempo esperado* $\Theta(1 + \alpha)$.

Esbozo. Para búsqueda fallida, el número esperado de elementos en la lista es α. Para búsqueda exitosa, se examina 1 + (número de elementos insertados después): esperado $\alpha/2$. □

Si $n = O(m)$, entonces $\alpha = O(1)$ y las operaciones son $O(1)$ esperado.

Resolución de Colisiones por Direccionamiento Abierto

Descripción

Todos los elementos se almacenan en la propia tabla. Cada slot contiene un elemento o NIL. Para insertar, se sondean slots hasta encontrar uno vacío.

Secuencia de Sondas

La función hash se extiende a $h : U \times \{0, 1, \ldots, m-1\} \to \{0, 1, \ldots, m-1\}$. Para cada clave k, la secuencia $\langle h(k, 0), h(k, 1), \ldots, h(k, m-1)\rangle$ es una permutación de $\{0, \ldots, m-1\}$.

Sondeo Lineal

$$h(k, i) = (h'(k) + i) \text{ mód } m$$

- **Ventaja:** Simple, buena localidad de referencia.
- **Desventaja: Agrupamiento primario**: largas secuencias de slots ocupados se forman, aumentando el tiempo de búsqueda.

Sondeo Cuadrático

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \text{ mód } m$$

- **Ventaja:** Mejor dispersión que lineal.
- **Desventaja: Agrupamiento secundario**: si dos claves tienen el mismo $h'(k)$, sus secuencias son iguales.

Doble Hashing

$$h(k,i) = (h_1(k) + i \cdot h_2(k)) \text{ mód } m$$

- **Ventaja:** Excelente dispersión, se aproxima al hash uniforme.
- **Requisito:** $h_2(k)$ debe ser relativamente primo con m (para que la secuencia sea una permutación).

Ejemplo

$m = 13$, $h_1(k) = k$ mód 13, $h_2(k) = 1 + (k$ mód $11)$. Para $k = 14$: $h_1 = 1$, $h_2 = 1 + 3 = 4$. Secuencia: $1, 5, 9, 0, 4, 8, 12, 3, 7, 11, 2, 6, 10$.

Operaciones

```
HASH-INSERT(T, k)
    i = 0
    repeat
        j = h(k, i)
        if T[j] == NIL
            T[j] = k
            return j
        i = i + 1
    until i == m
    error "table overflow"

HASH-SEARCH(T, k)
    i = 0
    repeat
        j = h(k, i)
        if T[j] == k
            return j
        i = i + 1
    until T[j] == NIL or i == m
    return NIL
```

Listing 11: Direccionamiento Abierto

Análisis de Rendimiento

Teorema 0.4. *En una tabla hash con direccionamiento abierto y hash uniforme, con factor de carga $\alpha < 1$:*

- *Número esperado de sondas en búsqueda fallida:* $\leq \frac{1}{1-\alpha}$.
- *Número esperado de sondas en búsqueda exitosa:* $\leq \frac{1}{\alpha} \ln \frac{1}{1-\alpha}$.

α	Búsqueda fallida	Búsqueda exitosa
0.50	2.00	1.39
0.75	4.00	2.08
0.90	10.00	2.56
0.95	20.00	3.15

Eliminación en Direccionamiento Abierto

La eliminación es problemática porque si marcamos un slot como NIL, se pueden perder claves posteriores en la secuencia de sonda.

Solución: Marcador DELETED

- Usar un valor especial DELETED.
- HASH-SEARCH trata DELETED como ocupado (continúa).
- HASH-INSERT trata DELETED como vacío (puede insertar).
- Desventaja: El rendimiento se degrada con muchas eliminaciones.

Hash Perfecto

Definición

Para conjuntos **estáticos** (sin inserciones ni eliminaciones), podemos construir un hash perfecto:

- **Hash perfecto:** Sin colisiones.
- **Hash mínimo perfecto:** Sin colisiones y $m = n$.

Estrategia de Dos Niveles

1. Primera función hash h a $m = n$ slots.
2. Para cada slot j con n_j elementos, usar segunda función hash h_j a $m_j = n_j^2$ slots.
3. Con probabilidad alta, no hay colisiones en el segundo nivel.
4. Espacio total esperado: $O(n)$.

Ejercicios Resueltos (Ampliados)

Ejercicio

Sea $m = 11$ y $h(k) = k$ mód 11. Inserte las claves 10, 22, 31, 4, 15, 28, 17, 88, 59 en una tabla con encadenamiento.

Solución

- $h(10) = 10$, $h(22) = 0$, $h(31) = 9$, $h(4) = 4$, $h(15) = 4$, $h(28) = 6$, $h(17) = 6$, $h(88) = 0$, $h(59) = 4$.
- Slot 0: 22, 88
- Slot 4: 4, 15, 59
- Slot 6: 28, 17
- Slot 9: 31
- Slot 10: 10

Ejercicio

Con $m = 10$, $h(k) = k$ mód 10, inserte 5, 15, 25, 35, 45. ¿Qué problema observa?

Solución

Todas hashean a 5. Es un mal ejemplo porque m es par y todas las claves son múltiplos de 5. m debería ser primo.

Ejercicio

Use el método de la multiplicación con $A = 0{,}618$, $m = 1000$, $k = 123456$. Calcule $h(k)$.

Solución

$kA = 123456 \times 0{,}618 \approx 76300{,}608$. kA mód $1 \approx 0{,}608$. $m \times 0{,}608 \approx 608$. $h(k) = 608$.

Ejercicio

Construya una familia universal de funciones hash para claves en $\{0, \ldots, 1000\}$ con $m = 100$.

Solución

Elija $p = 1009$ (primo ¿1000). Para $a \in \{1, \ldots, 1008\}$, $b \in \{0, \ldots, 1008\}$, $h_{a,b}(k) = ((ak + b) \text{ mód } 1009) \text{ mód } 100$.

Ejercicio

Demuestre que el número esperado de colisiones en una tabla hash con encadenamiento es $\binom{n}{2}/m$.

Solución

Sea $X_{ij} = I\{h(k_i) = h(k_j)\}$. $E[X_{ij}] = 1/m$. Hay $\binom{n}{2}$ pares. $E[\text{colisiones}] = \binom{n}{2}/m$.

Ejercicio

En direccionamiento abierto con $\alpha = 0{,}75$, ¿cuál es el número esperado de sondas para búsqueda fallida?

Solución

$1/(1-\alpha) = 1/0{,}25 = 4$ sondas esperadas.

Ejercicio

¿Por qué el doble hashing requiere que $h_2(k)$ sea relativamente primo con m?

Solución

Si $d = \gcd(h_2(k), m) > 1$, la secuencia solo visita m/d slots, no todos.

Ejercicio

¿Qué es el factor de carga y por qué debe ser menor que 1 en direccionamiento abierto?

Solución

$\alpha = n/m$. En direccionamiento abierto, la tabla se llena completamente cuando $n = m$, por lo que α debe ser < 1 para que haya espacio para insertar.

Ejercicio

Calcule la probabilidad de que dos claves colisionen en una tabla de tamaño m si h es uniforme.

Solución

$1/m$.

Ejercicio

¿Cuál es la complejidad esperada de búsqueda en una tabla hash con encadenamiento si $n = 2m$?

Solución

$\alpha = 2$, esperado $\Theta(1+\alpha) = \Theta(3) = O(1)$.

Ejercicio

Diseñe una función hash para cadenas usando el método polinomial con $B = 131$.

Solución

```
hash_string(s, m)
    h = 0
    for each char c in s
        h = (h * 131 + c) mod m
    return h
```

Ejercicio

¿Qué es el .ªgrupamiento primario.en sondeo lineal?

Solución

Es la tendencia a formar largas secuencias contiguas de slots ocupados, que aumentan el tiempo de búsqueda.

Ejercicio

Demuestre que el método de la división con $m = 2^p$ es malo porque $h(k)$ ignora los bits más significativos.

Solución

k mód 2^p son los p bits menos significativos. Claves que difieren solo en bits altos colisionan.

Ejercicio

¿Cuántas funciones hay en la familia universal $\mathcal{H}_{p,m}$?

Solución

$p(p-1)$ funciones (p opciones para b, $p-1$ para a).

Ejercicio

En doble hashing con $m = 13$, $h_1(k) = k$ mód 13, $h_2(k) = 1+(k$ mód $11)$, encuentre la secuencia para $k = 42$.

Solución

$h_1 = 42$ mód $13 = 3$, $h_2 = 1 + (42$ mód $11) = 1 + 9 = 10$. Secuencia: $3, 0, 10, 7, 4, 1, 11, 8, 5, 2, 12, 9, 6$.

Ejercicio

¿Qué es el hash perfecto y cuándo se usa?

Solución

Hash sin colisiones. Se usa para conjuntos estáticos (ej. palabras reservadas de un lenguaje, tablas de símbolos fijas).

Ejercicio

Calcule el número esperado de sondas para búsqueda exitosa con $\alpha = 0{,}9$ en direccionamiento abierto.

Solución

$\frac{1}{\alpha} \ln \frac{1}{1-\alpha} = \frac{1}{0{,}9} \ln \frac{1}{0{,}1} \approx 1{,}111 \times 2{,}303 \approx 2{,}56$.

Ejercicio

¿Por qué el sondeo cuadrático no visita todos los slots si m no es primo?

Solución

Puede haber ciclos cortos. Con $m = 2^p$, la secuencia es periódica con período $m/2$.

Ejercicio

¿Cuál es la probabilidad de que en una tabla de tamaño m con n claves, un slot específico esté vacío?

Solución

$(1 - 1/m)^n$.

Ejercicios Propuestos (Ampliados)

1. Inserte las claves 5, 28, 19, 15, 20, 33, 12, 17, 10 en una tabla hash con $m = 9$ y $h(k) = k \text{ mód } 9$ usando encadenamiento.
2. Inserte las mismas claves usando direccionamiento abierto con sondeo lineal.
3. Inserte las mismas claves usando doble hashing con $h_1(k) = k \text{ mód } 9$ y $h_2(k) = 1 + (k \text{ mód } 8)$.
4. Diseñe una función hash universal para claves de 4 bytes.
5. Demuestre que el número esperado de colisiones es $\binom{n}{2}/m$.
6. ¿Cuál es el factor de carga óptimo para una tabla hash con encadenamiento?
7. Implemente una tabla hash con encadenamiento en C.
8. Analice el rendimiento de una tabla hash cuando todas las claves hashean al mismo slot.

9. ¿Qué es un "hash criptográfico" en qué se diferencia de un hash de tabla?
10. Calcule la probabilidad de que al menos dos claves colisionen en una tabla de tamaño m con n claves (paradoja del cumpleaños).
11. Explique cómo manejar claves duplicadas en una tabla hash.
12. ¿Qué es el rehashing? cuándo es necesario?
13. Diseñe una función hash para números de punto flotante.
14. Implemente un diccionario con tablas hash en Python.
15. ¿Cuál es la complejidad esperada de INSERT en una tabla hash con encadenamiento?
16. ¿Qué es el "hash con direccionamiento abierto" por qué es más eficiente en espacio?
17. Compare las ventajas y desventajas de encadenamiento vs. direccionamiento abierto.
18. ¿Qué es el "hash perfecto mínimo" cómo se construye?
19. Investigue cómo se implementan los diccionarios en Python (CPython).
20. ¿Cómo afecta el tamaño de la tabla al rendimiento del hash?

Unidad VIII: Métodos de Resolución de Colisiones

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender la naturaleza y causas de las colisiones en tablas hash.
- Implementar y analizar la resolución de colisiones por encadenamiento.
- Implementar y analizar las variantes de direccionamiento abierto: lineal, cuadrático y doble hashing.
- Comparar el rendimiento de los diferentes métodos bajo distintas condiciones.
- Seleccionar el método apropiado según los requisitos de la aplicación.

Introducción: El Problema de las Colisiones

¿Por qué ocurren las colisiones?

Dado que el universo U de claves es mucho mayor que el tamaño m de la tabla, por el **principio del palomar**, existen al menos dos claves distintas $k_1 \neq k_2$ tales que $h(k_1) = h(k_2)$. Esto es inevitable.

Enfoques Fundamentales

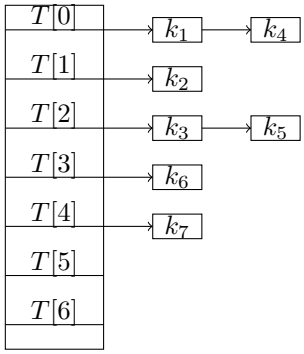
Hay dos grandes familias de métodos para resolver colisiones:

1. **Encadenamiento (Chaining):** Cada slot contiene una estructura (lista, árbol) que almacena múltiples elementos.
2. **Direccionamiento Abierto (Open Addressing):** Los elementos se almacenan en la propia tabla, buscando slots alternativos.

Resolución por Encadenamiento

Descripción General

En el encadenamiento, cada posición de la tabla $T[j]$ contiene un puntero al inicio de una lista enlazada (o cualquier otra estructura) que almacena todas las claves que hashean a j.



Implementación Detallada

Estructura de Nodo

```
struct Node {
    int key;
    void* data;
    struct Node* next;
    struct Node* prev;  // para eliminaci n O(1)
};

struct HashTable {
    struct Node** table;  // arreglo de punteros a listas
    int size;             // m
    int count;            // n
};
```

Listing 1: Estructura de nodo para encadenamiento

Inserción

```
void insert(HashTable* ht, int key, void* data) {
    int index = hash(key, ht->size);
    Node* new_node = malloc(sizeof(Node));
    new_node->key = key;
    new_node->data = data;
    new_node->next = ht->table[index];
```

```
    new_node->prev = NULL;

    if (ht->table[index] != NULL)
        ht->table[index]->prev = new_node;

    ht->table[index] = new_node;
    ht->count++;
}
```

Listing 2: Inserción con encadenamiento

Búsqueda

```
Node* search(HashTable* ht, int key) {
    int index = hash(key, ht->size);
    Node* current = ht->table[index];

    while (current != NULL && current->key != key)
        current = current->next;

    return current;
}
```

Listing 3: Búsqueda con encadenamiento

Eliminación

```
void delete(HashTable* ht, int key) {
    int index = hash(key, ht->size);
    Node* current = ht->table[index];

    while (current != NULL && current->key != key)
        current = current->next;

    if (current == NULL) return;  // no encontrado

    // Reajustar punteros
    if (current->prev != NULL)
        current->prev->next = current->next;
    else
        ht->table[index] = current->next;

    if (current->next != NULL)
        current->next->prev = current->prev;

    free(current);
    ht->count--;
}
```

Listing 4: Eliminación con encadenamiento

Variantes de Encadenamiento

- **Listas ordenadas:** Búsqueda más rápida (se puede detener antes), pero inserción más lenta.
- **Árboles balanceados:** Peor caso $O(\log n)$ en lugar de $O(n)$, útil para tablas muy cargadas.
- **Listas con cabecera:** Simplifica la inserción al inicio.

Análisis Detallado

Factor de Carga

$$\alpha = \frac{n}{m}$$

Tiempo Esperado de Búsqueda

Teorema 0.0.1. *Bajo hash uniforme simple:*

- *Búsqueda fallida:* $E[T] = \Theta(1 + \alpha)$
- *Búsqueda exitosa:* $E[T] = \Theta(1 + \alpha)$

Demostración detallada. **Búsqueda fallida:** La clave k hashea a cualquier slot con probabilidad $1/m$. El número esperado de elementos en ese slot es $n/m = \alpha$. Por lo tanto:

$$E[\text{sondas}] = 1 + \alpha$$

(1 por el slot en sí, más el recorrido de la lista).

Búsqueda exitosa: Sea X_i la variable indicadora de que el i-ésimo elemento insertado está antes que el elemento buscado en su lista. Por linealidad:

$$E[\text{elementos examinados}] = 1 + \sum_{i=1}^{n} \Pr(\text{elemento } i \text{ está antes})$$

La probabilidad de que el elemento i esté antes es $1/(\text{número de elementos en el slot cuando se insertó})$. Promediando:

$$E[T] = 1 + \frac{1}{n}\sum_{j=1}^{n}\frac{j-1}{m} = 1 + \frac{1}{nm}\cdot\frac{n(n-1)}{2} = 1 + \frac{n-1}{2m} = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n}$$

□

Ventajas y Desventajas del Encadenamiento

- **Ventajas:**
  - Simple de implementar.
  - Eliminación fácil.
  - Rendimiento predecible.
  - α puede ser mayor que 1.
- **Desventajas:**
  - Uso de memoria adicional para punteros.
  - Mala localidad de referencia.
  - Peor caso $O(n)$ si la función hash es mala.

Direccionamiento Abierto

Descripción General

En direccionamiento abierto, todos los elementos se almacenan en la propia tabla. Cada slot contiene un elemento o NIL. Cuando hay colisión, se busca un slot alternativo mediante una **secuencia de sonda**.

$T[0]$	$T[1]$	$T[2]$	$T[3]$	$T[4]$	$T[5]$	$T[6]$	$T[7]$
15	7	22	NIL	8	31	NIL	NIL
3	11	NIL	NIL	NIL	NIL	NIL	NIL
NIL	NIL	NIL	NIL	NIL	NIL	NIL	NIL
NIL	NIL	NIL	NIL	NIL	NIL	NIL	NIL

Sondeo Lineal

Definición

$$h(k, i) = (h'(k) + i) \text{ mód } m$$

donde h' es la función hash auxiliar.

Implementación

```c
int linear_probe(HashTable* ht, int key, int i) {
    return (hash(key, ht->size) + i) % ht->size;
}

void insert_linear(HashTable* ht, int key, void* data) {
    int i = 0;
    int index;
    do {
        index = linear_probe(ht, key, i);
        if (ht->table[index] == NULL || ht->table[index] ==
            DELETED) {
            ht->table[index] = create_node(key, data);
            ht->count++;
            return;
        }
        i++;
    } while (i < ht->size);
    error("table overflow");
}
```

Listing 5: Sondeo lineal

Análisis del Sondeo Lineal

- **Ventajas:** Muy simple, buena localidad de referencia.
- **Desventajas: Agrupamiento primario** - se forman largas secuencias de slots ocupados.

Teorema 0.0.2 (Agrupamiento primario)**.** *En sondeo lineal, la longitud esperada de la secuencia de slots ocupados que contiene un slot dado es* $\Theta(1/(1-\alpha)^2)$.

Sondeo Cuadrático

Definición

$$h(k,i) = (h'(k) + c_1 i + c_2 i^2) \text{ mód } m$$

donde c_1 y c_2 son constantes no nulas.

Implementación

```c
int quadratic_probe(HashTable* ht, int key, int i) {
    int h = hash(key, ht->size);
    return (h + c1*i + c2*i*i) % ht->size;
}
```

Listing 6: Sondeo cuadrático

Análisis del Sondeo Cuadrático

- **Ventajas:** Mejor dispersión que lineal, evita agrupamiento primario.
- **Desventajas: Agrupamiento secundario** - si $h'(k_1) = h'(k_2)$, las secuencias son iguales.
- **Restricciones:** m debe ser primo para garantizar que se visiten todos los slots.

Doble Hashing

Definición

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \text{ mód } m$$

donde h_1 y h_2 son funciones hash auxiliares.

Implementación

```c
int double_hash(HashTable* ht, int key, int i) {
    int h1 = hash1(key, ht->size);
    int h2 = hash2(key, ht->size);
    return (h1 + i * h2) % ht->size;
}

void insert_double(HashTable* ht, int key, void* data) {
    int i = 0;
    int index;
    do {
        index = double_hash(ht, key, i);
        if (ht->table[index] == NULL || ht->table[index] ==
            DELETED) {
            ht->table[index] = create_node(key, data);
            ht->count++;
            return;
        }
        i++;
    } while (i < ht->size);
    error("table overflow");
}
```

Listing 7: Doble hashing

Condición de Relativa Primalidad

Para que la secuencia de sonda sea una permutación completa, $h_2(k)$ debe ser relativamente primo con m. Formas de garantizarlo:

- Elegir m potencia de 2 y $h_2(k)$ impar.
- Elegir m primo y $1 \leq h_2(k) < m$.

Análisis del Doble Hashing

- **Ventajas:** Excelente dispersión, se aproxima al hash uniforme.
- **Desventajas:** Más lento por dos funciones hash.
- **Teóricamente óptimo:** Proporciona $\Theta(m^2)$ secuencias diferentes.

Comparativa de Métodos

Tabla Comparativa

Característica	**Encadenamiento**	**Sondeo Lineal**	**Doble Hashing**
Memoria adicional	Alta (punteros)	Baja	Baja
Localidad de referencia	Mala	Excelente	Buena
Eliminación	Fácil	Difícil (marcador)	Difícil (marcador)
α permitido	≥ 1	< 1	< 1
Peor caso búsqueda	$O(n)$	$O(n)$	$O(n)$
Esperado búsqueda	$1 + \alpha/2$	$\frac{1}{2}(1 + \frac{1}{(1-\alpha)^2})$	$\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$

Gráfica de Rendimiento

Eliminación en Direccionamiento Abierto

El Problema

Si simplemente marcamos un slot como NIL, se pueden perder claves que estaban después en la secuencia de sonda.

Solución con Marcador DELETED

```c
#define DELETED -1

void delete_open(HashTable* ht, int key) {
    int i = 0;
    int index;
    do {
        index = probe(ht, key, i);
        if (ht->table[index] != NULL && ht->table[index]->key == key) {
            free(ht->table[index]);
            ht->table[index] = DELETED;  // marcador
            ht->count--;
            return;
        }
        i++;
    } while (i < ht->size && ht->table[index] != NULL);
}
```

Listing 8: Eliminación con marcador

Inserción con DELETED

```c
void insert_open(HashTable* ht, int key, void* data) {
    int i = 0;
    int index;
    do {
        index = probe(ht, key, i);
        if (ht->table[index] == NULL || ht->table[index] == DELETED) {
            ht->table[index] = create_node(key, data);
            ht->count++;
            return;
        }
        i++;
    } while (i < ht->size);
    error("table overflow");
}
```

Listing 9: Inserción con marcador

Consecuencias del Uso de DELETED

- **Ventaja:** Permite eliminación sin reorganización.
- **Desventaja:** Degrada el rendimiento con muchas eliminaciones.
- **Solución:** Rehashing periódico.

Rehashing

Cuándo Rehashear

- Cuando α supera un umbral (ej. 0.75).
- Cuando se acumulan muchos marcadores DELETED.
- Cuando el rendimiento se degrada.

Algoritmo de Rehashing

```c
void rehash(HashTable* ht) {
    int old_size = ht->size;
    Node** old_table = ht->table;

    // Duplicar tama o (o elegir nuevo primo)
    ht->size = next_prime(2 * old_size);
    ht->table = calloc(ht->size, sizeof(Node*));
    ht->count = 0;

    // Reinsertar todos los elementos
    for (int i = 0; i < old_size; i++) {
        if (old_table[i] != NULL && old_table[i] != DELETED) {
            insert(ht, old_table[i]->key, old_table[i]->data);
            free(old_table[i]);
        }
    }
    free(old_table);
}
```

Listing 10: Rehashing

Análisis Amortizado

El costo de rehashing se amortiza entre las inserciones. Si duplicamos el tamaño cuando la tabla está llena, el costo amortizado por inserción es $O(1)$.

Ejercicios Resueltos (Ampliados)

Ejercicio 0.0.1. *Inserte las claves 10, 22, 31, 4, 15, 28, 17, 88, 59 en una tabla con* $m = 11$ *usando encadenamiento y* $h(k) = k \text{ mód } 11$.

Solución:

- $h(10) = 10$, $h(22) = 0$, $h(31) = 9$, $h(4) = 4$, $h(15) = 4$, $h(28) = 6$, $h(17) = 6$, $h(88) = 0$, $h(59) = 4$.
- Slot 0: $22 \to 88$
- Slot 4: $4 \to 15 \to 59$
- Slot 6: $28 \to 17$
- Slot 9: 31
- Slot 10: 10

Ejercicio 0.0.2. *Inserte las mismas claves usando sondeo lineal.*

Solución:

1. 10: slot 10
2. 22: slot 0
3. 31: slot 9
4. 4: slot 4
5. 15: slot 4 ocupado $\to$ slot 5
6. 28: slot 6
7. 17: slot 6 ocupado $\to$ slot 7
8. 88: slot 0 ocupado $\to$ slot 1
9. 59: slot 4 ocupado $\to$ slot 5 ocupado $\to$ slot 6 ocupado $\to$ slot 7 ocupado $\to$ slot 8

Tabla final: [22, 88, NIL, NIL, 4, 15, 28, 17, 59, 31, 10]

Ejercicio 0.0.3. *Inserte las mismas claves usando doble hashing con* $h_1(k) = k \text{ mód } 11$, $h_2(k) = 1 + (k \text{ mód } 10)$.

Solución:

1. 10: $h_1 = 10$, $h_2 = 1 + (10 \text{ mód } 10) = 1$, slot 10

2. 22: $h_1 = 0$, $h_2 = 1 + (22 \text{ mód } 10) = 3$, slot 0
3. 31: $h_1 = 9$, $h_2 = 1 + (31 \text{ mód } 10) = 2$, slot 9
4. 4: $h_1 = 4$, $h_2 = 1 + (4 \text{ mód } 10) = 5$, slot 4
5. 15: $h_1 = 4$, $h_2 = 1 + 5 = 6$, slot 4 ocupado $\rightarrow$ slot 10 (4+6) ocupado $\rightarrow$ slot 5 (4+12)
6. 28: $h_1 = 6$, $h_2 = 1 + 8 = 9$, slot 6
7. 17: $h_1 = 6$, $h_2 = 1 + 7 = 8$, slot 6 ocupado $\rightarrow$ slot 3 (6+8)
8. 88: $h_1 = 0$, $h_2 = 1 + 8 = 9$, slot 0 ocupado $\rightarrow$ slot 9 ocupado $\rightarrow$ slot 7 (0+18)
9. 59: $h_1 = 4$, $h_2 = 1 + 9 = 10$, slot 4 ocupado $\rightarrow$ slot 3 (4+10) ocupado $\rightarrow$ slot 2 (4+20)

Ejercicio 0.0.4. *Demuestre que en sondeo lineal, la longitud esperada de la secuencia de slots ocupados es* $\Theta(1/(1-\alpha)^2)$.

Solución:
La probabilidad de que una secuencia de k slots consecutivos esté ocupada es α^k. La longitud esperada es $\sum_{k=1}^{\infty} \alpha^k = \alpha/(1-\alpha)$. Pero esto es para un slot específico. La longitud esperada de una ejecución es $\Theta(1/(1-\alpha)^2)$ por propiedades de las ejecuciones en cadenas de Bernoulli.

Ejercicio 0.0.5. *¿Por qué en doble hashing se requiere que* $h_2(k)$ *sea relativamente primo con* m*?*

Solución:
Si $d = \gcd(h_2(k), m) > 1$, la secuencia $(h_1 + ih_2) \text{ mód } m$ solo visita m/d slots distintos, no todos.

Ejercicio 0.0.6. *Calcule el número esperado de sondas para búsqueda exitosa en direccionamiento abierto con* $\alpha = 0.8$ *usando la fórmula* $\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$.

Solución:
$\frac{1}{0.8} \ln \frac{1}{0.2} = 1.25 \times \ln 5 \approx 1.25 \times 1.609 = 2.011$ sondas.

Ejercicio 0.0.7. *¿Qué ventaja tiene el encadenamiento sobre direccionamiento abierto?*

Solución:

- Permite $\alpha > 1$.
- Eliminación más sencilla.

- No requiere marcadores DELETED.
- Mejor rendimiento en el peor caso con buena función hash.

Ejercicio 0.0.8. *¿Qué ventaja tiene direccionamiento abierto sobre encadenamiento?*

Solución:

- Mejor localidad de referencia (todo en un arreglo).
- Menor uso de memoria (sin punteros).
- Puede ser más rápido para tablas pequeñas.

Ejercicio 0.0.9. *¿Cuándo es preferible usar sondeo lineal?*

Solución:
Cuando la tabla es pequeña, la función hash es buena, y se espera baja carga. Por su simplicidad y localidad.

Ejercicio 0.0.10. *¿Cuándo es preferible usar doble hashing?*

Solución:
Cuando se necesita la mejor dispersión posible y se puede tolerar el costo extra de dos funciones hash.

Ejercicio 0.0.11. *Simule la inserción de 100 elementos en una tabla de tamaño 100 con sondeo lineal. ¿Cuál es el número esperado de colisiones?*

Solución:
Con $\alpha = 1$, el número esperado de sondas para búsqueda fallida es $1/(1-1) = \infty$, indicando que es probable que ocurran muchas colisiones.

Ejercicios Propuestos (Ampliados)

1. Inserte las claves 5, 28, 19, 15, 20, 33, 12, 17, 10 en una tabla con $m = 9$ usando encadenamiento.
2. Inserte las mismas claves usando sondeo lineal.
3. Inserte las mismas claves usando sondeo cuadrático con $c_1 = 1$, $c_2 = 3$.
4. Inserte las mismas claves usando doble hashing con $h_1(k) = k \bmod 9$, $h_2(k) = 1 + (k \bmod 8)$.
5. Implemente una tabla hash con encadenamiento en Python.

6. Implemente una tabla hash con sondeo lineal en C.
7. Analice el rendimiento de sondeo lineal con $\alpha = 0.5, 0.75, 0.9$.
8. Demuestre que el número esperado de sondas en búsqueda exitosa para direccionamiento abierto es $\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$.
9. ¿Qué es el "agrupamiento secundario" y qué métodos lo sufren?
10. ¿Cómo afecta la elección de m al rendimiento en sondeo cuadrático?
11. Diseñe una estrategia de rehashing que mantenga α entre 0.25 y 0.75.
12. Compare el uso de memoria de encadenamiento vs. direccionamiento abierto.
13. ¿Qué sucede si en direccionamiento abierto no usamos marcador DELETED y simplemente marcamos NIL?
14. Implemente búsqueda en una tabla con sondeo cuadrático.
15. ¿Cuál es la complejidad esperada de inserción en una tabla con encadenamiento?
16. ¿Por qué el doble hashing se considera el mejor método de direccionamiento abierto?
17. Simule la inserción de 50 elementos aleatorios en una tabla de tamaño 100 usando sondeo lineal. Calcule la longitud promedio de las ejecuciones.
18. ¿Cómo se puede adaptar el encadenamiento para usar árboles balanceados en lugar de listas?
19. ¿Qué es el "hash perfecto" y cómo se relaciona con la resolución de colisiones?
20. Investigue cómo manejan las colisiones los diccionarios en Python (CPython).

Unidad IX: Análisis de Rendimiento de Tablas Hash

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender los fundamentos probabilísticos del análisis de tablas hash.
- Analizar rigurosamente el rendimiento esperado de encadenamiento y direccionamiento abierto.
- Aplicar técnicas de variables aleatorias indicadoras en el análisis.
- Derivar y utilizar las fórmulas de complejidad esperada para búsquedas exitosas y fallidas.
- Comprender la importancia del factor de carga y cómo afecta al rendimiento.

Introducción: El Marco Probabilístico

Supuestos Fundamentales

Para analizar el rendimiento de tablas hash, realizamos los siguientes supuestos:

1. **Hash uniforme simple:** Cada clave tiene la misma probabilidad de hashear a cualquier slot, independientemente de otras claves.
2. **Independencia:** Los valores hash de diferentes claves son independientes.
3. **Tiempo constante:** Calcular $h(k)$ toma $O(1)$.

Variables Aleatorias Fundamentales

Definimos:

- n: número de claves almacenadas.
- m: tamaño de la tabla.
- $\alpha = n/m$: factor de carga.
- n_j: número de claves que hashean al slot j.

Análisis de Encadenamiento

Distribución del Número de Claves por Slot

Distribución Binomial

Para un slot fijo j, n_j sigue una distribución binomial:

$$\Pr(n_j = k) = \binom{n}{k} \left(\frac{1}{m}\right)^k \left(1 - \frac{1}{m}\right)^{n-k}$$

Momento de Primer Orden

$$E[n_j] = \frac{n}{m} = \alpha$$

Momento de Segundo Orden

$$E[n_j^2] = \frac{n}{m} + \frac{n(n-1)}{m^2} = \alpha + \alpha^2 - \frac{\alpha}{n}$$

Búsqueda Fallida

Definición

Una búsqueda fallida ocurre cuando la clave k no está en la tabla. Se debe examinar toda la lista $T[h(k)]$ hasta encontrar NIL.

Análisis

Teorema 0.0.3. *El número esperado de elementos examinados en una búsqueda fallida es* $1 + \alpha$.

Demostración. Sea X el número de elementos examinados. El primer elemento examinado es el slot $h(k)$. Luego se recorre la lista. El número esperado de elementos en la lista es $E[n_{h(k)}] = \alpha$. Por lo tanto:

$$E[X] = 1 + \alpha$$

El 1 corresponde al slot mismo, y α al recorrido de la lista. □

Búsqueda Exitosa

Definición

Una búsqueda exitosa ocurre cuando la clave k está en la tabla. Se debe examinar la lista hasta encontrar k.

Análisis con Variables Indicadoras

Teorema 0.0.4. *El número esperado de elementos examinados en una búsqueda exitosa es* $1 + \frac{\alpha}{2} - \frac{\alpha}{2n}$.

Demostración. Sean las claves $k_1, k_2, \ldots, k_n$ en orden de inserción. Para cada par (i, j) con $i < j$, definimos:

$$X_{ij} = I\{h(k_i) = h(k_j)\}$$

La probabilidad de colisión es $1/m$, por lo que $E[X_{ij}] = 1/m$.

El número de elementos que preceden a k_i en su lista es $\sum_{j=1}^{i-1} X_{ji}$. Por lo tanto, el número esperado de elementos examinados al buscar k_i es:

$$E[\text{elementos}] = 1 + \sum_{j=1}^{i-1} E[X_{ji}] = 1 + \frac{i-1}{m}$$

Promediando sobre todos los i:

$$E[T] = \frac{1}{n}\sum_{i=1}^{n}\left(1 + \frac{i-1}{m}\right) = 1 + \frac{1}{nm}\sum_{i=1}^{n}(i-1) = 1 + \frac{n(n-1)}{2nm} = 1 + \frac{n-1}{2m} = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n}$$

□

Varianza del Tiempo de Búsqueda

Teorema 0.0.5. *La varianza del número de elementos examinados en una búsqueda exitosa es* $O(\alpha)$.

Esbozo. Usando que $\mathrm{Var}(X) = E[X^2] - E[X]^2$ y que $E[X^2]$ se puede calcular similarmente a $E[X]$, se obtiene el resultado. □

Análisis de Direccionamiento Abierto

Supuestos Adicionales

- **Hash uniforme:** Cada clave tiene igual probabilidad de tener cualquier secuencia de sonda.
- $\alpha < 1$ (la tabla no se llena completamente).

Búsqueda Fallida

Definición

Una búsqueda fallida examina slots hasta encontrar uno vacío.

Análisis

Teorema 0.0.6. *El número esperado de sondas en una búsqueda fallida es* $\frac{1}{1-\alpha}$.

Demostración. Sea X el número de sondas. Entonces:

$$\Pr(X \geq i) = \frac{n}{m} \cdot \frac{n-1}{m-1} \cdots \frac{n-i+2}{m-i+2}$$

Para i fijo, $\Pr(X \geq i) \leq \alpha^{i-1}$ porque $(n-j)/(m-j) \leq n/m$ para $j \geq 0$.

Por lo tanto:

$$E[X] = \sum_{i=1}^{\infty} \Pr(X \geq i) \leq \sum_{i=1}^{\infty} \alpha^{i-1} = \frac{1}{1-\alpha}$$

□

Búsqueda Exitosa

Análisis

Teorema 0.0.7. *El número esperado de sondas en una búsqueda exitosa es* $\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$.

Demostración. Consideremos la inserción de la $(i+1)$-ésima clave. En ese momento, el factor de carga es i/m. El número esperado de sondas para esa inserción es $\frac{1}{1-i/m} = \frac{m}{m-i}$.

Promediando sobre todas las claves:

$$E[T] = \frac{1}{n} \sum_{i=0}^{n-1} \frac{m}{m-i} = \frac{m}{n} \sum_{i=0}^{n-1} \frac{1}{m-i}$$

$$= \frac{1}{\alpha} \sum_{k=m-n+1}^{m} \frac{1}{k} = \frac{1}{\alpha}(H_m - H_{m-n})$$

donde H_k es el k-ésimo número armónico.

Usando la aproximación integral:

$$H_m - H_{m-n} \approx \int_{m-n}^{m} \frac{1}{x} dx = \ln \frac{m}{m-n} = \ln \frac{1}{1-\alpha}$$

Por lo tanto:

$$E[T] \approx \frac{1}{\alpha} \ln \frac{1}{1-\alpha}$$

□

Comparación de Métodos de Sonda

Sondeo Lineal

Teorema 0.0.8. *En sondeo lineal, el número esperado de sondas es:*

$$\textit{Fallida: } \frac{1}{2}\left(1 + \frac{1}{(1-\alpha)^2}\right)$$

$$\textit{Exitosa: } \frac{1}{2}\left(1 + \frac{1}{1-\alpha}\right)$$

Sondeo Cuadrático y Doble Hashing

Ambos tienen rendimiento aproximado al hash uniforme:

$$\text{Fallida: } \frac{1}{1-\alpha}$$

$$\text{Exitosa: } \frac{1}{\alpha} \ln \frac{1}{1-\alpha}$$

Análisis de Rehashing

Costo Amortizado

Teorema 0.0.9. *El costo amortizado de inserción en una tabla hash que duplica su tamaño cuando está llena es $O(1)$.*

Demostración. Supongamos que duplicamos el tamaño cuando $n = m$. El costo de rehashing es $O(n)$ (reinsertar n elementos). El número de inserciones entre rehashes es n. Por lo tanto, el costo amortizado por inserción es:

$$\frac{O(n)}{n} + O(1) = O(1)$$

□

Estrategias de Crecimiento

- **Doblado:** $m \leftarrow 2m$. Amortizado $O(1)$.
- **Crecimiento lineal:** $m \leftarrow m + c$. Amortizado $O(n)$.
- **Próximo primo:** $m \leftarrow \text{next_prime}(2m)$. Mantiene propiedades de primalidad.

Análisis de Casos Extremos

Peor Caso

Teorema 0.0.10. *Existe una secuencia de n operaciones que hace que el tiempo de búsqueda sea $\Theta(n)$ para cualquier función hash determinista.*

Demostración. El adversario puede elegir n claves que todas hasheen al mismo slot. La función hash es fija, por lo que puede hacerlo conociéndola. □

Hash Universal

Teorema 0.0.11. *Con hash universal, el tiempo esperado de búsqueda es $O(1)$ para cualquier secuencia de operaciones.*

Demostración. La probabilidad de colisión para cualquier par de claves es $\leq 1/m$. Por linealidad, el número esperado de colisiones es $O(1)$. □

Análisis de la Paradoja del Cumpleaños

Relación con Hashing

La probabilidad de que dos claves colisionen después de n inserciones es:

$$\Pr(\text{colisión}) \approx 1 - e^{-n^2/(2m)}$$

Punto de Colisión Esperado

El número esperado de inserciones hasta la primera colisión es $\sqrt{\pi m/2}$.

Aplicación

Si $n = \sqrt{m}$, la probabilidad de colisión es aproximadamente $1 - e^{-1/2} \approx 0.393$.

Ejercicios Resueltos (Ampliados)

Ejercicio 0.0.12. *Una tabla hash tiene $m = 1000$ y $n = 500$ claves. Calcule α y el número esperado de sondas para búsqueda fallida en encadenamiento.*

Solución:
$\alpha = 500/1000 = 0.5$. Número esperado de sondas $= 1 + \alpha = 1.5$.

Ejercicio 0.0.13. *Misma tabla, calcule el número esperado de sondas para búsqueda exitosa en encadenamiento.*

Solución:
$1 + \alpha/2 - \alpha/(2n) = 1 + 0.25 - 0.0005 = 1.2495$.

Ejercicio 0.0.14. *Una tabla con direccionamiento abierto tiene $m = 1000$ y $n = 800$. Calcule el número esperado de sondas para búsqueda fallida.*

Solución:
$\alpha = 0.8$. Sondas $= 1/(1 - 0.8) = 5$.

Ejercicio 0.0.15. *Misma tabla, calcule el número esperado de sondas para búsqueda exitosa.*

Solución:
$\frac{1}{0.8} \ln \frac{1}{0.2} = 1.25 \times 1.609 = 2.011$.

Ejercicio 0.0.16. *Demuestre que en encadenamiento, $E[n_j^2] = \alpha + \alpha^2 - \alpha/n$.*

Solución:
$E[n_j^2] = \mathrm{Var}(n_j) + E[n_j]^2$. $\mathrm{Var}(n_j) = n \cdot \frac{1}{m} \cdot (1 - \frac{1}{m}) = \alpha(1 - 1/m)$. Entonces: $E[n_j^2] = \alpha(1 - 1/m) + \alpha^2 = \alpha + \alpha^2 - \alpha/m$.

Ejercicio 0.0.17. *¿Cuál es la probabilidad de que un slot específico esté vacío después de n inserciones?*

Solución:
$(1 - 1/m)^n \approx e^{-n/m} = e^{-\alpha}$.

Ejercicio 0.0.18. *¿Cuál es el número esperado de slots vacíos?*

Solución:
$m \cdot (1 - 1/m)^n \approx me^{-\alpha}$.

Ejercicio 0.0.19. *Demuestre que $\sum_{i=1}^{\infty} \alpha^{i-1} = 1/(1 - \alpha)$.*

Solución:
Es una serie geométrica con razón $\alpha < 1$: $\sum_{i=0}^{\infty} \alpha^i = 1/(1 - \alpha)$.

Ejercicio 0.0.20. *Para $\alpha = 0.9$, calcule las sondas esperadas en búsqueda exitosa y fallida en direccionamiento abierto.*

Solución:
Fallida: $1/(1-0.9) = 10$. Exitosa: $(1/0.9)\ln(1/0.1) = 1.111 \times 2.3026 = 2.558$.

Ejercicio 0.0.21. *¿Cuántas inserciones se necesitan en promedio para que ocurra la primera colisión en una tabla de tamaño $m = 1000$?*

Solución:
$\sqrt{\pi m/2} = \sqrt{3.1416 \times 500} = \sqrt{1570.8} \approx 39.6$ inserciones.

Ejercicio 0.0.22. *Demuestre que* $\frac{1}{\alpha} \ln \frac{1}{1-\alpha} \leq \frac{1}{1-\alpha}$ *para* $0 < \alpha < 1$.

Solución:
La desigualdad $\ln(1/(1-\alpha)) \leq \alpha/(1-\alpha)$ es equivalente a $\ln(1+x) \leq x$ con $x = \alpha/(1-\alpha)$. Esto es cierto porque $\ln(1+x) \leq x$ para $x > -1$.

Ejercicio 0.0.23. *Una tabla hash con encadenamiento tiene* $m = 100$ *y* $n = 200$. *¿Cuál es el número esperado de sondas para búsqueda exitosa?*

Solución:
$\alpha = 2$. Fórmula: $1 + \alpha/2 - \alpha/(2n) = 1 + 1 - 0.005 = 1.995$.

Ejercicio 0.0.24. *Demuestre que la varianza del número de claves por slot en encadenamiento es* $\alpha(1 - 1/m)$.

Solución:
$\mathrm{Var}(n_j) = n \cdot \frac{1}{m} \cdot (1 - \frac{1}{m}) = \alpha(1 - 1/m)$.

Ejercicio 0.0.25. *Para* $\alpha = 0.5$, *compare las sondas esperadas en búsqueda exitosa entre encadenamiento y direccionamiento abierto.*

Solución:
Encadenamiento: 1.25. Direccionamiento abierto: $(1/0.5)\ln(1/0.5) = 2 \times 0.693 = 1.386$. El encadenamiento es ligeramente mejor.

Ejercicio 0.0.26. *¿Cuál es el número esperado de colisiones totales en una tabla hash con* n *claves y* m *slots?*

Solución:
$E[\text{colisiones}] = \binom{n}{2} \cdot \frac{1}{m} = \frac{n(n-1)}{2m}$.

Ejercicio 0.0.27. *Demuestre que* $\Pr(\textit{slot } j \textit{ vacío}) \approx e^{-\alpha}$.

Solución:
$\Pr = (1 - 1/m)^n$. Usando $(1 - 1/m)^n \approx e^{-n/m} = e^{-\alpha}$.

Ejercicios Propuestos (Ampliados)

1. Calcule α para $n = 1000$, $m = 500$. ¿Es aceptable para direccionamiento abierto?

2. Para $\alpha = 0.75$, calcule sondas esperadas en búsqueda exitosa y fallida en direccionamiento abierto.

3. Para $\alpha = 0.75$, calcule sondas esperadas en búsqueda exitosa y fallida en encadenamiento.
4. Demuestre que $\sum_{i=0}^{n-1} \frac{m}{m-i} = m(H_m - H_{m-n})$.
5. ¿Cuál es el número esperado de slots con exactamente una clave?
6. Derive la fórmula para el número esperado de sondas en búsqueda fallida para sondeo lineal.
7. ¿Cómo cambia el rendimiento si m es primo vs. potencia de 2?
8. Demuestre que la probabilidad de que no haya colisiones después de n inserciones es $\prod_{i=0}^{n-1}(1 - i/m)$.
9. Calcule la probabilidad de que no haya colisiones para $n = 10$, $m = 100$.
10. ¿Cuál es el valor de α que minimiza el número esperado de sondas en búsqueda exitosa?
11. Analice el rendimiento de una tabla hash cuando todas las claves tienen el mismo hash.
12. Demuestre que el costo amortizado de rehashing es $O(1)$ usando el método potencial.
13. ¿Cuál es la relación entre α y la probabilidad de que un slot tenga más de k claves?
14. Para $\alpha = 2$, ¿cuál es el número esperado de sondas en búsqueda fallida en encadenamiento?
15. Demuestre que $H_m - H_{m-n} \approx \ln(1/(1 - \alpha))$.
16. ¿Cómo afecta la varianza del número de claves por slot al rendimiento?
17. Analice el rendimiento de una tabla hash con α cercano a 1.
18. ¿Cuál es la complejidad esperada de inserción en una tabla con direccionamiento abierto?
19. Derive la fórmula para el número esperado de sondas en búsqueda exitosa para sondeo cuadrático.
20. Compare el rendimiento de encadenamiento vs. direccionamiento abierto para diferentes valores de α.

Unidad X: Grafos (Introducción) - Representaciones y Recorridos

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender la definición formal de grafos y sus tipos.
- Implementar y elegir entre representaciones de grafos (matrices y listas de adyacencia).
- Implementar y analizar los algoritmos de búsqueda en profundidad (DFS) y amplitud (BFS).
- Aplicar recorridos para resolver problemas como orden topológico y detección de ciclos.
- Analizar la complejidad de los algoritmos de grafos.

Definiciones Fundamentales

¿Qué es un Grafo?

Un **grafo** $G = (V, E)$ consiste en:

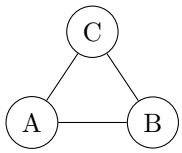
- V: conjunto finito de **vértices** (o nodos).
- E: conjunto de **aristas** (pares de vértices).

Tipos de Grafos

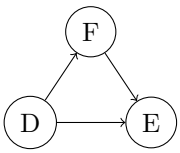
- **Grafo no dirigido:** Las aristas son pares no ordenados $\{u, v\}$.
- **Grafo dirigido (digrafo):** Las aristas son pares ordenados (u, v).
- **Grafo ponderado:** Cada arista tiene un peso asociado.

Terminología Básica

- **Grado:** Número de aristas incidentes a un vértice.
- **Grado de entrada (indegree):** En grafos dirigidos, número de aristas que entran.
- **Grado de salida (outdegree):** En grafos dirigidos, número de aristas que salen.
- **Camino:** Secuencia de vértices conectados por aristas.
- **Ciclo:** Camino que comienza y termina en el mismo vértice.
- **Grafo conexo:** Existe un camino entre cualquier par de vértices.



(a) Grafo no dirigido



(b) Grafo dirigido

Representaciones de Grafos

Matriz de Adyacencia

Definición

Para un grafo con n vértices, la matriz de adyacencia A es una matriz $n \times n$ donde:

$$A[i][j] = \begin{cases} 1 & \text{si } (i, j) \in E \\ 0 & \text{en caso contrario} \end{cases}$$

Ejemplo

Para el grafo no dirigido con vértices $\{1, 2, 3\}$ y aristas $(1, 2)$, $(1, 3)$, $(2, 3)$:

$$A = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}$$

Propiedades

- Para grafos no dirigidos: A es simétrica ($A[i][j] = A[j][i]$).
- Espacio: $\Theta(V^2)$.
- Tiempo para verificar si $(u, v) \in E$: $O(1)$.
- Tiempo para listar vecinos de u: $\Theta(V)$.

Ventajas y Desventajas

- **Ventajas:** Acceso $O(1)$ a aristas, simple de implementar.
- **Desventajas:** Ineficiente en espacio para grafos dispersos ($|E| \ll |V|^2$).

Listas de Adyacencia

Definición

Para cada vértice u, mantenemos una lista de todos los vértices v tales que $(u, v) \in E$.

Ejemplo

Para el mismo grafo:

```
1: 2, 3
2: 1, 3
3: 1, 2
```

Implementación

```
struct Graph {
    int V;                      // n mero de v rtices
    struct Node** adj;          // arreglo de listas
    int* visited;               // para recorridos
};

struct Node {
    int vertex;
    struct Node* next;
};
```

Listing 11: Estructura de listas de adyacencia

Propiedades

- Espacio: $\Theta(V + E)$.
- Tiempo para verificar si $(u, v) \in E$: $O(\text{grado}(u))$.
- Tiempo para listar vecinos de u: $\Theta(\text{grado}(u))$.

Ventajas y Desventajas

- **Ventajas:** Eficiente en espacio para grafos dispersos, iteración rápida sobre vecinos.
- **Desventajas:** Verificación de existencia de arista más lenta.

Comparativa

Característica	Matriz	Listas
Espacio	$\Theta(V^2)$	$\Theta(V + E)$
Verificar arista	$O(1)$	$O(\text{grado}(u))$
Listar vecinos	$\Theta(V)$	$\Theta(\text{grado}(u))$
Añadir arista	$O(1)$	$O(1)$
Eliminar arista	$O(1)$	$O(\text{grado}(u))$

Búsqueda en Amplitud (BFS)

Descripción

BFS explora el grafo nivel por nivel, comenzando desde un vértice fuente. Primero visita todos los vértices a distancia 1, luego distancia 2, etc.

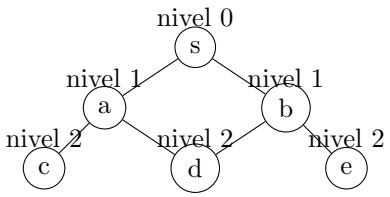
Algoritmo

```
BFS(G, s)
    for each u in G.V
        u.color = WHITE
        u.d = INFINITY
        u.parent = NIL
    s.color = GRAY
    s.d = 0
    s.parent = NIL
    Q = empty queue
    enqueue(Q, s)
    while Q not empty
        u = dequeue(Q)
        for each v in G.adj[u]
            if v.color == WHITE
                v.color = GRAY
```

```
                v.d = u.d + 1
                v.parent = u
                enqueue(Q, v)
        u.color = BLACK
```

Listing 12: BFS

Ejemplo Visual



Análisis de Complejidad

- Cada vértice se pone en cola una vez: $O(V)$.
- Cada arista se examina una vez (dos en no dirigido): $O(E)$.
- Total: $O(V + E)$.

Propiedades de BFS

Teorema 0.0.12. *BFS calcula correctamente la distancia mínima (número de aristas) desde s a cada vértice alcanzable.*

Aplicaciones de BFS

- Caminos más cortos en grafos no ponderados.
- Componentes conexas.
- Verificación de bipartitud.
- Encontrar el diámetro de un árbol.

Búsqueda en Profundidad (DFS)

Descripción

DFS explora tan profundo como sea posible antes de retroceder. Es un algoritmo recursivo que puede generar un bosque de árboles.

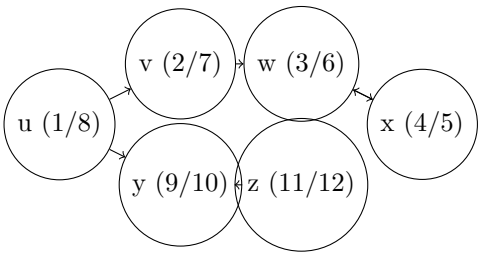
Algoritmo

```
DFS(G)
    for each u in G.V
        u.color = WHITE
        u.parent = NIL
    time = 0
    for each u in G.V
        if u.color == WHITE
            DFS-VISIT(u)

DFS-VISIT(u)
    u.color = GRAY
    time = time + 1
    u.d = time            // tiempo de descubrimiento
    for each v in G.adj[u]
        if v.color == WHITE
            v.parent = u
            DFS-VISIT(v)
    u.color = BLACK
    time = time + 1
    u.f = time            // tiempo de finalizaci n
```

Listing 13: DFS

Ejemplo Visual



Análisis de Complejidad

- Cada vértice se visita una vez: $O(V)$.
- Cada arista se examina una vez: $O(E)$.
- Total: $O(V + E)$.

Propiedades de DFS

Teorema del Paréntesis

Teorema 0.0.13. *Para dos vértices u y v, los intervalos $[d[u], f[u]]$ y $[d[v], f[v]]$ son disjuntos o uno está contenido en el otro.*

Clasificación de Aristas

- **Aristas de árbol:** (u, v) donde v fue descubierto desde u.
- **Aristas hacia atrás:** (u, v) donde v es ancestro de u.
- **Aristas hacia adelante:** (u, v) donde v es descendiente de u (no árbol).
- **Aristas cruzadas:** Las demás.

Aplicaciones de DFS

- Detección de ciclos.
- Orden topológico.
- Componentes fuertemente conexas.
- Puntos de articulación y puentes.

Orden Topológico

Definición

Un **orden topológico** de un grafo acíclico dirigido (DAG) es un ordenamiento lineal de sus vértices tal que para toda arista (u, v), u aparece antes que v.

Aplicaciones

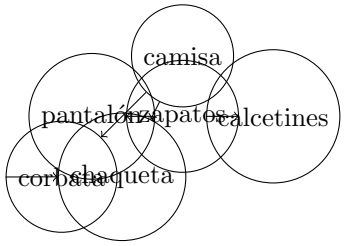
- Planificación de tareas con dependencias.
- Resolución de expresiones.
- Compilación de código.

Algoritmo basado en DFS

```
TOPOLOGICAL-SORT(G)
    call DFS(G) to compute finishing times f[v]
    as each vertex is finished, insert it at front of linked
        list
    return linked list
```

Listing 14: Orden topológico con DFS

Ejemplo



Teorema de Correctitud

Teorema 0.0.14. *TOPOLOGICAL-SORT produce un orden topológico de un DAG.*

Demostración. Para cualquier arista (u, v), en DFS $f[v] < f[u]$, por lo que u aparece antes que v en la lista ordenada por tiempos de finalización decrecientes. □

Detección de Ciclos

Ciclos en Grafos No Dirigidos

```
HAS-CYCLE-UNDIRECTED(G)
    for each u in G.V
        u.color = WHITE
    for each u in G.V
        if u.color == WHITE
            if DFS-CYCLE(u, NIL)
                return TRUE
    return FALSE

DFS-CYCLE(u, parent)
    u.color = GRAY
    for each v in G.adj[u]
        if v.color == WHITE
```

```
            if DFS-CYCLE(v, u)
                return TRUE
        else if v != parent and v.color == GRAY
            return TRUE
    u.color = BLACK
    return FALSE
```

Listing 15: Detección de ciclos en grafos no dirigidos

Ciclos en Grafos Dirigidos

```
HAS-CYCLE-DIRECTED(G)
    for each u in G.V
        u.color = WHITE
    for each u in G.V
        if u.color == WHITE
            if DFS-CYCLE-DIRECTED(u)
                return TRUE
    return FALSE

DFS-CYCLE-DIRECTED(u)
    u.color = GRAY
    for each v in G.adj[u]
        if v.color == WHITE
            if DFS-CYCLE-DIRECTED(v)
                return TRUE
        else if v.color == GRAY
            return TRUE
    u.color = BLACK
    return FALSE
```

Listing 16: Detección de ciclos en grafos dirigidos

Teorema de Detección de Ciclos

Teorema 0.0.15. *Un grafo dirigido tiene un ciclo si y solo si DFS encuentra una arista hacia atrás.*

Demostración. Si hay una arista hacia atrás (u, v), entonces v es ancestro de u, formando un ciclo. Recíprocamente, si hay un ciclo, el primer vértice descubierto en el ciclo tiene una arista hacia atrás desde el último vértice. □

Ejercicios Resueltos (Ampliados)

Ejercicio 0.0.28. *Represente el grafo con vértices* $\{1, 2, 3, 4\}$ *y aristas* $(1, 2)$, $(1, 3)$, $(2, 4)$, $(3, 4)$ *mediante matriz y listas de adyacencia.*

Solución:
Matriz:

$$\begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{pmatrix}$$

Listas:

```
1: 2, 3
2: 1, 4
3: 1, 4
4: 2, 3
```

Ejercicio 0.0.29. *Ejecute BFS desde el vértice 1 en el grafo anterior.*

Solución:

1. Inicial: $d[1] = 0$, cola: [1]
2. Procesar 1: visitar 2 y 3, $d[2] = 1$, $d[3] = 1$, cola: [2,3]
3. Procesar 2: visitar 4, $d[4] = 2$, cola: [3,4]
4. Procesar 3: 4 ya visitado, cola: [4]
5. Procesar 4: fin

Distancias: $d[1] = 0$, $d[2] = 1$, $d[3] = 1$, $d[4] = 2$.

Ejercicio 0.0.30. *Ejecute DFS en el mismo grafo.*

Solución:

1. Desde 1: visitar 2, desde 2 visitar 4, desde 4 no más, volver a 2, volver a 1.
2. Luego visitar 3 desde 1.

Tiempos: $d[1] = 1$, $f[1] = 6$; $d[2] = 2$, $f[2] = 5$; $d[4] = 3$, $f[4] = 4$; $d[3] = 7$, $f[3] = 8$.

Ejercicio 0.0.31. *Determine si el grafo dirigido con vértices* $\{1, 2, 3, 4\}$ *y aristas* $(1, 2)$, $(2, 3)$, $(3, 1)$, $(2, 4)$ *tiene ciclos.*

Solución:
Tiene ciclo $1 \to 2 \to 3 \to 1$. DFS detectará una arista hacia atrás cuando explore $(3, 1)$.

Ejercicio 0.0.32. *Ordene topológicamente el DAG:* $(1, 2)$, $(1, 3)$, $(2, 4)$, $(3, 4)$.

Solución:
Un orden posible: $1, 2, 3, 4$ o $1, 3, 2, 4$.

Ejercicio 0.0.33. *¿Cuál es la complejidad de BFS si el grafo se representa con matriz de adyacencia?*

Solución:
$O(V^2)$, porque para cada vértice se recorre toda la fila de la matriz.

Ejercicio 0.0.34. *¿Cuál es la complejidad de DFS si el grafo se representa con listas de adyacencia?*

Solución:
$O(V + E)$.

Ejercicio 0.0.35. *Demuestre que en un grafo no dirigido, BFS desde un vértice visita todos los vértices de su componente conexa.*

Solución:
Por inducción: si v está en la componente, hay un camino de longitud k desde s. BFS visitará todos los vértices a distancia k después de los de distancia $k - 1$.

Ejercicio 0.0.36. *¿Qué es una arista hacia atrás en DFS?*

Solución:
Es una arista (u, v) donde v es ancestro de u en el árbol DFS. Indica la presencia de un ciclo.

Ejercicio 0.0.37. *¿Puede un grafo no dirigido tener aristas cruzadas en DFS?*

Solución:
No. En grafos no dirigidos, solo hay aristas de árbol y hacia atrás.

Ejercicio 0.0.38. *¿Cuál es la utilidad del orden topológico?*

Solución:
Permite ordenar tareas con dependencias, resolver sistemas de ecuaciones, optimizar compilación.

Ejercicio 0.0.39. *¿Qué sucede si aplicamos orden topológico a un grafo con ciclos?*

Solución:
No es posible obtener un orden topológico porque existiría una dependencia circular.

Ejercicio 0.0.40. *En BFS, ¿cómo se pueden encontrar las distancias mínimas?*

Solución:
Se almacena $d[v]$, que es la distancia desde la fuente. Se actualiza cuando se descubre un vértice blanco.

Ejercicio 0.0.41. *En DFS, ¿para qué sirven los tiempos de descubrimiento y finalización?*

Solución:
Permiten clasificar aristas, detectar ciclos, y son útiles para orden topológico y componentes fuertemente conexas.

Ejercicios Propuestos (Ampliados)

1. Represente el grafo con vértices $\{A, B, C, D, E\}$ y aristas (A, B), (A, C), (B, D), (C, D), (D, E) mediante matriz y listas.
2. Ejecute BFS desde A en el grafo anterior.
3. Ejecute DFS en el mismo grafo.
4. Determine si el grafo con aristas $(1, 2)$, $(2, 3)$, $(3, 4)$, $(4, 1)$ tiene ciclos.
5. Ordene topológicamente: $(1, 2)$, $(1, 3)$, $(2, 4)$, $(3, 5)$, $(4, 6)$, $(5, 6)$.
6. ¿Qué diferencia hay entre BFS y DFS en términos de orden de visita?
7. Demuestre que BFS calcula la distancia mínima.
8. ¿Cómo se detectan componentes conexas con BFS?
9. ¿Cómo se detectan componentes fuertemente conexas?
10. Implemente BFS en C.
11. Implemente DFS recursivo y no recursivo.
12. ¿Qué es un grafo bipartito y cómo se detecta con BFS?
13. ¿Cuál es la complejidad de verificar si un grafo es bipartito?
14. ¿Cómo se encuentra el diámetro de un árbol con BFS?
15. ¿Qué es un puente en un grafo?
16. ¿Cómo se pueden encontrar puentes con DFS?
17. ¿Qué es un punto de articulación?
18. ¿Cómo se relaciona DFS con el orden topológico?
19. ¿Qué es un grafo acíclico dirigido (DAG)?
20. Investigue aplicaciones de grafos en redes sociales.

Unidad XI: Estrategias de Diseño de Algoritmos (Parte I)

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

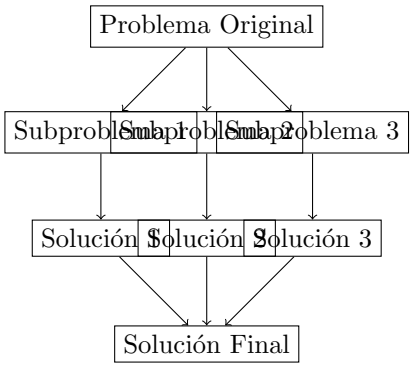
- Comprender el paradigma de divide y vencerás y su aplicación en problemas de ordenamiento.
- Analizar recurrencias utilizando el Teorema Maestro.
- Implementar y analizar Merge Sort y Quick Sort.
- Comprender el concepto de algoritmos greedy y la propiedad de elección local óptima.
- Implementar y analizar el algoritmo de Huffman para compresión de datos.
- Implementar y analizar los algoritmos de Kruskal y Prim para árboles de cubrimiento mínimo.

Parte I: Divide y Vencerás (Divide and Conquer)

Definición y Filosofía

El paradigma de **divide y vencerás** consiste en tres pasos fundamentales:

1. **Dividir:** Descomponer el problema en subproblemas más pequeños, independientes y de la misma naturaleza.
2. **Conquistar:** Resolver los subproblemas recursivamente (si son suficientemente pequeños, resolver directamente).
3. **Combinar:** Integrar las soluciones de los subproblemas para obtener la solución del problema original.



Análisis de Recurrencias

Forma General

La recurrencia típica de divide y vencerás es:

$$T(n) = aT(n/b) + f(n)$$

donde:

- a: número de subproblemas
- n/b: tamaño de cada subproblema
- $f(n)$: costo de dividir y combinar

Teorema Maestro

Teorema 0.0.16 (Teorema Maestro)**.** *Sean $a \geq 1$, $b > 1$ constantes, $f(n)$ una función, y $T(n)$ definida por:*

$$T(n) = aT(n/b) + f(n)$$

Entonces:

1. *Si $f(n) = O(n^{\log_b a - \epsilon})$ para algún $\epsilon > 0$, entonces $T(n) = \Theta(n^{\log_b a})$.*
2. *Si $f(n) = \Theta(n^{\log_b a} \log^k n)$ para algún $k \geq 0$, entonces $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$.*
3. *Si $f(n) = \Omega(n^{\log_b a + \epsilon})$ para algún $\epsilon > 0$, y si $af(n/b) \leq cf(n)$ para algún $c < 1$ y n suficientemente grande, entonces $T(n) = \Theta(f(n))$.*

Merge Sort

Descripción

Merge Sort es un algoritmo de ordenamiento basado en divide y vencerás que garantiza $O(n \log n)$ en el peor caso.

Pseudocódigo

```
MERGE-SORT(A, p, r)
    if p < r
        q = floor((p + r) / 2)
        MERGE-SORT(A, p, q)
        MERGE-SORT(A, q+1, r)
        MERGE(A, p, q, r)

MERGE(A, p, q, r)
    n1 = q - p + 1
    n2 = r - q
    L[1..n1+1], R[1..n2+1]
    for i = 1 to n1
        L[i] = A[p + i - 1]
    for j = 1 to n2
        R[j] = A[q + j]
    L[n1+1] = INFINITY
    R[n2+1] = INFINITY
    i = 1, j = 1
    for k = p to r
        if L[i] <= R[j]
            A[k] = L[i]
            i = i + 1
        else
            A[k] = R[j]
            j = j + 1
```

Listing 17: Merge Sort

Ejemplo Visual

```
[38, 27, 43, 3, 9, 82, 10]
        /           \
[38, 27, 43, 3]   [9, 82, 10]
    /      \          /    \
[38,27]  [43,3]    [9,82]  [10]
  / \      / \       / \     |
[38][27] [43][3]   [9][82] [10]
  \ /      \ /       \ /     |
[27,38]  [3,43]    [9,82]  [10]
    \      /          \    /
 [3,27,38,43]       [9,10,82]
        \               /
   [3,9,10,27,38,43,82]
```

Análisis de Complejidad

- Recurrencia: $T(n) = 2T(n/2) + \Theta(n)$
- Por Teorema Maestro: $\log_b a = \log_2 2 = 1$, $f(n) = \Theta(n) = \Theta(n^1)$
- Caso 2 con $k = 0$: $T(n) = \Theta(n \log n)$

Quick Sort

Descripción

Quick Sort es un algoritmo de ordenamiento que elige un pivote y particiona el arreglo alrededor de él. Su rendimiento promedio es $O(n \log n)$ pero puede ser $O(n^2)$ en el peor caso.

Pseudocódigo

```
QUICKSORT(A, p, r)
    if p < r
        q = PARTITION(A, p, r)
        QUICKSORT(A, p, q-1)
        QUICKSORT(A, q+1, r)

PARTITION(A, p, r)
    x = A[r]                // pivote
    i = p - 1
    for j = p to r-1
        if A[j] <= x
            i = i + 1
            swap(A[i], A[j])
    swap(A[i+1], A[r])
    return i + 1
```

Listing 18: Quick Sort

Ejemplo Visual

```
[5, 3, 8, 4, 2, 7, 1, 6]  (pivote = 6)
[5, 3, 4, 2, 1, 6, 8, 7]  (después de partición)
      /            \
[5, 3, 4, 2, 1]   [8, 7]
      |               |
[1, 3, 4, 2, 5]   [7, 8]
      /    \
[1, 3, 4, 2]  []
      |
[1, 2, 3, 4]
```

Variantes de Quick Sort

Quick Sort Aleatorizado

```
RANDOMIZED-PARTITION(A, p, r)
    i = RANDOM(p, r)
    swap(A[r], A[i])
    return PARTITION(A, p, r)
```

Listing 19: Quick Sort Aleatorizado

La aleatorización elimina la posibilidad de que una entrada específica cause el peor caso.

Mediana de Tres Se elige el pivote como la mediana de tres elementos (primero, medio, último) para mejorar el equilibrio.

Análisis de Complejidad

Teorema 0.0.17. *El tiempo esperado de Quick Sort aleatorizado es $O(n \log n)$.*

Esbozo. Sean X_{ij} variables indicadoras de que z_i y z_j se comparan. La probabilidad es $2/(j-i+1)$. Sumando:

$$E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{j-i+1} = O(n \log n)$$

□

Comparación: Merge Sort vs. Quick Sort

Característica	Merge Sort	Quick Sort
Peor caso	$O(n \log n)$	$O(n^2)$
Caso promedio	$O(n \log n)$	$O(n \log n)$
Mejor caso	$O(n \log n)$	$O(n \log n)$
In-place	No	Sí
Estable	Sí	No
Memoria extra	$O(n)$	$O(\log n)$

Parte II: Algoritmos Greedy (Voraces)

Definición y Filosofía

Un **algoritmo greedy** toma la decisión que parece mejor en el momento (elección localmente óptima), esperando que esto conduzca a una solución globalmente óptima.

Propiedades Fundamentales

1. **Propiedad de elección greedy:** Una solución óptima global puede ser alcanzada tomando elecciones localmente óptimas.

2. **Subestructura óptima:** Una solución óptima contiene dentro de sí soluciones óptimas a los subproblemas.

Código de Huffman para Compresión

Descripción

El algoritmo de Huffman construye un código de prefijo óptimo para compresión de datos, asignando códigos más cortos a caracteres más frecuentes.

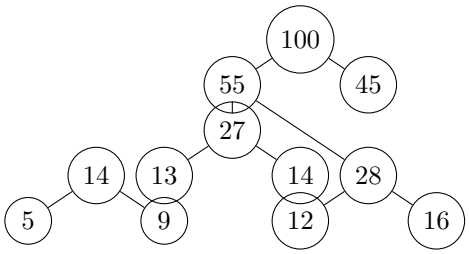
Pseudocódigo

```
HUFFMAN(C)
    n = |C|
    Q = C   // cola de prioridad m nima
    for i = 1 to n-1
        z = new node
        z.left = x = EXTRACT-MIN(Q)
        z.right = y = EXTRACT-MIN(Q)
        z.freq = x.freq + y.freq
        INSERT(Q, z)
    return EXTRACT-MIN(Q)   // ra z del  rbol
```

Listing 20: Algoritmo de Huffman

Ejemplo

Frecuencias: a:45, b:13, c:12, d:16, e:9, f:5



Códigos resultantes:

- a: 0
- b: 101

- c: 100
- d: 111
- e: 1101
- f: 1100

Análisis de Complejidad

- Construir cola de prioridad: $O(n)$
- $n-1$ extracciones e inserciones: $O(n \log n)$
- Total: $O(n \log n)$

Algoritmo de Kruskal para Árbol de Cubrimiento Mínimo

Descripción

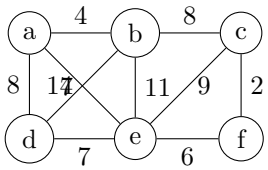
Kruskal construye un MST agregando aristas en orden creciente de peso, siempre que no formen ciclos.

Pseudocódigo

```
KRUSKAL(G, w)
    A = empty set
    for each vertex v in G.V
        MAKE-SET(v)
    sort edges of G.E by weight w
    for each edge (u,v) in sorted order
        if FIND-SET(u) != FIND-SET(v)
            A = A     {(u,v)}
            UNION(u,v)
    return A
```

Listing 21: Algoritmo de Kruskal

Ejemplo



Orden de adición: (c,f) peso 2, (a,b) peso 4, (d,e) peso 7, (a,e) peso 7, (b,c) peso 8, (e,f) peso 6? (e,f) no porque forma ciclo.

Análisis de Complejidad

- Ordenar aristas: $O(E \log E)$
- Operaciones de conjuntos disjuntos: $O(E\alpha(V))$
- Total: $O(E \log E) = O(E \log V)$

Algoritmo de Prim para Árbol de Cubrimiento Mínimo

Descripción

Prim construye un MST creciendo un árbol desde un vértice inicial, agregando siempre la arista de menor peso que conecta el árbol con un vértice externo.

Pseudocódigo

```
PRIM(G, w, r)
    for each u in G.V
        u.key = INFINITY
        u.parent = NIL
    r.key = 0
    Q = G.V   // cola de prioridad m nima
    while Q not empty
        u = EXTRACT-MIN(Q)
        for each v in G.adj[u]
            if v in Q and w(u,v) < v.key
                v.parent = u
                v.key = w(u,v)
                DECREASE-KEY(Q, v, v.key)
```

Listing 22: Algoritmo de Prim

Ejemplo

Mismo grafo, comenzando desde a:

1. a: key=0, visitar a: actualizar b(4), d(8), e(7)
2. Extraer b(4): visitar b: actualizar c(8), e(7? menor que 7? no)
3. Extraer e(7): visitar e: actualizar d(7? igual), f(6)
4. Extraer f(6): visitar f: actualizar c(2)
5. Extraer c(2): visitar c: fin
6. Extraer d(7): fin

Análisis de Complejidad

- Con montículo binario: $O(E \log V)$
- Con montículo de Fibonacci: $O(E + V \log V)$

Comparación: Kruskal vs. Prim

Característica	Kruskal	Prim
Estructura	Basado en aristas	Basado en vértices
Complejidad	$O(E \log E)$	$O(E \log V)$
Adecuado para	Grafos dispersos	Grafos densos
Estructura de datos	Conjuntos disjuntos	Cola de prioridad

Ejercicios Resueltos (Ampliados)

Ejercicio 0.0.42. *Resuelva la recurrencia $T(n) = 3T(n/4) + n^2$ usando el Teorema Maestro.*

Solución:
$a = 3$, $b = 4$, $\log_b a = \log_4 3 \approx 0.792$, $f(n) = n^2 = \Omega(n^{0.792+1.2})$. Verificar regularidad: $3(n/4)^2 = 3n^2/16 \leq cn^2$ con $c = 3/16 < 1$. Caso 3: $T(n) = \Theta(n^2)$.

Ejercicio 0.0.43. *Resuelva $T(n) = 4T(n/2) + n^2$.*

Solución:
$a = 4$, $b = 2$, $\log_b a = 2$, $f(n) = n^2 = \Theta(n^2)$. Caso 2 con $k = 0$: $T(n) = \Theta(n^2 \log n)$.

Ejercicio 0.0.44. *Aplique Merge Sort a* $[3, 41, 52, 26, 38, 57, 9, 49]$.

Solución:

1. Dividir: $[3, 41, 52, 26]$ y $[38, 57, 9, 49]$
2. Dividir: $[3, 41]$, $[52, 26]$, $[38, 57]$, $[9, 49]$
3. Dividir hasta un elemento, luego combinar ordenadamente.

Resultado: $[3, 9, 26, 38, 41, 49, 52, 57]$.

Ejercicio 0.0.45. *Aplique Quick Sort a* $[5, 3, 8, 4, 2, 7, 1, 6]$ *con pivote último.*

Solución:

1. Pivote=6: partición $[5, 3, 4, 2, 1, 6, 8, 7]$
2. Izquierdo $[5, 3, 4, 2, 1]$: pivote=1 $\rightarrow$ $[1, 3, 4, 2, 5]$
3. Derecho $[8, 7]$: pivote=7 $\rightarrow$ $[7, 8]$

4. Continuar recursivamente.

Ejercicio 0.0.46. *Construya el código de Huffman para frecuencias: a:10, b:15, c:12, d:3, e:4, f:13.*

Solución:

1. Combinar d(3) y e(4) $\rightarrow$ 7
2. Combinar 7 y c(12) $\rightarrow$ 19? No, combinar a(10) y 7 $\rightarrow$ 17
3. Combinar b(15) y f(13) $\rightarrow$ 28
4. Combinar 17 y c(12) $\rightarrow$ 29
5. Combinar 28 y 29 $\rightarrow$ 57

Ejercicio 0.0.47. *Aplique Kruskal al grafo de 6 vértices con aristas: (a,b,4), (a,e,7), (b,c,8), (b,e,11), (c,f,2), (d,e,7), (d,b,14), (e,f,6).*

Solución:
Orden: (c,f,2), (a,b,4), (a,e,7), (d,e,7), (e,f,6), (b,c,8), (b,e,11), (d,b,14). Se seleccionan: (c,f), (a,b), (a,e), (d,e), (e,f)? (e,f) forma ciclo con (c,f) y (a,e)? No, (e,f) con (c,f) y (a,e) forma ciclo? (a,e)-(e,f)-(c,f)-(a,b) no conecta, (a,e)-(e,f)-(c,f) no cierra. En realidad (e,f) forma ciclo con (a,e)-(a,b)-(b,c)-(c,f)? Sí, hay camino. Se selecciona (b,c,8) en su lugar.

Ejercicio 0.0.48. *Aplique Prim al mismo grafo comenzando desde a.*

Solución:

1. a: visitar b(4), e(7)
2. Extraer b(4): visitar c(8), e(7? igual)
3. Extraer e(7): visitar d(7), f(6)
4. Extraer f(6): visitar c(2)
5. Extraer c(2): visitar? fin
6. Extraer d(7): fin

Aristas seleccionadas: (a,b), (a,e), (e,f), (f,c), (e,d).

Ejercicio 0.0.49. *Demuestre que Merge Sort es estable.*

Solución:
En MERGE, cuando $L[i] = R[j]$, se toma de L primero, manteniendo el orden original.

Ejercicio 0.0.50. *¿Por qué Quick Sort puede tener peor caso $O(n^2)$?*

Solución:
Cuando el pivote es siempre el mínimo o máximo, una partición queda vacía y la otra con $n-1$ elementos, resultando $T(n) = T(n-1) + O(n) = O(n^2)$.

Ejercicio 0.0.51. *¿Cuál es la propiedad de elección greedy en Huffman?*

Solución:
Elegir los dos caracteres de menor frecuencia para fusionarlos primero lleva a un código óptimo.

Ejercicio 0.0.52. *¿Qué sucede si en Kruskal se eligen aristas en orden aleatorio?*

Solución:
El resultado no sería necesariamente el MST, ya que podría seleccionar aristas de mayor peso antes.

Ejercicio 0.0.53. *¿Cómo se implementa la cola de prioridad en Prim con montículo binario?*

Solución:
Se usa EXTRACT-MIN para obtener el vértice con menor clave, y DECREASE-KEY para actualizar claves de vecinos.

Ejercicios Propuestos (Ampliados)

1. Resuelva $T(n) = 9T(n/3) + n$.
2. Resuelva $T(n) = 2T(n/2) + n \log n$.
3. Resuelva $T(n) = 8T(n/2) + n^3$.
4. Implemente Merge Sort en C.
5. Implemente Quick Sort con pivote aleatorio.
6. Compare el rendimiento de Merge Sort y Quick Sort en arreglos ordenados y aleatorios.
7. Construya el código de Huffman para "algoritmos y estructuras de datos".
8. Aplique Kruskal al grafo con vértices A,B,C,D,E y aristas: (A,B,1), (A,C,4), (B,C,2), (B,D,5), (C,D,3), (C,E,6), (D,E,7).
9. Aplique Prim al mismo grafo.

10. Demuestre que el algoritmo de Huffman produce un código de prefijo óptimo.
11. ¿Qué es un código de prefijo y por qué es importante?
12. ¿Cómo se detectan ciclos en Kruskal?
13. ¿Qué ventaja tiene Prim sobre Kruskal en grafos densos?
14. ¿Qué es la "mediana de tres" en Quick Sort?
15. Demuestre que el tiempo esperado de Quick Sort aleatorizado es $O(n \log n)$.
16. ¿Cuál es la complejidad espacial de Merge Sort? ¿Y de Quick Sort?
17. ¿Qué es la propiedad de subestructura óptima en algoritmos greedy?
18. ¿Cómo se puede mejorar Quick Sort para evitar el peor caso?
19. ¿Qué es un árbol de cubrimiento mínimo y por qué es importante?
20. Investigue aplicaciones de Huffman en compresión de archivos (ZIP, JPEG).

Unidad XII: Recursos Complementarios y Apoyo Académico

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante tendrá acceso a:

- Una bibliografía completa y actualizada sobre algoritmos y estructuras de datos.
- Canales de YouTube de calidad para profundizar en los temas.
- Profesores particulares especializados en la materia.
- Información de contacto para apoyo académico personalizado.

Bibliografía Fundamental

Bibliografía Principal

- **Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009).** *Introduction to Algorithms* (3rd ed.). MIT Press.
  - **Comentario:** El "CLRS" es la biblia de los algoritmos. Cubre desde fundamentos hasta temas avanzados con rigor matemático. Es la referencia principal de este tratado.
  - **Capítulos clave:** 1-5 (Fundamentos), 6-9 (Ordenamiento), 10-14 (Estructuras de datos), 15-17 (Técnicas de diseño), 22-26 (Grafos).
- **Sedgewick, R., & Wayne, K. (2011).** *Algorithms* (4th ed.). Addison-Wesley.
  - **Comentario:** Excelente complemento al CLRS con un enfoque más práctico e implementaciones en Java. Incluye numerosos ejemplos y visualizaciones.

- **Capítulos clave:** 1 (Fundamentos), 2 (Ordenamiento), 3 (Búsqueda), 4 (Grafos), 5 (Cadenas).

- **Knuth, D. E. (1997-1998).** *The Art of Computer Programming* (Volúmenes 1-3, 3rd ed.). Addison-Wesley.
  - **Comentario:** La obra magna de Knuth. Profundidad sin igual en análisis de algoritmos. Recomendado para estudiantes avanzados que quieran comprender los fundamentos matemáticos.
  - **Volumen 1:** Algoritmos fundamentales.
  - **Volumen 2:** Matemáticas seminuméricas.
  - **Volumen 3:** Ordenamiento y búsqueda.

Bibliografía Complementaria

- **Kleinberg, J., & Tardos, É. (2006).** *Algorithm Design.* Addison-Wesley.
  - **Comentario:** Excelente para diseño de algoritmos y problemas de optimización. Enfoque en la construcción de soluciones.
- **Skiena, S. S. (2020).** *The Algorithm Design Manual* (3rd ed.). Springer.
  - **Comentario:** Muy práctico, incluye un catálogo de problemas y cómo resolverlos con las estructuras adecuadas.
- **Dasgupta, S., Papadimitriou, C. H., & Vazirani, U. V. (2008).** *Algorithms.* McGraw-Hill.
  - **Comentario:** Texto conciso y elegante. Buen equilibrio entre teoría y práctica.
- **Aho, A. V., Hopcroft, J. E., & Ullman, J. D. (1983).** *Data Structures and Algorithms.* Addison-Wesley.
  - **Comentario:** Clásico que sentó las bases de la enseñanza de algoritmos. Aún vigente en muchos conceptos fundamentales.

Bibliografía para Profundización

- **Tarjan, R. E. (1983).** *Data Structures and Network Algorithms.* SIAM.
  - **Comentario:** Imprescindible para grafos y estructuras de datos avanzadas. Del creador de los árboles splay y el análisis amortizado.
- **Okasaki, C. (1998).** *Purely Functional Data Structures.* Cambridge University Press.
  - **Comentario:** Para quienes quieran explorar estructuras de datos en programación funcional.

- **Mehlhorn, K., & Sanders, P. (2008).** *Algorithms and Data Structures: The Basic Toolbox*. Springer.
  - **Comentario:** Enfoque moderno con implementaciones eficientes en C++.

Canales de YouTube Recomendados

Canales en Español

- **Mitos y Leyendas de la Programación (Andrés)**
  - **URL:** youtube.com/c/MitosyLeyendasdelaProgramacion
  - **Contenido:** Excelentes explicaciones de algoritmos y estructuras de datos en español. Enfoque didáctico y claro.
  - **Recomendado para:** Algoritmos de ordenamiento, grafos, programación dinámica.
- **Programación ATS**
  - **URL:** youtube.com/c/ProgramacionATS
  - **Contenido:** Tutoriales prácticos de estructuras de datos en C y Java.
  - **Recomendado para:** Implementaciones prácticas de pilas, colas, listas, árboles.
- **Deusto Ciencia**
  - **URL:** youtube.com/c/DeustoCiencia
  - **Contenido:** Explicaciones rigurosas de algoritmos y complejidad computacional.
  - **Recomendado para:** Notación asintótica, recurrencias, teoría de la computación.
- **Algoritmos y Programación (UBA)**
  - **URL:** youtube.com/c/AlgoritmosyProgramacionUBA
  - **Contenido:** Material de las cátedras de la UBA, incluyendo Algo2 y Algo3.
  - **Recomendado para:** Seguir la cursada con material oficial.

Canales en Inglés

- **freeCodeCamp**
  - **URL:** youtube.com/c/freecodecamp

- **Contenido:** Cursos completos de algoritmos y estructuras de datos. Excelente producción.
- **Recomendado para:** Cursos intensivos de varias horas sobre temas específicos.

- **Abdul Bari**
  - **URL:** youtube.com/c/AbdulBari
  - **Contenido:** Explicaciones claras y detalladas de algoritmos. Muy popular en la comunidad.
  - **Recomendado para:** Algoritmos de grafos, programación dinámica, árboles.
- **Michael Sambol**
  - **URL:** youtube.com/c/MichaelSambol
  - **Contenido:** Visualizaciones animadas de algoritmos. Breves y claras.
  - **Recomendado para:** Entender visualmente cómo funcionan los algoritmos.
- **CS Dojo**
  - **URL:** youtube.com/c/CSDojo
  - **Contenido:** Explicaciones amigables de conceptos de programación y algoritmos.
  - **Recomendado para:** Introducción a estructuras de datos.
- **Reducible**
  - **URL:** youtube.com/c/Reducible
  - **Contenido:** Animaciones de alta calidad explicando algoritmos complejos.
  - **Recomendado para:** Algoritmos de grafos, teoría de la complejidad.
- **Back To Back SWE**
  - **URL:** youtube.com/c/BackToBackSWE
  - **Contenido:** Enfoque en entrevistas técnicas y problemas de algoritmos.
  - **Recomendado para:** Preparación para entrevistas y problemas específicos.

Canales de Divulgación Matemática

- **3Blue1Brown**
  - **URL:** youtube.com/c/3blue1brown
  - **Contenido:** Visualizaciones matemáticas impresionantes. Útil para entender análisis asintótico y complejidad.
- **Numberphile**
  - **URL:** youtube.com/c/numberphile
  - **Contenido:** Matemáticas recreativas. Útil para entender números de Fibonacci, primos y teoría de números.

Profesores Particulares Especializados

Profesores Destacados

- **Nahuel Villafañe (@clasesconnahu)**
  - **Especialidad:** Fisica y Matematicas (UBA Exactas).
  - **Experiencia:** Estudiante avanzado de Fisica en la UBA.
  - **Modalidad:** Clases particulares presenciales (CABA) y virtuales por Zoom/Meet.
  - **Contacto:**
    - Instagram: `@clasesconnahu`
  - **Promoción:** Primera clase de diagnóstico gratuita. Material incluido.
- **Martín Diéguez (@martindieguez)**
  - **Especialidad:** Algoritmos y Programación (UBA).
  - **Experiencia:** Docente de Algoritmos y Programación en la UBA. Amplia experiencia en la enseñanza de fundamentos de programación y estructuras de datos.
  - **Contacto:** Instagram `@martindieguez`
- **Facundo Palavecino**
  - **Especialidad:** Matemática discreta, Algoritmos y Complejidad.
  - **Experiencia:** Ayudante en materias de matemática de Exactas. Especialista en demostraciones y fundamentos teóricos.
  - **Contacto:** Disponible por consulta en el Departamento de Computación.

Plataformas de Tutorías

- **Clases Particulares UBA**
  - **URL:** clasesparticularesuba.com.ar
  - **Contenido:** Buscador de profesores particulares especializados en materias de Exactas.
- **Superprof Argentina**
  - **URL:** superprof.com.ar
  - **Contenido:** Plataforma de tutores con reseñas y perfiles detallados.

Recursos Adicionales

Sitios Web Educativos

- **VisuAlgo**
  - **URL:** visualgo.net
  - **Contenido:** Visualizaciones interactivas de algoritmos. Excelente para entender el funcionamiento paso a paso.
- **GeeksforGeeks**
  - **URL:** geeksforgeeks.org
  - **Contenido:** Artículos, implementaciones y ejercicios de algoritmos. Muy completo.
- **LeetCode**
  - **URL:** leetcode.com
  - **Contenido:** Plataforma de práctica con problemas de algoritmos. Ideal para preparación de entrevistas.
- **HackerRank**
  - **URL:** hackerrank.com
  - **Contenido:** Desafíos de algoritmos con sistemas de evaluación automática.
- **Coursera**
  - **URL:** coursera.org
  - **Contenido:** Cursos de algoritmos de universidades como Princeton, Stanford y MIT.

Grupos de Estudio y Comunidades

- **Discord: Algoritmos UBA**
  - **Contenido:** Comunidad de estudiantes de Exactas para consultas y estudio colaborativo.
  - **Link:** Disponible por invitación en los grupos de la materia.
- **Telegram: Algo2 - Algo3 UBA**
  - **Contenido:** Grupos de discusión de las materias, con material compartido y consultas.
- **Reddit: r/algorithms**
  - **URL:** reddit.com/r/algorithms
  - **Contenido:** Discusión global sobre algoritmos y estructuras de datos.
- **Stack Overflow**
  - **URL:** stackoverflow.com
  - **Contenido:** Consultas específicas sobre implementaciones y conceptos.

Consejos para el Estudio de Algoritmos

Estrategias Efectivas

1. **Entender antes de implementar:** Lee la descripción del algoritmo, analiza su invariante y comprende por qué funciona antes de escribir código.
2. **Visualizar:** Usa herramientas como VisuAlgo o dibuja manualmente la ejecución paso a paso.
3. **Implementar desde cero:** Evita copiar código; escribilo vos mismo para internalizar los detalles.
4. **Analizar complejidad:** Para cada algoritmo que implementes, calcula su complejidad temporal y espacial.
5. **Resolver ejercicios variados:** No te quedes solo con los ejemplos del libro. Practica con problemas de LeetCode, HackerRank o parciales viejos.
6. **Estudiar en grupo:** Explicar un algoritmo a otros es la mejor forma de asegurarte de que lo entendiste.
7. **Repasar demostraciones:** En Algo2 y Algo3, las demostraciones de corrección son parte fundamental de la evaluación.

Preparación para Parciales y Finales

- **Parciales:** Enfocarse en ejercicios de implementación y análisis de complejidad. Practicar con parciales viejos de la cátedra.
- **Finales:** Preparar demostraciones de corrección de los algoritmos principales (MST, Dijkstra, Floyd-Warshall, etc.). Conocer los teoremas fundamentales (Teorema Maestro, propiedades de hashing, etc.).
- **Recursos:** Consultar las guías de ejercicios de la materia y los apuntes de cátedra disponibles en el Departamento de Computación.

Sobre el Autor del Tratado

[colback=blue!5!white, colframe=blue!75!black, title=**Nahuel Villafañe**]
Estudiante avanzado de Ciencias de la Computación

Universidad de Buenos Aires (UBA) - Facultad de Ciencias Exactas y Naturales
Experiencia:

- Estudiante de Algoritmos y Estructuras de Datos II y III
- Ayudante ad-honorem en materias del Departamento de Computación
- Desarrollador de material didáctico para estudiantes de Exactas
- Autor de este tratado: *"Tratado de Algoritmos y Datos II"*

Clases Particulares:

- Especialidad: Fisica y Matematicas

Contacto:

- Instagram: `@clasesconnahu`

Epílogo: El Camino del Algoritmo

Querido estudiante,

Has completado este tratado, pero tu camino en el mundo de los algoritmos apenas comienza. Los conceptos que has aprendido —desde el análisis asintótico hasta los árboles de cubrimiento mínimo— son las herramientas fundamentales que te permitirán resolver problemas cada vez más complejos.

Recuerda que un buen programador no es quien sabe más algoritmos de memoria, sino quien sabe identificar cuál es el algoritmo adecuado para cada problema y adaptarlo a las necesidades específicas. La práctica constante, la

curiosidad intelectual y la humildad para aprender de los errores son tus mejores aliados.

Este tratado fue escrito con la esperanza de acompañarte en ese camino, ofreciéndote no solo conocimiento, sino también la confianza para enfrentar los desafíos que vendrán en Algoritmos II, Algoritmos III y más allá.

Que el análisis de complejidad te acompañe, que tus invariantes de bucle sean siempre correctos, y que tus árboles nunca se desbalanceen.

Con cariño,

Nahuel Villafañe

Buenos Aires, 2026

Figura 1: Enter Caption